WNDRVR
SUPPORT NETWORK

API REFERENCE: KERNEL STORAGE, SR0650

Copyright Notice

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

Corporate Headquarters

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

API Reference: Kernel Storage, SR0650

20 July 2020

1. BIO

Name

bio - buffer IO Implementation

Layer

BDM

Routines

bioInit( ) - initialize the bio library **bio_done( )** - terminates a bio operation **bio_alloc( )** - allocate memory blocks **bio_cacheable_alloc( )** - allocate cache able memory blocks **bio_free( )** - free the bio memory **bio_cacheable_free( )** - free the bio cache able memory

Description

This library implements the buffer IO (BIO) library.

Include files

drv/xbd/bio.h, **drv/xbd/xbd.h**

Name

bioInit( ) - initialize the bio library

Layer

BDM

Synopsis

```
STATUS bioInit (void)
```

Description

none

Returns

OK

Errno

Not Available

See also

bio

Name

bio_done( ) - terminates a bio operation

Layer

BDM

Synopsis

```
void bio_done
    (
    struct bio * pBio,   /* pointer to bio structure */
    int          error   /* error code */
    )
```

Description

none

Returns

N/A

Errno

Not Available

See also

bio

Name

bio_alloc( ) - allocate memory blocks

Layer

BDM

Synopsis

```
void * bio_alloc
    (
    device_t xbd,        /* XBD for which to allocate */
    int      numBlocks   /* number of blocks to allocate */
    )
```

Description

This routine allocates numBlocks of memory. The size of the block is extracted from the xbd.

Returns

pointer to the allocated memory, **NULL** on error

Errno

Not Available

See also

bio

Name

bio_cacheable_alloc( ) - allocate cache able memory blocks

Layer

BDM

Synopsis

```
void * bio_cacheable_alloc
    (
    device_t xbd,        /* XBD for which to allocate */
    int      numBlocks   /* number of blocks to allocate */
    )
```

Description

This routine allocates numBlocks of cached memory. The size of the block is extracted from the xbd.

Returns

pointer to the allocated memory, **NULL** on error

Errno

Not Available

See also

bio

Name

bio_free( ) - free the bio memory

Layer

BDM

Synopsis

```
void bio_free
    (
    void * pBioData   /* pointer to data to free */
    )
```

Description

This routine frees the memory in a bio structure. Note that bio_data is NOT a pointer to the the bio structure, but rather the pointer to the memory to free.

Returns

N/A

Errno

Not Available

See also

bio

Name

bio_cacheable_free( ) - free the bio cache able memory

Layer

BDM

Synopsis

```
void bio_cacheable_free
    (
    void * pBioData   /* pointer to data to free */
    )
```

Description

This routine frees the memory in a bio structure. Note that bio_data is NOT a pointer to the the bio structure, but rather the pointer to the memory to free.

Returns

N/A

Errno

Not Available

See also

bio

2. CDROMFSLIB

Name

cdromFsLib - ISO 9660 CD-ROM read-only file system library

Layer

FS_CDROMFS

Routines

cdromFsInit( ) - initialize the VxWorks CD-ROM file system **cdromFsVolConfigShow( )** - show the volume configuration information **cdromFsVersionDisplay( )** - display the cdromFs version number **cdromFsVersionNumGet( )** - return the cdromFs version number **cdromFsDevDelete( )** - delete a CD-ROM filesystem (cdromFs) I/O device **cdromFsDevCreate( )** - create a CD-ROM filesystem (cdromFs) I/O device.

Description

This library implements the VxWorks CD-ROM file system (cdromFs). This file system permits the usage of standard POSIX I/O calls, e.g. **open( )**, **read( )**, **ioctl( )**, and **close( )**, to read data from a CD-ROM, CD-R, or CD-RW disk formatted according to the ISO 9660 specification. This library supports multiple devices, concurrent access from multiple tasks, and multiple open files.

The component **INCLUDE_CDROMFS** must be configured into the system to obtain the VxWorks CD-ROM file system (cdromFs).

This file system provides access to CD file systems using any standard eXtended Block Device (XBD). Note that the old-style block device (**BLK_DEV**) is no longer directly supported by **cdromFsLib**.

The creation and deletion of individual instances of cdromFs file systems are typically handled by the file system monitor (FSM). The file system monitor component (**INCLUDE_FS_MONITOR**) is automatically included in the system when **INCLUDE_CDROMFS** is included.

The underlying XBD driver will raise an **insertion** event when a CD-ROM device is connected to the target, via a USB port for example, or when media is present in the tray. The file system monitor will handle the insertion event and invoke the appropriate cdromFs create routine to instantiate a cdromFs on the device.

Likewise, when the CD-ROM device is disconnected from the target, or when the media is removed, the XBD driver will raise a **removal** event. This event is handled directly by **cdromFsLib**, and will result in the deletion of the associated cdromFs.

As an example, assume that a target has a CD-ROM drive connected as the master device on the primary ATA controller. In order to instantiate a CD-ROM file system (cdromFs) on this device named "/cdrom", the **INCLUDE_ATA** component should be included in the system, and the **FS_NAMES_ATA_PRIMARY_MASTER** configuration parameter should be set to "/cdrom".

Manual instantiation of cd-rom file systems

The following steps can be followed to "manually" instantiate a CD-ROM file system. For example, this procedure will be required if a custom XBD driver doesn't support the raising of **insertion** and **removal** events as described above.

1. Create an XXX block device (XBD) on the physical device:

```
    device_t xxxXbd = xxxXbdDevCreate (...);
```

2. Create a CD-ROM file system (cdromFs) on the XBD device:

```
    CDROM_VOL_DESC_ID cdVolDescId = cdromFsDevCreate ("/cdrom", xxxXbd);
```

This will result in the creation of the "/cdrom" I/O device, as shown by the output of the **devs** target shell command:

```
-> devs
drv name
  0 /null
  1 /tyCo/0
  2 /aioPipe/0x61723b98
  6 /vio
  7 /tgtsvr
  4 /cdrom
value = 0 = 0x0
```

The CD-ROM file system can be subsequently deleted by the following step.

1. Delete the CD-ROM file system (cdromFs)

```
    STATUS status = cdromFsDevDelete (cdVolDescId);
```

In the event that an old-style block device driver (**BLK_DEV**) is being utilized, the "XBD Block Device Wrapper" can be used to translate the **BLK_DEV** interface to an XBD interface. The component **INCLUDE_XBD_BLK_DEV** must be included in a system to obtain the "XBD Block Device Wrapper". The following steps can be followed to "manually" instantiate a CD-ROM using an old-style block device driver.

1. Create an XXX block device (**BLK_DEV**) on the physical device:

```
    BLK_DEV * xxxBlkDevId = xxxBlkDevCreate (...);
```

2. Associate an I/O device name with a block device name

```
   fsmNameInstall ("xxxDisk0:0", "/cdrom");
```

3. Create an XBD wrapper device around the **BLK_DEV** device:

```
    device_t xxxXbd = xbdBlkDevCreate (xxxBlkDevId, "xxxDisk0");
```

The **xbdBlkDevCreate( )** invocation will result in the raising of an **insertion** event, i.e. an explicit **cdromFsDevCreate( )** is not required.

Iso 9660 file and directory naming

The strict ISO 9660 specification allows only uppercase file names consisting of 8 characters plus a 3 character suffix.

To accommodate users familiar with MS-DOS, **cdromFsLib** lets you use lowercase name arguments to access files with names consisting entirely of uppercase characters. Mixed-case file and directory names are accessible only if you specify their exact case-correct names.

Joliet extensions file and directory naming

The Joliet extensions to the ISO 9660 specification are designed to handle long file names up to 340 characters long.

File names must be case correct. The above use of lowercase characters to access files named entirely with uppercase characters is not supported.

The Joliet extensions to **cdromFsLib** do support Unicode file names. Filenames and other identifiers are encoded in 16 bit Unicode (UCS-2) on the media. These are converted to UTF-8 by **cdromFsLib** before being passed back "up" to the I/O system, or before being compared with a string passed "down" from the I/O system.

File and directory naming common to iso 9660 and the joliet extensions

To support multiple versions of the same file, the ISO 9660 specification also supports version numbers. When specifying a file name in an **open( )** call, you can select the file version by appending the file name with a semicolon (;) followed by a decimal number indicating the file version. If you omit the version number, **cdromFsLib** opens the latest version of the file.

For the time being, **cdromFsLib** further accommodates MS-DOS users by allowing "\" (backslash) instead of "/" in pathnames. However, the use of the backslash is discouraged because it may not be supported in future versions of **cdromFsLib**.

Finally, **cdromFsLib** uses an 8-bit clean implementation of ISO 9660. Thus, **cdromFsLib** is compatible with CDs using either Latin or Asian characters in the file names.

Ioctl codes supported

FIOGETNAME

Returns the file name for a specific file descriptor.

FIOLABELGET

Retrieves the volume label. This code can be used to verify that a particular volume has been inserted into the drive.

FIOWHERE

Determines the current file position, returns value range from 0 to (**UINT_MAX** - 1), **UINT_MAX**(-1) reserved for status **ERROR**.

FIOWHERE64

Determines the current file position. This is the 64 bit version.

```
long long position = 0;
status = ioctl (fd, FIOWHERE64, (_Vx_ioctl_arg_t) & position);
```

FIOSEEK

Changes the current file position ranges from 0 to **UINT_MAX**, use FIOSEEK64 for setting larger offset, negative offset would cause unexpected result.

```
status = ioctl (fd, FIOSEEK, newOffset);
```

FIOSEEK64

Changes the current file position. This is the 64 bit version.

```
long long newOffset = ...;
status = ioctl (fd, FIOSEEK64, (_Vx_ioctl_arg_t)&newOffset);
```

FIONREAD

Tells you the number of bytes between the current location and the end of this file, range from 0 to **UINT_MAX**.

```
unsigned int unreadCount = 0;
status = ioctl (fd, FIONREAD, (_Vx_ioctl_arg_t)&unreadCount);
```

FIONREAD64

Tells you the number of bytes between the current location and the end of this file. This is the 64 bit version.

```
long long unreadCount = 0;
status = ioctl (fd, FIONREAD64, (_Vx_ioctl_arg_t)&unreadCount);
```

FIOREADDIR

Reads the next directory entry.

FIOUNMOUNT

Announces that the a disk has been removed (all currently open file descriptors are invalidated).

FIOFSTATGET

Gets the file status information (directory entry data).

CDROMFS_DIR_MODE_SET

This is part of the Joliet extensions. It sets the directory mode to the **ioctl( )** arg value. That controls whether a file is opened with or without the Joliet extensions. Settings **MODE_ISO9660**, **MODE_JOLIET**, and **MODE_AUTO** do not use Joliet, use Joliet, or try opening the directory first without Joliet and then with Joliet, respectively.

FIOFSTATFSGET

Return file system status information. The information contains file system type id, block size, total blocks, and file system id.

```
struct statfs data;
status = ioctl (fd, FIOFSTATFSGET, (_Vx_ioctl_arg_t)&data);
```

FIOFSTATFSGET64

Return file system status information. The information contains file system type id, block size, total blocks, and file system id. This is the 64 bit version of FIOFSTATFSGET.

```
struct statfs64 data;
status = ioctl (fd, FIOFSTATFSGET64, (_Vx_ioctl_arg_t)&data);
```

This **ioctl( )** unmounts the file system. Thus any open file descriptors are marked obsolete.

CDROMFS_DIR_MODE_GET

This is part of the Joliet extensions. It gets and returns the directory mode set by **CDROMFS_DIR_MODE_SET**.

CDROMFS_STRIP_SEMICOLON

This sets the **readdir( )** strip semicolon setting to **FALSE** if arg is 0, and **TRUE** otherwise. If **TRUE**, **readdir( )** removes the semicolon and following version number from the directory entries returned.

CDROMFS_GET_VOL_DESC

This returns the primary or supplementary volume descriptor by which the volume is mounted in arg. arg must be type **T_ISO_PVD_SVD_ID** as defined in **cdromFsLib.h**. The result is the volume descriptor adjusted for the endianness of the processor, not the raw volume descriptor from the CD. The result is directly usable by the processor. The result also includes some information not in the volume descriptor, for example which volume descriptor is in-use.

Caveats

The VxWorks CD-ROM file system does not support CD sets containing multiple disks.

Include files

cdromFsLib.h

See also

ioLib, ISO 9660 Specification, Joliet extension Specification

Name

cdromFsInit( ) - initialize the VxWorks CD-ROM file system

Layer

FS_CDROMFS

Synopsis

```
STATUS cdromFsInit
    (
    UINT32 commonBufferSize   /* common buffer size */
    )
```

Description

This routine initializes the VxWorks CD-ROM file system. It is automatically called when the **INCLUDE_CDROMFS** component is configured into the system.

Returns

OK or **ERROR**, if driver can not be installed.

Errno

S_iosLib_DRIVER_GLUT

See also

cdromFsLib, **cdromFsDevCreate( )**, **iosLib.h**

Name

cdromFsVolConfigShow( ) - show the volume configuration information

Layer

FS_CDROMFS

Synopsis

```
VOID cdromFsVolConfigShow
    (
```

```
    void * arg   /* device name or CDROM_VOL_DESC * */
    )
```

Description

This routine retrieves the volume configuration for the named **cdromFsLib** device and prints it to standard output. The information displayed is retrieved from the **BLK_DEV** structure for the specified device.

Returns

N/A

Errno

Not Available

See also

cdromFsLib, N/A

Name

cdromFsVersionDisplay( ) - display the cdromFs version number

Layer

FS_CDROMFS

Synopsis

```
void cdromFsVersionDisplay
    (
    int level   /* level of display, not used */
    )
```

Description

This routine displays the cdromFs version number. This routine has been deprecated.

Returns

N/A

Errno

Not Available

See also

cdromFsLib, **cdromFsVersionNumGet( )**, **cdromFsVolConfigShow( )**

Name

cdromFsVersionNumGet( ) - return the cdromFs version number

Layer

FS_CDROMFS

Synopsis

```
uint32_t cdromFsVersionNumGet
    (
    void
    )
```

Description

This routine returns the cdromFs version number. This routine has been deprecated.

Returns

the cdromFs version number.

Errno

Not Available

See also

cdromFsLib, **cdromFsVersionDisplay( )**

Name

cdromFsDevDelete( ) - delete a CD-ROM filesystem (cdromFs) I/O device

Layer

FS_CDROMFS

Synopsis

```
STATUS cdromFsDevDelete
    (
    CDROM_VOL_DESC_ID pVolDesc   /* ptr to CDROM_VOL_DESC */
    )
```

Description

This routine deletes the specified volume. This involves removing the "device" from the I/O system, and freeing all resources associated with the volume.

Returns

OK if specified volume was successfully deleted, otherwise **ERROR**

Errno

Not Available

See also

cdromFsLib, **cdromFsInit( )**, **cdromFsDevCreate( )**

Name

cdromFsDevCreate( ) - create a CD-ROM filesystem (cdromFs) I/O device.

Layer

FS_CDROMFS

Synopsis

```
CDROM_VOL_DESC_ID cdromFsDevCreate
    (
    char *   devName,    /* device name */
    device_t device      /* underlying block device handle */
    )
```

Description

This routine creates an instance of a cdromFs device in the I/O system. As input, this function requires an eXtended Block Device (XBD) identifier (device_t) for the CD drive on which to create a cdromFs I/O device. Thus, **xxxXbdDevCreate( )**, for example, should have already been called to create the XBD device. Alternatively, **xxxBlkDevCreate( )**, for example, can be called to create a legacy **BLK_DEV** driver followed by **xbdBlkDevCreate( )** to create an XBD wrapper around the **BLK_DEV** device.

Returns

CDROM_VOL_DESC_ID, or **NULL** if error.

Errno

S_memLib_NOT_ENOUGH_MEMORY

See also

cdromFsLib, **cdromFsInit( )**

3. DOSFSCACHELIB

Name

dosFsCacheLib - MS-DOS media-compatible Cache library

Layer

FS_DOSFS

Routines

dosFsCacheLibInit( ) - initialize dosFsCache library **dosFsDefaultDataDirCacheSizeGet( )** - get the default data and dir cache size **dosFsDefaultFatCacheSizeGet( )** - get the default FAT cache size **dosFsDefaultCacheSizeSet( )** - set the default disk cache size **dosFsCacheCreate( )** - create cache for a DosFS volume **dosFsCacheDelete( )** - delete the disk cache for a dosFs volume **dosFsCacheTune( )** - tune a cache's settings **dosFsCacheInfo( )** - retrieve a cache's settings

Description

This library implements a disk cache mechanism for the VxWorks MS-DOS compatible file system.

Disk cache is created on a per volume basis. This cache is used as a read/write cache for the File Allocation Table, directory entries, and data blocks.

To have dosFs cache support on a VxWorks image, the component **INCLUDE_DOSFS_CACHE** must be included. Also, the parameters **DOSFS_DEFAULT_DATA_CACHE_SIZE**, **DOSFS_DEFAULT_DIR_CACHE_SIZE** and **DOSFS_DEFAULT_FAT_CACHE_SIZE** should be set to appropriate values. Note that the values of these parameters will be used when automatically creating a disk cache for all the dosFs volumes in the system.

This automatic cache creation is done by the file system monitor (please refer to the FS monitor documentation for details), either at boot time or whenever a storage device with a MS-DOS compatible file system is detected, like for example when a FAT formatted floppy disk or USB device is inserted and accessed for the first time. If this behavior is not desired, the **DOSFS_DEFAULT_DATA_CACHE_SIZE**, **DOSFS_DEFAULT_DIR_CACHE_SIZE** and **DOSFS_DEFAULT_FAT_CACHE_SIZE** parameters can then be set to zero, or alternatively the cache for a specific volume can be removed using the API **dosFsCacheDelete( )**, and a disk cache for every dosFs instantiation can be created manually using the API **dosFsCacheCreate( )**. The value of this parameter can be get/set at runtime using the APIs **dosFsDefaultDataCacheSizeGet( )**, **dosFsDefaultDirCacheSizeGet( )**, **dosFsDefaultFatCacheSizeGet( )**.

The **dosFsCacheShow( )** routine gives a description of an specific cache in terms of its size, its current allocation status, and its hit/miss ratio.

Disk cache algorithm

The disk cache is composed internally of a number cache blocks, of the same size as the disk physical block (sector). These cache blocks are maintained in a list in "Most Recently Used" order, that is, blocks which are used are moved to the top of this list. When a block needs to be relinquished, and made available to contain a new disk block, the Least Recently Used block will be used for this purpose.

Include files

dosFsLib.h

See also

VxWorks 7 File Systems and I/O Programmer's Guide

WNDRVR

Name

dosFsCacheLibInit( ) - initialize dosFsCache library

Layer

FS_DOSFS

Synopsis

```
void dosFsCacheLibInit
    (
    u_int defaultDataDirCacheSize,
    u_int defaultFatCacheSize,
    BOOL  backgrndFlushTskEnable
    )
```

Description

none

Returns

N/A.

/NOMANUAL

Errno

Not Available

See also

dosFsCacheLib

Name

dosFsDefaultDataDirCacheSizeGet( ) - get the default data and dir cache size

Layer

FS_DOSFS

Synopsis

```
u_int dosFsDefaultDataDirCacheSizeGet
    (
    void
    )
```

Description

This routine gets the default data and dir cache size.

Returns

value of the default data and dir cache size.

Errno

Not Available

See also

dosFsCacheLib

Name

dosFsDefaultFatCacheSizeGet( ) - get the default FAT cache size

Layer

FS_DOSFS

Synopsis

```
u_int dosFsDefaultFatCacheSizeGet
    (
    void
    )
```

Description

This routine gets the default FAT cache size.

Returns

value of the default FAT cache size.

Errno

Not Available

See also

dosFsCacheLib

Name

dosFsDefaultCacheSizeSet( ) - set the default disk cache size

Layer

FS_DOSFS

Synopsis

```
void dosFsDefaultCacheSizeSet
    (
    u_int newDataDirDefaultSize,   /* new default size for dosFs DATA/DIR cache */
    u_int newFatDefaultSize        /* new default size for dosFs FAT cache */
    )
```

Description

This routine sets the default disk cache size to be used for the next dosFs instantiations.

Returns

N/A

Errno

Not Available

See also

dosFsCacheLib

Name

dosFsCacheCreate( ) - create cache for a DosFS volume

Layer

FS_DOSFS

Synopsis

```
STATUS dosFsCacheCreate
    (
    char * volName,
    char * dataDirCacheAddr,
    u_int  dataDirCacheSize,
    char * fatCacheAddr,
    u_int  fatCacheSize
    )
```

Description

This routine will create data and dir cache, fat cache.

volName is a **NULL** terminated character string identifying the name of the DosFS device. dataDirCacheAddr is data and dir cache memory address.If **NULL**, there will malloc. dataDirCacheSize is size of data and dir cache. fatCacheAddr is fat cache memory address.If **NULL**, there will malloc. fatCacheSize is size of fat cache.

Returns

OK on success, **ERROR** otherwise

Errno

Not Available

See also

dosFsCacheLib

Name

dosFsCacheDelete( ) - delete the disk cache for a dosFs volume

Layer

FS_DOSFS

Synopsis

```
STATUS dosFsCacheDelete
    (
    const char * volName
    )
```

Description

This routine removes the disk cache for volName and frees the allocated memory if it was requested from the system memory pool.

volName is pointer to dosFs volume name.

Returns

STATUS.

Errno

Not Available

See also

dosFsCacheLib

Name

dosFsCacheTune( ) - tune a cache's settings

Layer

FS_DOSFS

Synopsis

```
STATUS dosFsCacheTune
    (
    char * volName,                    /* Name of the DosFS volume */
    u_int  clustsPerDataDirCacheGroup,   /* Clusts in one Data and Dir cache group */
    u_int  blocksPerFatCacheGroup      /* Blocks in one fat cache group */
    )
```

Description

This routine allows the user to re-set the block group size for the Data/Dir cache and the Fat cache so as to get better read/write performance for various hardware environments.

The Data/Dir cache or the Fat cache is divided into multiple of block groups. And the concept of "block group" can be depicted by the below figure:

---------- ---------- ----|block grp|----|block grp|---- ... ---------- ---------- Data/Dir cache or Fat cache

The default block group size for Data/Dir cache and Fat cache are defined by **DOSFS_BLOCKS_PER_DATA_DIR_CACHE_GROUP** and **DOSFS_BLOCKS_PER_FAT_CACHE_GROUP**, but they mightnot suit all cases. By the tuning, the Data/Dir cache and Fat cache will be re-created with the newly supplied parameters. This way, the user can adapt the cache to his specific hardware.

volName is a **NULL** terminated character string identifying the name of the DosFS device. ClustsPerDataDirCacheGroup is used to specify how many _clusters_ forming a block group in Data/Dir cache. BlocksPerFatCacheGroup is used to specify how many _sectors_ forming a block group in Fat cache.

Note, ClustsPerDataDirCacheGroup and BlocksPerFatCacheGroup specify the size of a block group in different unit for Data/Dir cache and Fat cache: one is in clusters and the other in sectors.

Returns

OK on success, **ERROR** otherwise

Errno

Not Available

See also

dosFsCacheLib

Name

dosFsCacheInfo( ) - retrieve a cache's settings

Layer

FS_DOSFS

Synopsis

```
STATUS dosFsCacheInfo
    (
    char * volName,                    /* Name of the DosFS volume */
    UINT * pClustsPerDataDirCacheGroup,   /* Clusts in one data and dir cache group */
    UINT * pBlocksPerFatCacheGroup     /* Blocks in one fat cache group */
    )
```

Description

This routine allows the user to retrieve the cache's **data and dir group size** or **fat grop size** parameters.

volName is a **NULL** terminated character string identifying the name of the DosFS device. pClustsPerDataDirCacheGroup points to the clusts in one data and dir cache group. pBlocksPerFatCacheGroup points to the blocks in one fat cache group.

Returns

OK on success, **ERROR** otherwise

Errno

Not Available

See also

dosFsCacheLib

4. DOSFSFMTLIB

Name

dosFsFmtLib - MS-DOS media-compatible file system formatting library

Layer

FS_DOSFS

Routines

dosFsVolFormat( ) - format an MS-DOS compatible volume **dosFsVolFormatWithLabel( )** - format an MS-DOS compatible volume with volume label **dosFsVolFormatFd( )** - format an MS-DOS compatible volume via an opened FD

Description

This module is a scaleable companion module for **dosFsLib**, and is intended to facilitate high level formatting of disk volumes.

Calling **dosFsVolFormat( )** routine allows complete control over the format used, parameters and allows one to supply a hook routine which could, for instance, interactively prompt the user to modify disk parameters.

Availability

This routine is an optional part of the MS-DOS file system, and may be included in a target system if it is required to be able to format new volumes.

In order to include this option, the following function needs to be invoked during system initialization:

```
void dosFsFmtLibInit( void );
```

See reference page **dosFsVolFormat( )** for complete description of supported formats, options and arguments.

Include files

none

See also

dosFsLib, VxWorks 7 File Systems and I/O Programmer's Guide, [Overview of dosFs](https://docs.windriver.com/bundle/vxworks_7_file_systems_and_IO_programmers_guide_sr0650/page/eri1450462381714.html

Name

dosFsVolFormat( ) - format an MS-DOS compatible volume

Layer

FS_DOSFS

Synopsis

```
STATUS dosFsVolFormat
    (
    const char * path,                /* path for volume to format */
    int          opt,                 /* bit-wise or'ed options */
    FORMAT_PTR   _func_FormatPrompt
                                      /* interactive parameter change callback */
    )
```

Description

This utility routine performs the initialization of file system data structures on a disk. It supports FAT12 for small disks, FAT16 for medium size and FAT32 for large volumes. The device argument is a device name known to the I/O system.

The opt argument is a bit-wise or'ed combination of options controlling the operation of this routine as follows:

DOS_OPT_DEFAULT

If the current volume boot block is reasonably intact, use existing parameters, else calculate parameters based only on disk size, possibly reusing only the volume label and serial number.

DOS_OPT_PRESERVE

Attempt to preserve the current volume parameters even if they seem to be somewhat unreliable.

DOS_OPT_BLANK

Disregard the current volume parameters, and calculate new parameters based only on disk size.

DOS_OPT_QUIET

Do not produce any diagnostic output during formatting.

DOS_OPT_FAT16

Format the volume with FAT16. Valid on volumes up to 2 Gbytes big. For larger ones, FAT32 must be used.

DOS_OPT_FAT32

Format the volume with FAT32, even if the disk is smaller than 2 Gbytes, but is larger then 512 Mbytes.

DOS_OPT_VXLONGNAMES

Note that this option is deprecated. Calling dosFsVolFormat with this option will result in an error.

DOS_OPT_FSTYPE

Format the volume with file system type and calculate new parameters according to file system type, which must be set along with **DOS_OPT_FAT16** or **DOS_OPT_FAT32**, otherwise format fail.

The third argument, pPromptFunc is an optional pointer to a function that may interactively prompt the user to change any of the modifiable volume parameters before formatting:

```
void formatPromptFunc( DOS_VOL_CONFIG *pConfig );
```

The *pConfig structure upon entry to **formatPromptFunc( )** will contain the initial volume parameters, some of which can be changed before it returns. pPromptFunc should be **NULL** if no interactive prompting is required.

Compatibility

Although this routine tries to format the disk to be compatible with Microsoft implementations of the FAT and FAT32 file systems, there may be differences which are not under WRS control. For this reason, it is highly recommended that any disks which are expected to be interchanged between vxWorks and Windows should be formatted under Windows to provide the best interchangeability. The WRS implementation is more flexible, and should be able to handle the differences when formatting is done on Windows, but Windows implementations may not be able to handle minor differences between their implementation and ours.

Availability

This function is an optional part of the MS-DOS file system, and may be included in a target system if it is required to be able to format new volumes.

Returns

OK or **ERROR** if was unable to format the disk.

Errno

Not Available

See also

dosFsFmtLib

Name

dosFsVolFormatWithLabel( ) - format an MS-DOS compatible volume with volume label

Layer

FS_DOSFS

Synopsis

```
STATUS dosFsVolFormatWithLabel
    (
    const char * path,                /* path for volume to format */
    int          opt,                 /* bit-wise or'ed options */
    const char * labelName,           /* volume label */
    FORMAT_PTR   _func_FormatPrompt   /* interactive parameter change callback */
    )
```

Description

This utility routine is the implementation of **dosFsVolFormat( )**. It has one more parameter than **dosFsVolFormat( )** that allows user to set the volume label when formatting disk.

The labelName argument is limit to 11 characters.

Returns

OK or **ERROR** if was unable to format the disk.

Errno

Not Available

See also

dosFsFmtLib

Name

dosFsVolFormatFd( ) - format an MS-DOS compatible volume via an opened FD

Layer

FS_DOSFS

Synopsis

```
STATUS dosFsVolFormatFd
    (
    int        fd,                 /* rawFs file descriptor */
    int        opt,                /* bit-wise or'ed options */
    FORMAT_PTR func_FormatPrompt
                                   /* interactive parameter change callback */
    )
```

Description

Refer to **dosFsVolFormat( )**'s documentation. It should also be noted that the file descriptor parameter will be closed regardless of the routine's outcome.

Returns

OK or **ERROR** if was unable to format the disk.

Errno

Not Available

See also

dosFsFmtLib

5. DOSFSLIB

Name

dosFsLib - MS-DOS media-compatible file system library

Layer

FS_DOSFS

Routines

dosfsHostToDisk32( ) - convert uint32_t from host to on-disk format **dosfsHostToDisk16( )** - convert uint16_t from host to on-disk format **dosfsDiskToHost32( )** - convert uint32_t from on-disk to host format **dosfsDiskToHost16( )** - convert uint16_t from on-disk to host format **dosFsVolumeOptionsSet( )** - set this volume's disk options **dosFsVolumeOptionsGet( )** - get this volume's disk options **dosSetVolCaseSens( )** - set case sensitivity of volume **dosFsVolDescGet( )** - convert a device name into a DOS volume descriptor pointer. **dosFsVolUnmount( )** - unmount a dosFs volume **dosFsChkDsk( )** - make volume integrity checking. **dosFsVolIsFat12( )** - determine if a MSDOS volume is FAT12 or FAT16 **dosFsFdFree( )** - free a file descriptor **dosFsFdGet( )** - get an available file descriptor **dosPathParse( )** - parse a full pathname into an array of names. **dosFsOpen( )** - open a file on a dosFs volume **dosFsClose( )** - close a dosFs file **dosFsIoctl( )** - do device specific control function **dosFsLastAccessDateEnable( )** - enable last access date updating for this volume **dosFsLibInit( )** - prepare to use the dosFs library **dosFsDevCreate( )** - create file system device. **dosFsDevDelete( )** - delete a dosFs volume **dosFsMonitorDevCreate( )** - create a dosFs volume through the fs monitor **dosFsDiskProbe( )** - probe if a device contains a valid dosFs **dosFsHdlrInstall( )** - install handler. **dosFsXbdBlkRead( )** - read blocks from the underlying XBD block device. **dosFsXbdBlkWrite( )** - write blocks to the underlying XBD block device. **dosFsXbdBytesRW( )** - read/write bytes to/from the underlying XBD block device. **dosFsXbdBlkCopy( )** - copy blocks on the underlying XBD block device. **dosFsXbdIoctl( )** - Misc control operations **dosFsDiscard( )** - discard operations

Description

This library implements the MS-DOS compatible file system. This is a multi-module library, which depends on sub-modules to perform certain parts of the file system functionality. A number of different file system format variations are supported.

Using this library

The various routines provided by the VxWorks DOS file system (dosFs) may be separated into three broad groups: general initialization, device initialization, and file system operation.

The **dosFsLibInit( )** routine is the principal initialization function. It is called once by the system initialization when support for dosFs is included, regardless of how many dosFs devices are to be used.

You use a different dosFs routine for device initialization. For each dosFs device, **dosFsDevCreate( )** must be called to install the device in VxWorks device list. When using partitioned disks, **dosFsDevCreate( )** must be called for each anticipated partition. This will associate it with a logical device name, so that it can be accessed later via the I/O system. The job of instantiating file systems is done automatically by the File System Monitor module, either at boot time or whenever removable media is inserted in the system (such as a floppy disk or a USB device). Please refer to the File System Monitor documentation for further details.

In case of removable media, device access and file system instantiation will be done only when the logical device is first accessed by the application.

More detailed information on all of these routines is provided below.

File attributes

Directory entries on dosFs volumes contain an attribute byte consisting of bit-flags that specify various characteristics of the entry. The attributes which are identified are: read-only file, hidden file, system file, volume label, directory, and archive. The VxWorks symbols for these attribute bit-flags are:

DOS_ATTR_RDONLY

File is write-protected, cannot be modified or deleted.

DOS_ATTR_HIDDEN

this attribute is not used by VxWorks.

DOS_ATTR_SYSTEM

this attribute is not used by VxWorks.

DOS_ATTR_VOL_LABEL

directory entry describes a volume label, this attribute cannot be set or used directly, see **ioctl( )** command FIOLABELGET and FIOLABELSET below for volume label manipulation.

DOS_ATTR_DIRECTORY

directory entry is a subdirectory, this attribute cannot be set directly.

DOS_ATTR_ARCHIVE

this attribute is not used by VxWorks.

All the flags in the attribute byte, except the directory and volume label flags, may be set or cleared using the **ioctl( )** FIOATTRIBSET function. This function is called after opening the specific file whose attributes you want to change. The attribute byte value specified in the FIOATTRIBSET call is copied directly. To preserve existing flag settings, the current attributes should first be determined via **fstat( )**, and the appropriate flag(s) changed using bitwise AND/OR operations. For example, to make a file read-only, while leaving other attributes intact:

```
    struct stat fileStat;

    fd = open ("file", O_RDONLY, 0);     /* open file          */
    fstat (fd, &fileStat);               /* get file status    */

    ioctl (fd, FIOATTRIBSET, (fileStat.st_attrib | DOS_ATTR_RDONLY));
                                         /* set read-only flag */
    close (fd);                          /* close file         */
```

See also the reference manual entry for **attrib( )** and **xattrib( )** for user-level utility routines which control the attributes of files or file hierarchy.

Ioctl functions

The dosFs file system supports the following **ioctl( )** functions. The functions listed are defined in the header **ioLib.h**. Unless stated otherwise, the file descriptor used for these functions may be any file descriptor that is opened to a file or directory on the volume, or to the volume itself. There are some **ioctl( )** commands, that expect a 32-bit integer result (FIONFREE, FIOWHERE, etc.). Disks and files that are greater than 4GB are supported. To solve this problem, new **ioctl( )** functions have been added to support 64-bit integer results. They have the same name as basic functions, but with suffix 64, namely: FIONFREE64, FIOWHERE64 and so on. These commands expect a pointer to a 64-bit integer, i.e.:

```
long long *arg ;
```

as the 3rd argument to the **ioctl( )** function. If using a 32-bit **ioctl( )** command to request a value too large to be represented in the 32-bit variable, **ioctl( )** will return **ERROR**, and errno will be set to **EOVERFLOW**.

FIOUNMOUNT

Unmounts a disk volume. It performs the same function as **dosFsVolUnmount( )**. This function must not be called from interrupt level:

```
status = ioctl (fd, FIOUNMOUNT, 0);
```

FIOGETNAME

Gets the file name of the file descriptor and copies it to the buffer nameBuf. Note that nameBuf must be large enough to contain the largest possible path name.

```
status = ioctl (fd, FIOGETNAME, &nameBuf);
```

FIORENAME

Renames the file or directory to the string newname:

```
fd = open("oldname", O_RDONLY, 0);
status = ioctl (fd, FIORENAME, "newname");
```

FIOUPDATE

Updates the dosFs create options to the new value newoptions

```
int newOptions;
status = ioctl (fd, FIOUPDATE, newOptions);
```

FIOMOVE

Moves the file or directory to the string newname:

```
fd = open("oldname", O_RDONLY, 0);
status = ioctl (fd, FIOMOVE, "newname");
```

FIOSEEK

Sets the current byte offset in the file to the position specified by newOffset. This function supports offsets range from 0 to **UINT_MAX**. Use FIOSEEK64 for larger offset:

```
status = ioctl (fd, FIOSEEK, newOffset);
```

FIOSEEK64

Sets the current byte offset in the file to the position specified by newOffset. This function supports offsets in 64-bit value range:

```
long long   newOffset = ...;
status = ioctl (fd, FIOSEEK64, (_Vx_ioctl_arg_t) & newOffset);
```

FIOWHERE

Returns the current byte position in the file. This is the byte offset of the next byte to be read or written. This function returns a 32-bit value range from 0 to **UINT_MAX** - 1, and returns **ERROR**(-1) on failure. It takes no additional argument:

```
position = ioctl (fd, FIOWHERE, 0);
```

FIOWHERE64

Returns the current byte position in the file. This is the byte offset of the next byte to be read or written. This function returns a 64-bit value in position:

```
long long position = 0;
status = ioctl (fd, FIOWHERE64, (_Vx_ioctl_arg_t) & position);
```

FIOFLUSH

Flushes disk cache buffers. It guarantees that any output that has been requested is actually written to the device:

```
status = ioctl (fd, FIOFLUSH, 0);
```

FIOSYNC

Updates the FAT copy for the passed file descriptor, then flushes and invalidates the dosFs cache buffers for the file descriptor's volume. FIOSYNC ensures that any outstanding output requests for the passed file descriptor are written to the device and a subsequent I/O operation will fetch data directly from the physical medium. To safely sync a volume for shutdown, all open file descriptor's should at the least be FIOSYNC'd by the application. Optimally, the application should close all open FD's should and the volume should be unmounted via FIOUNMOUNT.

```
status = ioctl (fd, FIOSYNC, 0);
```

FIOTRUNC

Sets the specified file's length to newLength bytes, supports 32-bit length range from 0 to **UINT_MAX**. Any disk clusters that had been allocated to the file but are now unused are deallocated while additional clusters are zeroed, and the directory entry for the file is updated to reflect the new length. Only regular files may be truncated; attempts to use FIOTRUNC on directories will return an error.

```
status = ioctl (fd, FIOTRUNC, newLength);
```

FIOTRUNC64

Similar to FIOTRUNC, but can be used for files larger, than 4GB.

```
long long newLength = .....;
status = ioctl (fd, FIOTRUNC, (_Vx_ioctl_arg_t) & newLength);
```

FIONREAD

Copies to unreadCount the number of unread bytes in the file, if the number of unread bytes may exceed **UINT_MAX**, use FIONREAD64 instead:

```
unsigned int unreadCount = 0;
status = ioctl (fd, FIONREAD, (_Vx_ioctl_arg_t)&unreadCount);
```

FIONREAD64

Copies to unreadCount the number of unread bytes in the file. This function returns a 64-bit integer value:

```
long long unreadCount = 0;
status = ioctl (fd, FIONREAD64, (_Vx_ioctl_arg_t)&unreadCount);
```

FIONFREE

Copies to freeCount the amount of free space, in bytes, on the volume:

```
unsigned int freeCount = 0;
status = ioctl (fd, FIONFREE, (_Vx_ioctl_arg_t)&freeCount);
```

FIONFREE64

Copies to freeCount the amount of free space, in bytes, on the volume. This function can return value in 64-bit range:

```
long long freeCount = 0;
status = ioctl (fd, FIONFREE64, &freeCount);
```

FIOMKDIR

Creates a new directory with the name specified as dirName:

```
status = ioctl (fd, FIOMKDIR, "dirName");
```

FIORMDIR

Removes the directory whose name is specified as dirName:

```
status = ioctl (fd, FIORMDIR, "dirName");
```

FIOLABELGET

Gets the volume label (located in root directory) and copies the string to labelBuffer. If the label contains **DOS_VOL_LABEL_LEN** significant characters, resulting string is not **NULL** terminated:

```
char    labelBuffer [DOS_VOL_LABEL_LEN];
status = ioctl (fd, FIOLABELGET, (_Vx_ioctl_arg_t)labelBuffer);
```

FIOLABELSET

Sets the volume label to the string specified as newLabel. The string may consist of up to eleven ASCII characters:

```
status = ioctl (fd, FIOLABELSET, (_Vx_ioctl_arg_t)"newLabel");
```

FIOATTRIBSET

Sets the file attribute byte in the DOS directory entry to the new value newAttrib. The file descriptor refers to the file whose entry is to be modified:

```
status = ioctl (fd, FIOATTRIBSET, newAttrib);
```

FIOCONTIG

Allocates contiguous disk space for a file or directory. The number of bytes of requested space is specified in bytesRequested, it should be neither larger than **UINT_MAX**, nor negative values. Generally, contiguous space should be allocated immediately after the file is created:

```
status = ioctl (fd, FIOCONTIG, bytesRequested);
```

FIOCONTIG64

Allocates contiguous disk space for a file or directory. The number of bytes of requested space is specified in bytesRequested. Generally, contiguous space should be allocated immediately after the file is created. This function accepts a 64-bit value:

```
long long bytesRequested = 0;
status = ioctl (fd, FIOCONTIG64, &bytesRequested);
```

FIONCONTIG

Copies to maxContigBytes the size of the largest contiguous free space, in bytes, on the volume:

```
unsigned int maxContigBytes = 0;
status = ioctl (fd, FIONCONTIG, &maxContigBytes);
```

FIONCONTIG64

Copies to maxContigBytes the size of the largest contiguous free space, in bytes, on the volume. This function returns a 64-bit value:

```
long long maxContigBytes = 0;
status = ioctl (fd, FIONCONTIG64, &maxContigBytes);
```

FIOFSTATFSGET

Returns file system status information. The information contains file system type id, block size, total blocks, free blocks, and file system id.

```
struct statfs data;
status = ioctl (fd, FIOFSTATFSGET, (_Vx_ioctl_arg_t)&data);
```

FIOFSTATFSGET64

Returns file system status information. The information contains file system type id, block size, total blocks, free blocks, and file system id. This is the 64-bit version of FIOFSTATFSGET.

```
struct statfs64 data;
status = ioctl (fd, FIOFSTATFSGET64, (_Vx_ioctl_arg_t)&data);
```

FIOREADDIR

Reads the next directory entry. The argument dirStruct is a DIR directory descriptor. Normally, the **readdir( )** routine is used to read a directory, rather than using the FIOREADDIR function directly. See **dirLib**.

```
DIR dirStruct;
fd = open ("directory", O_RDONLY);
status = ioctl (fd, FIOREADDIR, &dirStruct);
```

FIOFSTATGET

Gets file status information (directory entry data). The argument statStruct is a pointer to a stat structure that is filled with data describing the specified file. Normally, the **stat( )** or **fstat( )** routine is used to obtain file information, rather than using the FIOFSTATGET function directly. See **dirLib**.

```
struct stat statStruct;
fd = open ("file", O_RDONLY);
status = ioctl (fd, FIOFSTATGET, (int)&statStruct);
```

FIOTIMESET

Update time on a file. arg shall be a pointer to a utimbuf structure, see **utime.h**. If arg is value **NULL**, the current system time is used for both actime and modtime members. If arg is not **NULL** then the utimbuf structure members actime and modtime are used as passed. If actime is zero value, the file access time is not updated (the operation is ignored). If modtime is zero, the file modification time is not updated (the operation is ignored). See also **utime( )**

```
struct utimbuf newTimeBuf;;
newTimeBuf.modtime = newTimeBuf.actime = fileNewTime;
fd = open ("file", O_RDONLY);
status = ioctl (fd, FIOTIMESET, (_Vx_ioctl_arg_t)&newTimeBuf);
```

FIOCHKDSK

This function invokes the integral consistency checking. During the test, the file system will be blocked from application code access, and will emit messages describing any inconsistencies found on the disk, as well as some statistics, depending on the verbosity level in the flags argument. Depending on the repair permission value in flags argument, the

inconsistencies will be repaired, and changes written to disk or only reported. Argument flags should be composed of bitwise or-ed verbosity level value and repair permission value. Possible repair levels are:

DOS_CHK_ONLY (1)

Only report errors, do not modify disk.

DOS_CHK_REPAIR (2)

Repair any errors found.

Possible verbosity levels are:

DOS_CHK_VERB_SILENT (0xff00)

Do not emit any messages, except errors encountered.

DOS_CHK_VERB_1 (0x0100)

Display some volume statistics when done testing, as well

DOS_CHK_VERB_2 (0x0200)

In addition to the above option, display path of every file, while it is being checked. This option may significantly slow down the test process.

NOTE

In environments with reduced RAM size check disk uses reserved FAT copy as temporary buffer, it can cause respectively long time of execution on a slow CPU architectures..

See also the reference manual **usrFsLib** for the **chkdsk( )** user level utility which may be used to invoke the FIOCHKDSK **ioctl( )**. The volume root directory should be opened, and the resulting file descriptor should be used:

```
int fd = open (device_name, O_RDONLY, 0);
status = ioctl (fd, FIOCHKDSK, DOS_CHK_REPAIR | DOS_CHK_VERB_1);
close (fd);
```

Any other **ioctl( )** function codes are passed to the underlying XBD modules for handling.

Caution

dosFs file names are limited to 255 characters, not including the trailing NUL. The total path length cannot exceed 1024 characters, including the trailing NUL. While dosFs supports path names of 1024 characters, using a path name of more than 259 characters makes it incompatible with Windows (that is, the file cannot be used on a Windows host).

If **DOSFS_COMPAT_NT** (a directory entry NT flag compatibility with Windows that only affects disks with the FAT32 format under 2GB in size) is set **TRUE**, then VxWorks will interpret the field in the same way as Windows. Otherwise VxWorks will interpret the directory entry field at offset 0x0c in a incompatible way.

Include files

dosFsLib.h

See also

ioLib, **iosLib**, **dirLib**, **usrFsLib**, **dosFsCacheLib**, **dosFsFmtLib**, **dosChkLib**, Microsoft MS-DOS Programmer's Reference, (Microsoft Press), Advanced MS-DOS Programming, (Ray Duncan, Microsoft Press), VxWorks 7 File Systems and I/O Programmer's Guide:MS-DOS Compatible File System: dosFs

Name

dosfsHostToDisk32( ) - convert uint32_t from host to on-disk format

Layer

FS_DOSFS

Synopsis

```
void dosfsHostToDisk32
    (
    uint32_t  src,
    uint8_t * pDest
    )
```

Description

This routine converts a uint32_t from host's memory to on-disk format.

Returns

N/A

/NOMANUAL

Errno

Not Available

See also

dosFsLib

Name

dosfsHostToDisk16( ) - convert uint16_t from host to on-disk format

Layer

FS_DOSFS

Synopsis

```
void  dosfsHostToDisk16
    (
    uint16_t  src,
    uint8_t * pDest
    )
```

Description

This routine converts a uint16_t from host's memory to on-disk format.

Returns

N/A

/NOMANUAL

Errno

Not Available

See also

dosFsLib

Name

dosfsDiskToHost32( ) - convert uint32_t from on-disk to host format

Layer

FS_DOSFS

Synopsis

```
uint32_t  dosfsDiskToHost32
    (
    uint8_t * pSrc
    )
```

Description

This routine converts a uint32_t from on-disk format to host's endian-ness.

Returns

uint32_t in host's endian-ness

/NOMANUAL

Errno

Not Available

See also

dosFsLib

Name

dosfsDiskToHost16( ) - convert uint16_t from on-disk to host format

Layer

FS_DOSFS

Synopsis

```
uint16_t  dosfsDiskToHost16
    (
    uint8_t * pSrc
    )
```

Description

This routine converts a uint16_t from on-disk format to host's endian-ness.

Returns

uint16_t in hosts's endian-ness

/NOMANUAL

Errno

Not Available

See also

dosFsLib

Name

dosFsVolumeOptionsSet( ) - set this volume's disk options

Layer

FS_DOSFS

Synopsis

```
UINT dosFsVolumeOptionsSet
    (
    char * volName,   /* dosFs volume name */
    UINT   options    /* new options */
    )
```

Description

This routine sets the volume options for the dosFs volume volName. Currently the only option available is **DOS_VOLUME_VOL_NO_DMA**, which means that DosFS does not need to use DMA-safe buffers on volName when large transfers to and from the media are requested. This option may gain performance on large transfers, and is recommended for USB devices. It replaces the routine **dosFsCacheOptionsSet( )**.

volName is a **NULL** terminated character string that corresponds to the name of the desired DosFS volume. Valid settings for options are currently **DOS_VOLUME_VOL_NO_DMA** or 0.

Returns

new value of the volume's options, or **ERROR** on failure.

Errno

Not Available

See also

dosFsLib

Name

dosFsVolumeOptionsGet( ) - get this volume's disk options

Layer

FS_DOSFS

Synopsis

```
UINT dosFsVolumeOptionsGet
    (
    char * volName   /* dosFs volume name */
    )
```

Description

This routine gets the volume options for the dosFs volume volName. It replaces the routine **dosFsCacheOptionsGet( )**.

volName is a **NULL** terminated character string identifying the DosFS volume.

Returns

DOS_VOLUME_VOL_NO_DMA if that option is enabled; 0 if no options are enabled; **ERROR** on failure

Errno

Not Available

See also

dosFsLib

Name

dosSetVolCaseSens( ) - set case sensitivity of volume

Layer

FS_DOSFS

Synopsis

```
STATUS dosSetVolCaseSens
    (
    DOS_VOLUME_DESC_ID pVolDesc,
    BOOL               sensitivity
    )
```

Description

Pass **TRUE** to setup a case sensitive volume. Pass **FALSE** to setup a case insensitive volume. Note this affects rename lookups only. It is invalidation for short name that conform to 8.3 format.

Returns

TRUE if pVolDesc pointed to a DOS volume.

Errno

Not Available

See also

dosFsLib

Name

dosFsVolDescGet( ) - convert a device name into a DOS volume descriptor pointer.

Layer

FS_DOSFS

Synopsis

```
DOS_VOLUME_DESC_ID dosFsVolDescGet
    (
    void *    pDevNameOrPVolDesc,   /* device name or pointer to dos vol desc */
    u_char ** ppTail                /* return ptr for name, used in iosDevFind */
    )
```

Description

This routine validates pDevNameOrPVolDesc to be a DOS volume descriptor pointer else a path to a DOS device. This routine uses the standard **iosLib** function **iosDevFind( )** to obtain a pointer to the device descriptor. If device is eligible, ppTail is filled with the pointer to the first character following the device name. Note that ppTail is passed to **iosDevFind( )**. ppTail may be passed as **NULL**, in which case it is ignored.

Returns

A **DOS_VOLUME_DESC_ID** or **NULL** if not a DOSFS device.

Errno

EINVAL

See also

dosFsLib

Name

dosFsVolUnmount( ) - unmount a dosFs volume

Layer

FS_DOSFS

Synopsis

```
STATUS dosFsVolUnmount
    (
    void * pDevNameOrPVolDesc   /* device name or ptr to */
                                /* volume descriptor */
    )
```

Description

This routine is called when I/O operations on a volume are to be discontinued. This is the preferred action prior to changing a removable disk.

All buffered data for the volume is written to the device (if possible, with no error returned if data cannot be written), any open file descriptors are marked as obsolete, and the volume is marked as not currently mounted.

When a subsequent **open( )** operation is initiated on the device, new volume will be mounted automatically.

Once file descriptors have been marked as obsolete, any attempt to use them for file operations will return an error. (An obsolete file descriptor may be freed by using **close( )**. The call to **close( )** will return an error, but the descriptor will in fact be freed).

This routine may also be invoked by calling **ioctl( )** with the FIOUNMOUNT function code.

This routine must not be called from interrupt level.

Returns

OK, or **ERROR** if the volume was not mounted.

/NOMANUAL

Errno

Not Available

See also

dosFsLib

Name

dosFsChkDsk( ) - make volume integrity checking.

Layer

FS_DOSFS

Synopsis

```
STATUS dosFsChkDsk
    (
    FAST DOS_FILE_DESC_ID pFd,     /* file descriptor of root dir */
    u_int                 params   /* check level and verbosity */
    )
```

Description

This library does not make the integrity check process itself, instead it uses a routine provided by **dosChkLib**. This routine prepares parameters and invokes checking routine via preinitialized function pointer. If **dosChkLib** is not configured into vxWorks, this routine returns **ERROR**.

Ownership on device should be taken by an upper level routine.

Returns

STATUS as returned by volume checking routine or **ERROR**, if such routine is not installed.

Errno

ENOTSUP

See also

dosFsLib

Name

dosFsVolIsFat12( ) - determine if a MSDOS volume is FAT12 or FAT16

Layer

FS_DOSFS

Synopsis

```
int dosFsVolIsFat12
    (
```

```
    u_char * pBootBuf   /* boot parameter block buffer */
    )
```

Description

This routine is the container for the logic which determines if a dosFs volume is using FAT12 or FAT16. Two methods are implemented. Both methods use information from the volumes boot parameter block fields found in the boot sector.

The first FAT determination method follows the recommendations outlined in the Microsoft document:

"Hardware White Paper Designing Hardware for Microsoft Operating Systems FAT: General Overview of On-Disk Format Version 1.02, May 5, 1999 Microsoft Corporation"

This method is used in the hopes that greater compatability with MSDOS formatted media will be achieved. The Microsoft recommended method for FAT type determination between FAT12 and FAT16 is done via the count of clusters on the volume.

The Microsoft recommended approach is as follows:

1.) Determine the count of sectors occupied by the root directory entries for this volume, rounding up:

rootDirSecs = ((rootEntCount * dirEntSz) + (bytesPerSec-1)) / bytesPerSec;

Where dirEntSz is 32 for MSDOS 8.3, and 64 for VXLONGNAMES.

2.) Determine the count of sectors occupied by the volumes data region:

dataRgnSecs = totalSecs - (reservedSecs + (nFats * fatSecs) + rootDirSecs);

3.) determine the count of clusters, rounding down:

countOfClusts = dataSecs / secsPerClust; /* Note: this rounds down. */

Note: countOfClusts represents the count of data clusters, starting at two.

4.) determine the FAT types based on the count of clusters on the volume,

if (countOfClusts < 4085) /* Microsoft recommends using "less than" */ { /* Volume is FAT12 */ } else { /* Volume is FAT16 */ }

An alternate method is used when mounting a known VxWorks DOSFS-1.0 volume. This method is used for greater backward compatability with VxWorks DOSFS-1.0 volumes. See also: SPR#34704. The VxWorks dosFs1 method deviates from the Microsoft currently recommened method.

This is the VxWorks DOSFS1 method per **dosFsVolDescFill( )**, **dosFsLib.c**, dosFs 1.0, revision history: "03l,16mar99,dgp". Using the identical method here will help ensure backward compatablitity when mounting volumes formatted by the VxWorks dosFs1.0 code.

The VxWorks DOSFS 1.0 approach is as follows:

1.) Get starting sector of the root directory:

rootSec = reservedSecs + (nFats * secsPerFat);

2.) Get the size of the root dir in bytes:

rootBytes = (nRootEnts * dirEntSz):

Where dirEntSz is 32 for MSDOS 8.3, and 64 for VXLONGNAMES.

WNDRVR

3.) Get the starting sector of the data area:

dataSec = rootSec + ((rootBytes + bytesPerSec-1) / bytesPerSec);

4.) Get the number of "FAT entries":

countOfClusts = (((totalSecs - dataSecs) / secsPerClust) + **DOS_MIN_CLUST**);

5.) Choose the FAT type based on the count of clusters, note DOSFS1 uses less than or equal here.

if (countOfClusts <= 4085) /* VxDosFs1 uses less than or equal to. */ { /* use FAT12 */ } else { /* use FAT16 */ }

By mimicking the dosFs 1.0 approach, we should be able to mount all dosFs 1.0 volumes correctly. By using the microsoft recommened approach in all other cases, we should be as compatable as possible with Microsoft OS's.

The volumes Boot Parameter Block fields MUST be validated for sanity before this routine is called.

pBootBuf is not verified, DO NOT pass this routine a **NULL** pointer. This routine is also used by **dosFsFmtLib.c**

Returns

TRUE if the FAT type is FAT12, **FALSE** if the FAT type is FAT16, or **ERROR** if the data is invalid.

/NOMANUAL

Errno

Not Available

See also

dosFsLib

Name

dosFsFdFree( ) - free a file descriptor

Layer

FS_DOSFS

Synopsis

```
void dosFsFdFree
    (
    DOS_FILE_DESC_ID pFd
    )
```

Description

This routine marks a file descriptor as free and decreases reference count of a referenced file handle.

Returns

N/A.

/NOMANUAL

Errno

Not Available

See also

dosFsLib

Name

dosFsFdGet( ) - get an available file descriptor

Layer

FS_DOSFS

Synopsis

```
DOS_FILE_DESC_ID dosFsFdGet
    (
    DOS_VOLUME_DESC_ID pVolDesc
    )
```

Description

This routine obtains a free dosFs file descriptor.

Returns

Pointer to file descriptor, or **NULL**, if none available.

Errno

ENFILE /NOMANUAL

See also

dosFsLib

Name

dosPathParse( ) - parse a full pathname into an array of names.

Layer

FS_DOSFS

Synopsis

```
int dosPathParse
    (
    u_char *     path,
    PATH_ARRAY * pnamePtrArray,
    size_t       sizeArray
    )
```

Description

This routine is similar to **pathParse( )**, but on the contrary it does not allocate additional buffers nor changes the path string.

Parses a path in directory tree which has directory names separated by **/** or ''s. It fills the supplied array of structures with pointers to directory and file names and correspondence name length. All occurrences of **//**, **.** and **..** are right removed from path. All tail dots and spaces are broken from each name, that is name like "abc. . ." is treated as just "abc".

For instance, "/usr/vw/data/../dir/file" gets parsed into

```
                        namePtrArray
                       |---------|
  ---------------------------o    |
  |                    |    3    |
  |                    |---------|
  |   -----------------------o    |
  |   |                |    2    |
  |   |                |---------|
  |   |          ------------o    |
  |   |          |     |    3    |
  |   |          |     |---------|
  |   |          |   --------o    |
  |   |          |   |  |    4    |
  |   |          |   |  |---------|
  v   v          v   v  |   NULL  |
  |   |          |   |  |    0    |
  |   |          |   |  |---------|
  v   v          v   v
 |------------------------|
 |usr/vw/data/../dir/file |
 |-------\-----/----------|
         ignored
```

Note that UTF-8 bytes that are not representing ASCII characters **.**, **/**, etc., never compare equal to **.**, **/**, etc., so that no special work is required here for Unicode.

In the future, the "../" erasure trick is likely to vanish. Do not rely on the fact that you can currently access "**nosuchdir/../file.txt**" when "nosuchdir" does not exist.

Returns

number of levels in path.

Errno

ENOENT EINVAL

See also

dosFsLib

Name

dosFsOpen( ) - open a file on a dosFs volume

Layer

FS_DOSFS

Synopsis

```
DOS_FILE_DESC_ID dosFsOpen
    (
    DOS_VOLUME_DESC_ID pVolDesc,   /* pointer to volume descriptor */
    char *             pPath,      /* dosFs full path/filename */
    int                flags,      /* file open flags */
    int                mode        /* file open permissions (mode) */
    )
```

Description

This routine opens the file name with the specified mode (**O_RDONLY**/**O_WRONLY**/**O_RDWR**/CREATE/TRUNC). The directory structure is searched, and if the file is found a dosFs file descriptor is initialized for it. Extended flags are provided by DOS FS for more efficiency:

- **DOS_O_CONTIG_CHK** - to check file for contiguity.
- **DOS_O_CASENS** - force the file name lookup in case insensitive manner, (if directory format provides such opportunity)

If this is the very first open for the volume, configuration data will be read from the disk automatically (via **dosFsVolMount( )**).

Returns

A pointer to a dosFs file descriptor, or **ERROR** if the volume is not available, or there are no available dosFs file descriptors, or there is no such file and **O_CREAT** was not specified, or file cannot be opened with such permissions.

Errno

EINVAL EROFS ENOENT EEXIST /NOMANUAL

See also

dosFsLib

Name

dosFsClose( ) - close a dosFs file

Layer

FS_DOSFS

Synopsis

```
STATUS dosFsClose
    (
    DOS_FILE_DESC_ID pFd   /* file descriptor pointer */
    )
```

Description

This routine closes the specified dosFs file. If file contains excess clusters beyond **EOF** they are freed, when last file descriptor is being closed for that file.

Returns

OK, or **ERROR** if directory couldn't be flushed or entry couldn't be found.

Errno

EINVAL ENOENT /NOMANUAL

See also

dosFsLib

Name

dosFsIoctl( ) - do device specific control function

Layer

FS_DOSFS

Synopsis

```
STATUS dosFsIoctl
    (
    FAST DOS_FILE_DESC_ID pFd,        /* fd of file to control */
    int                   function,   /* function code */
    _Vx_ioctl_arg_t       arg         /* some argument */
    )
```

Description

This routine performs the following ioctl functions.

Any ioctl function codes, that are not supported by this routine are passed to the underlying XBD module for handling.

There are some **ioctl( )** functions, that suppose to receive as result a 32-bit numeric value (FIONFREE, FIOWHERE and so on), however disks and files with size grater, than 4GB are supported. In order to solve this contradiction new **ioctl( )** functions are

provided. They have the same name as basic functions, but with suffix **64**: FIONFREE64, FIOWHERE64 and so on. These functions gets pointer to **long long** as an argument. Also FIOWHERE64 returns value via argument, but not as **ioctl( )**returned value. If an ioctl fails, the task's status (see **errnoGet( )**) indicates the nature of the error.

Returns

OK or current position in file for FIOWHERE, or **ERROR** if function failed or driver returned error, or if function supposes 32 bit result value, but actual result overloads this restriction.

Errno

EINVAL ENXIO ENOENT EOVERFLOW /NOMANUAL

See also

dosFsLib

Name

dosFsLastAccessDateEnable( ) - enable last access date updating for this volume

Layer

FS_DOSFS

Synopsis

```
STATUS dosFsLastAccessDateEnable
    (
    DOS_VOLUME_DESC_ID dosVolDescId,   /* dosfs volume ID to alter */
    BOOL               enable          /* TRUE = enable update, FALSE = disable update */
    )
```

Description

This function enables or disables updating of the last access date directory entry field on open-read-close operations for the given dosFs volume. The last access date file indicates the last date that a file has been read or written. When the optional last access date field update is enabled, read operations on a file will cause a write to the media.

Returns

OK or **ERROR** if the volume is invalid or enable is not **TRUE** or **FALSE**.

Errno

Not Available

See also

dosFsLib

Name

dosFsLibInit( ) - prepare to use the dosFs library

Layer

FS_DOSFS

Synopsis

```
STATUS dosFsLibInit
    (
    int maxFiles,
    int options
    )
```

Description

This routine initializes the dosFs library. This initialization is enabled when the configuration macro **INCLUDE_DOSFS** is defined. This routine installs **dosFsLib** as a driver in the I/O system driver table, and allocates and sets up the necessary structures. The driver number assigned to **dosFsLib** is placed in the global variable dosFsDrvNum.

Returns

OK or **ERROR**, if driver cannot be installed.

Errno

Not Available

See also

dosFsLib

Name

dosFsDevCreate( ) - create file system device.

Layer

FS_DOSFS

Synopsis

```
STATUS dosFsDevCreate
    (
    char *   pDevName,             /* device name */
    device_t device,               /* underlying XBD block device */
    int      maxFiles,             /* max no. of simultaneously open files */
    int      dosDevCreateOptions   /* write option & volume integrity */
    )
```

Description

This routine associates an XBD device with a logical I/O device name and prepare it to perform file system functions. It takes an XBD device handle, typically created by **xbdBlkDevCreate( )** or **xbdPartitionDevCreate( )**, and defines it as a dosFs volume. As

a result, when high-level I/O operations (e.g., **open( )**, **write( )**) are performed on the device, the calls will be routed through **dosFsLib**. The device parameter is the handle of the underlying partition or block device XBD.

The argument maxFiles specifies the number of files that can be opened at once on the device.

The volume structure integrity can be automatically checked during volume mounting. Parameter dosDevCreateOptions defines checking level (**DOS_CHK_ONLY** or **DOS_CHK_REPAIR**), that can be bitwise or-ed with check verbosity level value (**DOS_CHK_VERB_SILENT**, **DOS_CHK_VERB_1** or **DOS_CHK_VERB_2**).

If the value of dosDevCreateOptions is 0, the default checking level is used. The default level is (**DOS_CHK_ONLY** | **DOS_CHK_VERB_2**).

To suppress the automatic check disk, bitwise or (**DOS_CHK_NONE**) or set dosDevCreateOptions to NONE.

Disk checking is normally suppressed on volumes marked clean. To force a disk-check, bitwise or (**DOS_CHK_FORCE**).

The volume may be configured to request **DOS_WRITE_THROUGH** writes for some or all of the disk operations. Additional bits of parameter dosDevCreateOptions define the volume's write-through setting. The default (zero) is to use copyback writes (**DOS_WRITE**) for all write operations. The default is the fastest configuration.

To writethrough all FAT table write operations, or in **DOS_WRITE_THROUGH_FAT** To writethrough all directory entry write operations, or in **DOS_WRITE_THROUGH_DIR** To writethrough all user data buffers, or in **DOS_WRITE_THROUGH_USR** To writethrough both FAT and DIRENT operations, or them together. (**DOS_WRITE_THROUGH_DIR** | **DOS_WRITE_THROUGH_FAT** | **DOS_CHK_NONE**)

User data writes will still use copyback **DOS_WRITE** operations when using (**DOS_WRITE_THROUGH_DIR** | **DOS_WRITE_THROUGH_FAT**)

To write-through all write operations, including all user data, or in **DOS_WRITE_THROUGH_ALL**. This is the slowest operation and all write operation made by the file system will be **DOS_WRITE_THROUGH**.

To enable Unicode filenames, or in **DOS_FILENAMES_UNICODE**. Case insensitivity (if enabled) currently applies only to ASCII values even when Unicode is turned on. For instance, a German eszet is never considered the same as two uppercase S characters, but two uppercase S characters can match two lowercase S characters because these are both in the first 128 character codes.

Note

Setting parameter dosDevCreateOptions to NONE (-1) will both disable the automated chkdsk and force copyback (**DOS_WRITE**) operation. Unicode filenames will not be enabled.

Note that during a call to **dosFsDevCreate( )** actual disk accesses are deferred to the time when **open( )** or **creat( )** are first called. That is also when the automatic disk checking will take place. Therefore this function will succeed in cases where a removable disk is not present in the drive.

Returns

OK, or **ERROR** if the device name is already in use or insufficient memory.

/NOMANUAL

Errno

Not Available

See also

dosFsLib

Name

dosFsDevDelete( ) - delete a dosFs volume

Layer

FS_DOSFS

Synopsis

```
STATUS dosFsDevDelete
    (
    DOS_VOLUME_DESC_ID pVolDesc   /* pointer to volume descriptor */
    )
```

Description

This routine deletes a dosFs volume.

Returns

OK on success, **ERROR** otherwise

Errno

Not Available

See also

dosFsLib

Name

dosFsMonitorDevCreate( ) - create a dosFs volume through the fs monitor

Layer

FS_DOSFS

Synopsis

```
STATUS dosFsMonitorDevCreate
    (
    device_t xbdId,     /* XBD for the device on which to mount. */
    char *   pDevName   /* Name of the DOS FS device (mount point). */
    )
```

Description

This routine creates an DOS FS device.

Returns

OK if successful. **ERROR** otherwise.

/NOMANUAL

Errno

Not Available

See also

dosFsLib

Name

dosFsDiskProbe( ) - probe if a device contains a valid dosFs

Layer

FS_DOSFS

Synopsis

```
STATUS dosFsDiskProbe
    (
    device_t xbdDevice   /* XBD device to probe */
    )
```

Description

This routine probes if a device (or a partition) contains a valid DOS FS.

Returns

OK if successful. **ERROR** otherwise.

/NOMANUAL

Errno

Not Available

See also

dosFsLib

Name

dosFsHdlrInstall( ) - install handler.

Layer

FS_DOSFS

Synopsis

```
STATUS dosFsHdlrInstall
    (
    DOS_HDLR_DESC_ID hdlrsList,   /* appropriate list */
    DOS_HDLR_DESC_ID hdlr         /* ptr on handler descriptor */
    )
```

Description

This library does not directly access directory structure, nor FAT, rather it uses particular handlers to serve such accesses. This function is intended for use by the **dosFsLib** sub-modules only.

This routine installs a handler into DOS FS handlers list. There are two such lists: FAT Handlers List (dosFatHdlrsList) and Directory Handlers List (dosDirHdlrsList). Each handler must provide its unique Id (see **dosFsLibP.h**) and pointer to appropriate list to install it to. All lists are sorted by Id-s in ascending order. Every handler is tried to be mounted on each new volume in accordance to their order in list, until succeeded. So preferable handlers, that supports the same type of volumes must have less Id values.

Returns

STATUS.

/NOMANUAL

Errno

Not Available

See also

dosFsLib

Name

dosFsXbdBlkRead( ) - read blocks from the underlying XBD block device.

Layer

FS_DOSFS

Synopsis

```
STATUS dosFsXbdBlkRead
    (
    DOS_VOLUME_DESC * pVolDesc,     /* volume descriptor */
    sector_t          startBlock,   /* starting block of transfer */
    sector_t          numBlocks,    /* number of blocks to transfer */
    addr_t            buffer        /* address of the memory buffer */
    )
```

Description

none

Returns

STATUS.

/NOMANUAL

Errno

Not Available

See also

dosFsLib

Name

dosFsXbdBlkWrite( ) - write blocks to the underlying XBD block device.

Layer

FS_DOSFS

Synopsis

```
STATUS dosFsXbdBlkWrite
    (
    DOS_VOLUME_DESC * pVolDesc,     /* volume descriptor */
    sector_t          startBlock,   /* starting block of write */
    sector_t          numBlocks,    /* number of blocks to write */
    addr_t            buffer,       /* address of the memory buffer */
    DOS_RW            operation     /* DOS_WRITE/DOS_WRITE_THROUGH */
    )
```

Description

none

Returns

STATUS.

/NOMANUAL

Errno

Not Available

See also

dosFsLib

Name

dosFsXbdBytesRW( ) - read/write bytes to/from the underlying XBD block device.

Layer

FS_DOSFS

Synopsis

```
STATUS dosFsXbdBytesRW
    (
    DOS_VOLUME_DESC * pVolDesc,     /* volume descriptor */
    sector_t          startBlock,   /* starting block of the transfer */
    off_t             offset,       /* offset into block in bytes */
    addr_t            buffer,       /* address of data buffer */
    size_t            nBytes,       /* number of bytes to transfer */
    u_int             operation     /* DOS_READ/DOS_WRITE/WRITE_THROUGH */
    )
```

Description

none

Returns

STATUS.

/NOMANUAL

Errno

Not Available

See also

dosFsLib

Name

dosFsXbdBlkCopy( ) - copy blocks on the underlying XBD block device.

Layer

FS_DOSFS

Synopsis

```
STATUS dosFsXbdBlkCopy
    (
    DOS_VOLUME_DESC * pVolDesc,   /* volume descriptor */
    sector_t          srcBlock,
    sector_t          dstBlock,
```

```
    sector_t          numBlocks
    )
```

Description

none

Returns

STATUS.

/NOMANUAL

Errno

Not Available

See also

dosFsLib

Name

dosFsXbdIoctl( ) - Misc control operations

Layer

FS_DOSFS

Synopsis

```
STATUS dosFsXbdIoctl
    (
    DOS_VOLUME_DESC * pVolDesc,
    UINT32            command,
    addr_t            arg
    )
```

Description

This performs the requested old CBIO **ioctl( )** operations.

RETURNS **OK** or **ERROR** and may otherwise set errno.

/NOMANUAL

Returns

Not Available

Errno

Not Available

See also

dosFsLib

Name

dosFsDiscard( ) - discard operations

Layer

FS_DOSFS

Synopsis

```
int dosFsDiscard
    (
    DOS_VOLUME_DESC * pVolDesc,
    SECTOR_RANGE *    range,
    BOOL              flush
    )
```

Description

This performs discard operations for some devices.

RETURNS **OK** or **ERROR** and may otherwise set errno.

/NOMANUAL

Returns

Not Available

Errno

Not Available

See also

dosFsLib

6. DOSFSSHOW

Name

dosFsShow - DosFS Show routines

Layer

FS_DOSFS

Routines

dosFsShow( ) - display dosFs volume configuration data. **dosFsCacheShow( )** - show information regarding a dosFs volume's cache **dosFsTCacheShow( )** - show information regarding a dosFs volume's tiny cache

Description

This library implements the DosFS Show routines.

Include files

dosFsLib.h

Name

dosFsShow( ) - display dosFs volume configuration data.

Layer

FS_DOSFS

Synopsis

```
STATUS dosFsShow
    (
    void * pDevName,    /* name of device */
    u_int  level        /* detail level */
    )
```

Description

This routine obtains the dosFs volume configuration for the named device, formats the data, and displays it on the standard output.

If no device name is specified, the current default device is described.

Returns

OK or **ERROR**, if no valid device specified.

Errno

Not Available

See also

dosFsShow

Name

dosFsCacheShow( ) - show information regarding a dosFs volume's cache

Layer

FS_DOSFS

Synopsis

```
void dosFsCacheShow
    (
    const char * volName,    /* volume name */
    u_int        level       /* verbosity level */
    )
```

Description

This routine displays information regarding an specific dosFs volume volName cache. If level is zero, a summary is displayed, otherwise the current state of the internal hash table and LRU list is displayed.

Returns

N/A

Errno

Not Available

See also

dosFsShow

Name

dosFsTCacheShow( ) - show information regarding a dosFs volume's tiny cache

Layer

FS_DOSFS

Synopsis

```
void dosFsTCacheShow
    (
    const char * volName    /* volume name */
    )
```

Description

This routine displays information regarding an specific dosFs volume volName tiny cache.

Returns

N/A

Errno

Not Available

See also

dosFsShow

7. FSEVENTUTILLIB

Name

fsEventUtilLib - Event Utility functions for different file systems

Layer

FS

Routines

fsEventUtilInit( ) - Initialize the file system event utlility library **fsPathAddedEventSetup( )** - Setup to wait for a path **fsPathAddedEventRaise( )** - Raise a "path added" event **fsWaitForPath( )** - wait for a path **fsPathAddEventHandler( )** - add an event handler for path event

Description

This library contains file systems event utility routines.

Include files

fsEventUtilLib.h

Name

fsEventUtilInit( ) - Initialize the file system event utlility library

Layer

FS

Synopsis

```
STATUS fsEventUtilLibInit
    (
    void
    )
```

Description

none

Returns

OK on success, **ERROR** on failure

Errno

Not Available

See also

fsEventUtilLib

Name

fsPathAddedEventSetup( ) - Setup to wait for a path

Layer

FS

Synopsis

```
STATUS fsPathAddedEventSetup
    (
    FS_PATH_WAIT_STRUCT * pWaitData,
    char *                path
    )
```

Description

This routine registers with the ERF to wait for the specified path to be added to core I/O by a file system. This is mainly used my file system formatters when the eject the current file system and wait for rawFs to instantiate.

Returns

OK on success or **ERROR** on error.

Errno

Not Available

See also

fsEventUtilLib

Name

fsPathAddedEventRaise( ) - Raise a "path added" event

Layer

FS

Synopsis

```
void fsPathAddedEventRaise
    (
    char * coreIOPath
    )
```

Description

This routine raises an event with the event reporting framework when the specified path has been added to core I/O by a file system. This routine will cause the wait for path handler(s) to run.

Returns

Not Available

Errno

Not Available

See also

fsEventUtilLib

Name

fsWaitForPath( ) - wait for a path

Layer

FS

Synopsis

```
STATUS fsWaitForPath
    (
    FS_PATH_WAIT_STRUCT *pWaitData
    )
```

Description

This routine waits for a path to be added to core I/O. The function, fsPathAddedEventSetup, must be prior for pWaitData to be setup. This function simply waits on the semaphore provided in pWaitData which will be given by the wait for path handler.

Returns

OK on success or **ERROR** on failure

Errno

Not Available

See also

fsEventUtilLib

Name

fsPathAddEventHandler( ) - add an event handler for path event

Layer

FS

Synopsis

```
STATUS fsPathAddEventHandler
    (
    erfHandlerPrototype * pEventHandler,   /* Pointer to Event Handler */
    void *                pUserData,       /* User data to be sent to Handler */
    UINT16                flags            /* Event Processing flags */
    )
```

Description

This routine adds an event handler for event that a path to be added to core I/O. It call **erfHandlerRegister( )** to register the handler.

Returns

OK on success or **ERROR** on failure

Errno

Not Available

See also

fsEventUtilLib

8. FSIMLIB

Name

fsimLib - flash simulation library

Layer

BDM_SIM

Routines

fsimCreate( ) - create a flash simulator **fsimDelete( )** - delete a flash simulator **fsimMount( )** - mount a flash simulator from a given device or file **fsimUnmount( )** - unmount a flash simulator **fsimStart( )** - start a flash simulator **fsimRun( )** - run a flash simulator **fsimStop( )** - stop a flash simulator **fsimRead( )** - read data from a given raw address **fsimWrite( )** - write data to a given raw address **fsimReadPage( )** - read data from a given page **fsimWritePage( )** - wrte data to a given page **fsimReadPages( )** - read a number of pages from a simulated flash **fsimWritePages( )** - write a number of pages to a simulated flash **fsimIsBlankPage( )** - Check if a page is blank or not **fsimErase( )** - erase a number of blocks of a simulated flash **fsimIoctl( )** - perform an I/O control function on a simulated flash **fsimShow( )** - show the information of a flash simulator **fsimShowAll( )** - show all the flash simulators **fsimDump( )** - dump the flash content of given pages **fsimDumpData( )** - dump the flash content of given pages **fsimDumpOob( )** - dump the oob of given pages **fsimImageSave( )** - save the whole flash simulator to a given file **fsimImageClone( )** - clone the given image to another given medium **fsimStatChip( )** - get the statistics of the chip of a flash simulator **fsimStatBlk( )** - get the statistics of given blocks **fsimStatPage( )** - get the statistics of given pages **fsimMarkBad( )** - mark a block as bad **fsimUnmarkBad( )** - unmark a bad block **fsimIsBadBlock( )** - Check if a block is bad **fsimInjectError( )** - inject error to a page **fsimErrorGenerator( )** - randomly inject errors to flash chip **fsimRemoveError( )** - remove error from a page **fsimReboot( )** - trigger a warm system reboot **fsimInvalidateMap( )** - invalidate all the pages mapped in the map window **fsimIsPageMapped( )** - check if the given page is mapped **fsimUpdateMapStatus( )** - update the map status of the given page

Description

This library intends to provide a unified and flexible solution for simulating flash devices, so as to assist the development and sustaining of flash-specific software.

Since flash device has limited lifespan and most develop boards do not support hot-plug of raw flash device, it might not be a good idea to run a software under development directly on the flash device. For example, if the software is bugging, the wearing of the flash device might not be even, which, in turn, might cause the flash device inoperable in a short period. But the flash simulator does not have such kind of concern. A flash simulator can be created, used and deleted repeately on a device like widely used hard disk which is cheaper and easier to change.

In addition, the flash simulator provides facilities to intentionally inject error to the simulated flash to help verify the correctness of Flash-specific software. Since flash devices are more error-prone than other devices like hard disks, and so most flash-specific software should be able to handle the bad block or error page in flash, but verifing the correctness of this aspect of the software might not be easy on a actual flash device because it is uncertain when a block will become bad and when a page will have error. A bad block might occur at the point close to the end of the lifespan of the whole flash device. But with the flash simulator, we can intentionally inject errors to any location of the simulated flash at any time, create a bad block at any location as well.

Another benefit is that user of flash simulator is that it is flexible enough to simulate various size of flash with different page size and block size and so on. User doesn't have to bother setting up the different flash on different board, all user has to do is preparing a device (e.g. a hard disk) on which the flash simulator is to be created and provides the specification of the to-be-simulated flash.

On the negative side, the flash simulator does have limitations. The most obvious one is that it is not an ideal software for performance testing. Besides this, simulating some advanced flash like FusionFlash is not supported.

Design

The flash simulator creates flash devices on the user-specified mass storage medium like hard disk, USB, etc. Whatever the underlying medium is, the flash simulator access it through a uniform interface - standard I/O interface. This requires a valid fd which would be got through opening the given device or file name When creating flash simulator. To be able to open a device directly, the device should have been formatted as rawFs in the case of VxWorks. In VxWorks, any block devices will be instantiated as rawFs if no other file system is detected on the device. Moreover, the rawFs is a very light file system which just managing the whole device as a single file and provides basic I/O access to it. However, it fits into the flash simulator design perfectly for accessing underlying medium.

Besides simulating the flash itself, the flash simulator saves some usefull statistics information to the underlying medium to help diagnose the upper flash-specific software. For example, the erase count of each block is saved as statistic information. It can help check whether the flash-specific software manages the erase count of each block well enough.

In addition, in order to preserve the simulated flash in the case of power-cut, the flash simulator saves the configuration provided by user in the underlying non-volatile medium so that the flash simulator can be re-mounted manually at reboot.

Given a device or file on which a flash simulator is created, the internal layout of the simulated flash device is as follows:

```
-------------------------------------------------------------
|               |            |                                |
| configuration | statistics |            flash chip          |
|               |            |                                |
-------------------------------------------------------------
```

Creating flash simulator

A flash simulator can be created through **fsimCreate( )**. A identity of type **FSIM_ID** is returned upon success for latter access to the flash simulator. Following example creates a flash simulator named "/fsim0" on the device "/ram0" using the default configuration for a 256MB size NAND flash. See the API manual of **fsimCreate( )** for more information.

```
    {
    FSIM_ID fsimId;
    fsimId = fsimCreate ("/fsim0", "/ram0", &cfgNand256MB);
    }
```

Using flash simulator

After a flash simulator has been created, the simulated flash can be read, written or erased as a actual flash device. Following is an example which writes two pages of data to the simulated flash starting from page 8.

```
    {
    FSIM_ID fsimId;
    char *  pBuf = NULL;
    size_t  bufLen;
    int     result = -1;
    int     option = FSIM_OPT_DATA | FSIM_OPT_OOB | FSIM_OPT_ECC;

    fsimId = fsimCreate(...);
    ...       // initialize the pBuf, etc.
    result = fsimWritePages (fsimId, 8, 2, pBuf, bufLen, option);
    if (result != 0)
        {
        // error handling
```

```
        }
    }
```

Deleting flash simulator

A flash simulator can be deleted through either its unique identity or name.

```
    fsimDelete (fsimIdOrName);
```

Interfaces with end user

This library provides interfaces for performing basic operations like create, delete, mount and unmount. Besides this, interfaces for performing basic I/O operations like read, write, and erase are provided as well as interfaces for performing advanced operations like injecting errors, marking bad blocks, dumping flash contents, showing flash information, etc.

Interface with upper layer

Based on different implementation of upper layer flash-specific software, different wrapper layer need to be implemented for seamless integration with the upper layer.

Configuration

Following configuration options are provided in **fsimLib.h** for configuring the flash simulator.

FSIM_CONFIG_NUM_FACTORY_BAD_BLKS	configure the number of factory bad blocks
FSIM_CONFIG_DELAY_READ_US	read delay in microseconds
FSIM_CONFIG_DELAY_WRITE_US	write delay in microseconds
FSIM_CONFIG_DELAY_ERASE_US	erase delay in microseconds

Include files

fsimLib.h

See also

VxWorks 7 File Systems and I/O Programmer's Guide, VxWorks 7 BSP and Driver Guide

Name

fsimCreate( ) - create a flash simulator

Layer

BDM_SIM

Synopsis

```
FSIM_ID fsimCreate
    (
```

```
    const char * fsimName,     /* name for the flash simulator */
    const char * mediumName,   /* name of the underlying medium */
    FSIM_CFG *   pCfgArg       /* configuration for the simulator */
    )
```

Description

This routine creates a flash simulator on given underlying medium with given configuration.

The pCfgArg parameter is a pointer to a structure which contains a number of configurable attibutes of the flash simulator.

typedef struct fsim_cfg { int magic; **FLASH_TYPE_T** flashType; **FLASH_CELL_T** cellType; UINT64 totalSize; int pageSize; int nPagesPerBlk; }**FSIM_CFG**;

flashType: the type of flash to be simulated, following types are
supported:
- **FLASH_TYPE_NAND**: NAND flash
- **FLASH_TYPE_NOR**: NOR flash
cellType: the cell type of the simulated flash, following types are
supported:
- **FLASH_CELL_SLC**: SLC NAND flash
- **FLASH_CELL_MLC**: MLC NAND flash
totalSize: the total size of the simulated flash device.
pageSize: the page size, it should be power of 2. For NOR flash, it
should be set to 1.
nPagesPerBlk: the number of pages per block, it should be a multiple of
32.

Here is an example of configuring a simulated SLC NAND flash of 256MB size, whose page size is 2KB and each block has 64 pages.

```
const FSIM_CFG cfgNand256MB = {FLASH_TYPE_NAND, FLASH_CELL_SLC,
              256 * SIZE_MB, 2 * SIZE_KB, 64};
```

Following example creates a flash simulator "/fsim0" on the ramdisk "/ram0" using the above configuration. NOTE that the actual size of the created NAND flash depends on the actual accessible size of the given device and the overhead of managing statistics.

```
    FSIM_ID fsimId;
    fsimId = fsimCreate ("/fsim0", "/ram0", &cfgNand256MB);
    fsimShow (fsimId);
```

Returns

The ID of the created flash simulator, otherwise **NULL**.

Errno

NONE

See also

fsimLib, **fsimDelete( )**, **fsimMount( )**, **fsimUnmount( )**

Name

fsimDelete( ) - delete a flash simulator

Layer

BDM_SIM

Synopsis

```
STATUS fsimDelete
    (
    FSIM_ID fsimIdOrName
    )
```

Description

none

Returns

OK upon success, otherwise **ERROR**.

Errno

NONE

See also

fsimLib, **fsimMount( )**, **fsimUnmount( )**, **fsimCreate( )**, **fsimDelete( )**

Name

fsimMount( ) - mount a flash simulator from a given device or file

Layer

BDM_SIM

Synopsis

```
FSIM_ID fsimMount
    (
    const char * fsimName,   /* flash simulator name */
    const char * devName     /* device name or file name */
    )
```

Description

Following example intends to mount a flash simulator on device "/disk0" and name the flash simulator as "/fsim0".

```
    fsimMount("/fsim0", "/disk0");
```

Returns

fsim id upon success, otherwise **NULL**.

Errno

NONE

See also

fsimLib, **fsimUnmount( )**, **fsimCreate( )**, **fsimDelete( )**

Name

fsimUnmount( ) - unmount a flash simulator

Layer

BDM_SIM

Synopsis

```
STATUS fsimUnmount
    (
    FSIM_ID fsimIdOrName   /* device name or file name */
    )
```

Description

none

Returns

OK upon success, otherwise **ERROR**.

Errno

NONE

See also

fsimLib, **fsimMount( )**, **fsimCreate( )**, **fsimDelete( )**

Name

fsimStart( ) - start a flash simulator

Layer

BDM_SIM

Synopsis

```
STATUS fsimStart
    (
    FSIM_ID fsimIdOrName   /* flash simulator identity */
    )
```

Description

Flash simulators will be started automatically after being created. Flash simulators can be operated only after it has been started.

Returns

OK upon success, otherwise **ERROR**

Errno

NONE

See also

fsimLib, **fsimStop( )**, **fsimRun( )**

Name

fsimRun( ) - run a flash simulator

Layer

BDM_SIM

Synopsis

```
STATUS fsimRun
    (
    FSIM_ID fsimIdOrName
    )
```

Description

none

Returns

OK upon success, otherwise **ERROR**

Errno

NONE

See also

fsimLib, **fsimStart( )**, **fsimStop( )**

Name

fsimStop( ) - stop a flash simulator

Layer

BDM_SIM

Synopsis

```
STATUS fsimStop
    (
    FSIM_ID fsimIdOrName
    )
```

Description

none

Returns

OK upon success, otherwise **ERROR**

Errno

NONE

See also

fsimLib, **fsimStart( )**, **fsimRun( )**

Name

fsimRead( ) - read data from a given raw address

Layer

BDM_SIM

Synopsis

```
int fsimRead
    (
    FSIM_ID fsimId,     /* flash simulator ID */
    UINT64  fromAddr,   /* from where to read */
    size_t  length,     /* length of data to be read */
    char *  pBuf        /* buffer to contains the data to be written */
    )
```

Description

This routine reads data from a given raw address, not necessarilly page size aligned. For NAND flash, the oob area is ignored when addressing.

Returns

0 upon success, otherwise a error number

FSIM_ERR_ECC_REC - Recoverable ECC error
FSIM_ERR_ECC_NOT_REC - Unrecoverable ECC error
FSIM_ERR_NOT_REC - Permanent broken page
FSIM_ERR_PARAM - Invalid parameter
FSIM_ERR_UNKNOWN - Unknown error

Errno

NONE

See also

fsimLib, **fsimReadPages( )**, **fsimWrite( )**, **fsimErase( )**, **fsimWritePages( )**

Name

fsimWrite( ) - write data to a given raw address

Layer

BDM_SIM

Synopsis

```
int fsimWrite
    (
```

```
    FSIM_ID fsimId,     /* flash simulator ID */
    UINT64  fromAddr,   /* from where to read */
    size_t  length,     /* length of data to be read */
    char *  pBuf        /* buffer to contains the data to be written */
    )
```

Description

This routine writes data to a given raw address, not nessarilly page size aligned. For NAND flash, the oob area is ignored when addressing.

Returns

0 upon success, otherwise a error number

FSIM_ERR_ECC_REC - Recoverable ECC error
FSIM_ERR_ECC_NOT_REC - Unrecoverable ECC error
FSIM_ERR_NOT_REC - Permanent broken page
FSIM_ERR_PARAM - Invalid parameter
FSIM_ERR_UNKNOWN - Unknown error

Errno

NONE

See also

fsimLib, **fsimReadPages( )**, **fsimWrite( )**, **fsimErase( )**, **fsimWritePages( )**

Name

fsimReadPage( ) - read data from a given page

Layer

BDM_SIM

Synopsis

```
int fsimReadPage
    (
    FSIM_ID fsimId,   /* flash simulator ID */
    size_t  pageNo,   /* page to be read */
    off_t   offset,   /* offset in page from where to read */
    size_t  length,   /* length of data to be read */
    char *  pBuf      /* buffer to contains the data to be written */
    )
```

Description

This routine is NAND page aware. So it can be used to read any bytes in the page (oob area included).

Returns

0 upon success, otherwise a error number

FSIM_ERR_ECC_REC - Recoverable ECC error
FSIM_ERR_ECC_NOT_REC - Unrecoverable ECC error
FSIM_ERR_NOT_REC - Permanent broken page
FSIM_ERR_PARAM - Invalid parameter
FSIM_ERR_UNKNOWN - Unknown error

Errno

NONE

See also

fsimLib, **fsimReadPages( )**, **fsimWrite( )**, **fsimErase( )**, **fsimWritePages( )**

Name

fsimWritePage( ) - wrte data to a given page

Layer

BDM_SIM

Synopsis

```
int fsimWritePage
    (
    FSIM_ID fsimId,    /* flash simulator ID */
    size_t  pageNo,    /* page to be write */
    off_t   offset,    /* offset in page from where to write */
    size_t  length,    /* length of data to be write */
    char *  pBuf       /* buffer to contains the data to be written */
    )
```

Description

This routine is NAND page aware. So it can be used to write any address in the page (oob area included).

Returns

0 upon success, otherwise a error number

FSIM_ERR_ECC_REC - Recoverable ECC error	
FSIM_ERR_ECC_NOT_REC - Unrecoverable ECC error	
FSIM_ERR_NOT_REC - Permanent broken page	
FSIM_ERR_PARAM - Invalid parameter	
FSIM_ERR_UNKNOWN - Unknown error	

Errno

NONE

See also

fsimLib, **fsimReadPages( )**, **fsimWritePages( )**, **fsimErase( )**, fsimRead ()

Name

fsimReadPages( ) - read a number of pages from a simulated flash

Layer

BDM_SIM

Synopsis

```
int fsimReadPages
    (
    FSIM_ID fsimId,     /* flash simulator ID */
    size_t  fromPage,   /* from page */
    size_t  nPages,     /* number of pages to be written */
    char *  pBuf,       /* buffer to contains the data to be written */
    size_t  bufLen,     /* buffer length */
    int     option      /* FSIM_OPT_DATA or FSIM_OPT_OOB or */
    )
```

Description

This routine reads a number of pages from the simulated flash and put the data to user buffer.

The options parameter is an additional options supported by this routine. Following options are supported:

FSIM_OPT_DATA	read the data area of a page
FSIM_OPT_OOB	read the oob area of a page
FSIM_OPT_ECC	Do ecc verification

Returns

0 upon success, otherwise a error number

FSIM_ERR_ECC_REC - Recoverable ECC error	
FSIM_ERR_ECC_NOT_REC - Unrecoverable ECC error	
FSIM_ERR_NOT_REC - Permanent broken page	
FSIM_ERR_PARAM - Invalid parameter	
FSIM_ERR_UNKNOWN - Unknown error	

Errno

NONE

See also

fsimLib, **fsimRead( )**, **fsimWrite( )**, **fsimErase( )**, **fsimWritePage( )**

Name

fsimWritePages( ) - write a number of pages to a simulated flash

Layer

BDM_SIM

Synopsis

```
int fsimWritePages
    (
    FSIM_ID fsimId,     /* flash simulator ID */
    size_t  fromPage,   /* from page */
    size_t  nPages,     /* number of pages to be written */
    char *  pBuf,       /* buffer to contains the data to be written */
    size_t  bufLen,     /* buffer length */
    int     option      /* FSIM_OPT_DATA or FSIM_OPT_OOB or */
    )
```

Description

This routine writes a number of pages to the simulated flash.

The options parameter gives some additional options. Following options are supported:

FSIM_OPT_DATA	write the data area of a page
FSIM_OPT_OOB	write the oob area of a page

Returns

0 upon success, otherwise a non-zero error number

FSIM_ERR_NOT_REC - Permanent broken page
FSIM_ERR_PARAM - Invalid parameter
FSIM_ERR_UNKNOWN - Unknown error

Errno

NONE

See also

fsimLib, **fsimRead( )**, **fsimErase( )**

Name

fsimIsBlankPage( ) - Check if a page is blank or not

Layer

BDM_SIM

Synopsis

```
int fsimIsBlankPage
    (
    FSIM_ID fsimId,   /* flash simulator ID */
    size_t  pageNo,   /* pageNo */
    BOOL *  pResult   /* output: true or false */
    )
```

Description

none

Returns

OK if the page is read out and checked, otherwise **ERROR**

Errno

NONE

See also

fsimLib, **fsimMarkBad( )**, **fsimUnmarkBad( )**

Name

fsimErase( ) - erase a number of blocks of a simulated flash

Layer

BDM_SIM

Synopsis

```
int fsimErase
    (
    FSIM_ID fsimId,   /* flash simulator ID */
    size_t  blkNo,    /* starting block number */
    size_t  nBlks     /* number of blocks to be erased */
    )
```

Description

none

Returns

0 upon success, otherwise a non-zero error number

FSIM_ERR_BAD_BLOCK - One or more blocks to be erased are bad.
FSIM_ERR_PARAM - Invalid parameter
FSIM_ERR_UNKNOWN - Unknown error

Errno

NONE

See also

fsimLib, **fsimRead( )**, **fsimWrite( )**, **fsimReadPages( )**, **fsimWritePages( )**

Name

fsimIoctl( ) - perform an I/O control function on a simulated flash

Layer

BDM_SIM

Synopsis

```
FSIM_ID fsimIoctl
    (
    FSIM_ID          fsimId,
    int              cmd,
    FSIM_IOCTL_ARG_T arg
    )
```

Description

Currently only the following command is supported.

FSIM_IOCTL_GET_INFO - get information of the flash simulator

The arg argument is used to provide additional information for the cmd or receive the result from performing the function.

Following example intends to get the basic information of the flash simulator.

```
    FSIM_INFO fsimInfo;
    fsimIoctl (fsimId, FSIM_IOCTL_GET_INFO, (FSIM_IOCTL_ARG_T)&fsimInfo);
```

Returns

OK upon success, otherwise **ERROR**.

Errno

NONE

See also

fsimLib, **fsimRead( )**, **fsimWrite( )**, **fsimErase( )**

Name

fsimShow( ) - show the information of a flash simulator

Layer

BDM_SIM

Synopsis

```
STATUS fsimShow
    (
    FSIM_ID fsimIdOrName
    )
```

Description

This routine shows the current information of given simulated flash.

Returns

OK upon success, otherwise **ERROR**

Errno

NONE

See also

fsimLib, **fsimShowAll( )**, **fsimDump( )**, **fsimStat( )**

Name

fsimShowAll( ) - show all the flash simulators

Layer

BDM_SIM

Synopsis

```
STATUS fsimShowAll
    (
    void
    )
```

Description

This routine shows all the flash simulators. It calls **fsimShow( )** to show each listed flash simulator.

Returns

OK upon success, otherwise **ERROR**

Errno

NONE

See also

fsimLib, **fsimShow( )**, **fsimDump( )**, **fsimStat( )**

Name

fsimDump( ) - dump the flash content of given pages

Layer

BDM_SIM

Synopsis

```
STATUS fsimDump
    (
    FSIM_ID fsimIdOrName,   /* flash simulator identity */
    int     dumpFd,         /* stdout or a given fd */
    int     type,           /* data, oob or data & oob */
    size_t  pageNo,         /* starting page */
    size_t  nPages          /* number of pages */
    )
```

Description

This routine dumps the flash content of given pages into given opened file.

The type parameter specifies what content of a page should be dumped. There are two types supported:

FSIM_DUMP_DATA	dump the data area of a page
FSIM_DUMP_OOB	dump the oob area of a page

Returns

OK if all the give pages are dumped, otherwise **ERROR**.

Errno

Not Available

See also

fsimLib, **fsimShow( )**, **fsimShowAll( )**, **fsimStat( )**

Name

fsimDumpData( ) - dump the flash content of given pages

Layer

BDM_SIM

Synopsis

```
STATUS fsimDumpData
    (
    FSIM_ID fsimIdOrName,   /* flash simulator identity */
    size_t  pageNo,         /* starting page */
    size_t  nPages          /* number of pages */
    )
```

Description

This routine dumps the flash content of given pages into given opened file.

Returns

OK if all the give pages are dumped, otherwise **ERROR**.

Errno

NONE

See also

fsimLib, **fsimShow( )**, **fsimShowAll( )**, **fsimStat( )**

Name

fsimDumpOob( ) - dump the oob of given pages

Layer

BDM_SIM

Synopsis

```
STATUS fsimDumpOob
    (
    FSIM_ID fsimIdOrName,   /* flash simulator identity */
    size_t  pageNo,         /* starting page */
    size_t  nPages          /* number of pages */
    )
```

Description

This routine dumps the oob content of given pages

Returns

OK if all the give pages are dumped, otherwise **ERROR**.

Errno

NONE

See also

fsimLib, **fsimShow( )**, **fsimShowAll( )**, **fsimStat( )**

Name

fsimImageSave( ) - save the whole flash simulator to a given file

Layer

BDM_SIM

Synopsis

```
STATUS fsimImageSave
    (
    FSIM_ID      fsimIdOrName,   /* flash simulator identity */
    const char * fileName        /* file where the image to be saved */
    )
```

Description

none

Returns

OK if the image is saved successfully, otherwise **ERROR**.

Errno

Not Available

See also

fsimLib, **fsimShow( )**, **fsimShowAll( )**, **fsimStat( )**, **fsimDump( )**

Name

fsimImageClone( ) - clone the given image to another given medium

Layer

BDM_SIM

Synopsis

```
STATUS fsimImageCopy
    (
    const char * srcName,
    const char * dstName
    )
```

Description

none

Returns

OK if the image is saved successfully, otherwise **ERROR**.

Errno

Not Available

See also

fsimLib, **fsimShow( )**, **fsimShowAll( )**, **fsimStat( )**, **fsimDump( )**

Name

fsimStatChip( ) - get the statistics of the chip of a flash simulator

Layer

BDM_SIM

Synopsis

```
STATUS fsimStatChip
    (
    FSIM_ID fsimIdOrName,   /* flash simulator identity */
    int     statFd          /* stdout or a given fd */
    )
```

Description

none

Returns

OK upon success, otherwise **ERROR**.

Errno

Not Available

See also

fsimLib, **fsimShow( )**, **fsimShowAll( )**, **fsimDump( )**, **fsimStatBlk( )**, **fsimStatPage( )**

Name

fsimStatBlk( ) - get the statistics of given blocks

Layer

BDM_SIM

Synopsis

```
STATUS fsimStatBlk
    (
    FSIM_ID fsimIdOrName,   /* flash simulator identity */
    int     statFd,         /* stdout or a given fd */
    size_t  blkNo,          /* block number */
    size_t  nBlks           /* number of blocks */
    )
```

Description

none

Returns

OK upon success, otherwise **ERROR**.

Errno

Not Available

See also

fsimLib, **fsimShow( )**, **fsimShowAll( )**, **fsimDump( )**, **fsimStatChip( )**, **fsimStatPage( )**

Name

fsimStatPage( ) - get the statistics of given pages

Layer

BDM_SIM

Synopsis

```
STATUS fsimStatPage
    (
    FSIM_ID fsimIdOrName,   /* flash simulator identity */
    int     statFd,         /* stdout or a given fd */
    size_t  pageNo,         /* page number in block */
    size_t  nPages          /* number of pages */
    )
```

Description

none

Returns

OK upon success, otherwise **ERROR**.

Errno

Not Available

See also

fsimLib, **fsimShow( )**, **fsimShowAll( )**, **fsimDump( )**, **fsimStatChip( )**, **fsimStatBlk( )**

Name

fsimMarkBad( ) - mark a block as bad

Layer

BDM_SIM

Synopsis

```
STATUS fsimMarkBad
    (
    FSIM_ID fsimIdOrName,
    size_t  blkNo
    )
```

Description

This routine marks a given block as bad. This routine might be usefull for debugging the upper layer software to verify if the bad block management mechnism works well or not. Since in real environment, a bad block would not be likely to occur unless the block has been erased thousands of times. With the help of this routine, user can mark any block bad at any time.

Returns

OK upon success, otherwise **ERROR**.

Errno

NONE

See also

fsimLib, **fsimUnmarkBad( )**

Name

fsimUnmarkBad( ) - unmark a bad block

Layer

BDM_SIM

Synopsis

```
STATUS fsimUnmarkBad
    (
    FSIM_ID fsimIdOrName,
    size_t  blkNo
    )
```

Description

This routine removes the **bad** mark of a block set by fsimMarkBad.

Returns

OK upon success, otherwise **ERROR**.

Errno

NONE

See also

fsimLib, **fsimMarkBad( )**

Name

fsimIsBadBlock( ) - Check if a block is bad

Layer

BDM_SIM

Synopsis

```
bool fsimIsBadBlock
    (
    FSIM_ID fsimIdOrName,
    size_t  blkNo
    )
```

Description

none

Returns

TRUE if it is a bad block, otherwise **FALSE**.

Errno

NONE

See also

fsimLib, **fsimMarkBad( )**, **fsimUnmarkBad( )**

Name

fsimInjectError( ) - inject error to a page

Layer

BDM_SIM

Synopsis

```
STATUS fsimInjectError
    (
    FSIM_ID fsimIdOrName,
    size_t  pageNo,
    int     error           /* ECC recoverable, not recoverable, etc */
    )
```

Description

This routine injects error to a given page.

The error parameter gives the error to be injected to the page. Following errors are supported. Multiple errors can be injected by OR-ing following errors.

FSIM_ERR_ECC_REC	recoverable ECC error
FSIM_ERR_ECC_NOT_REC	unrecoverable ECC error
FSIM_ERR_NOT_REC	permanent unrecoverable error

Returns

OK upon success, otherwise **ERROR**.

Errno

NONE

See also

fsimLib, **fsimRemoveError( )**

Name

fsimErrorGenerator( ) - randomly inject errors to flash chip

Layer

BDM_SIM

Synopsis

```
STATUS fsimErrorGenerator
    (
    FSIM_ID fsimIdOrName
    )
```

Description

none

Returns

OK upon success, otherwise **ERROR**.

Errno

NONE

See also

fsimLib, **fsimInjectError( )**, **fsimRemoveError( )**

Name

fsimRemoveError( ) - remove error from a page

Layer

BDM_SIM

Synopsis

```
STATUS fsimRemoveError
    (
    FSIM_ID fsimIdOrName,
    size_t  pageNo,
    int     error
    )
```

Description

This routine removes the error of a given page injected by fsimInjectError.

Returns

OK upon success, otherwise **ERROR**.

Errno

NONE

See also

fsimLib, **fsimRemoveError( )**

Name

fsimReboot( ) - trigger a warm system reboot

Layer

BDM_SIM

Synopsis

```
STATUS fsimReboot
    (
    FSIM_ID        fsimIdOrName,   /* flash simulator identity */
    int            type,           /* clean or unclean reboot */
    TASK_ID        taskId,         /* trigger the reboot in which task's context */
```

```
    REBOOT_FUNCPTR rebootFunc      /* the callback before reboot */
    )
```

Description

This routine requests flash simulator to trigger a warm system reboot. User can specify a clean reboot which happens after the flash simulator has done its current operation, or a unclean reboot which happens in the middle of the next operation requested by given task. User can also specify a hook which will be called before the reboot event is triggered.

The purpose of introducing this routine is to help upper layer software developer mimic power failure that might happen at any point in the real world. It is a little tricking about how to use it if user want to control the reboot event to happen at a specific line of the source code of the upper layer software. First when debugging, user should stop the running software to the target line of source code (most debugging facility provide this functionality of stopping execution at any specific line), and then call this routine in the context of another task (typically tShell under VxWorks), finally continue running the software.

Here is a use case for debugging VxWorks FTL software layer. Since VxWorks FTL provide a feature of power failure tolerance, there should be an efficient way to test whether this feature is supported well. For example, whether the FTL could get recovered from a power failure event, more specificly whether the FTL could prevent its meta-data from corrupted from a interrupted write, GC, or WL operation. To verify the feature, we could reboot the system in the middle of a write or GC or WL operation with the help of this routine.

The type parameter specifies the type of reboot. Currently two types of reboot are supported, clean and unclean. The clean reboot means before the system is rebooted, the flash simulator will first enter a clean status without any on-going operations that might change the flash simulator. The unclean reboot will happen in the middle of the next operation performed on the simulated flash. For example, if the next operation is a write operation, system will reboot after portion of the data is written.

FSIM_REBOOT_UNCLEAN - unclean reboot
FSIM_REBOOT_CLEAN - clean reboot

Following example requests a unclean reboot at the context of the current task. No additional action is specified in this case.

```
    fsimStatPage(fsimId, FSIM_REBOOT_UNCLEAN, taskIdSelf(), NULL);
```

Returns

OK upon success, otherwise **ERROR**.

Errno

NONE

See also

fsimLib

Name

fsimInvalidateMap( ) - invalidate all the pages mapped in the map window

Layer

BDM_SIM

Synopsis

```
int fsimInvalidateMap
    (
    FSIM_ID fsimId
    )
```

Description

none

Returns

OK, or **ERROR** if invalid fsim.

Errno

NONE

See also

fsimLib, **fsimMarkBad( )**, **fsimUnmarkBad( )**

Name

fsimIsPageMapped( ) - check if the given page is mapped

Layer

BDM_SIM

Synopsis

```
int fsimIsPageMapped
    (
    FSIM_ID fsimId,
    size_t  pageNo
    )
```

Description

none

Returns

TRUE if the page is mapped, otherwise **ERROR**

Errno

NONE

See also

fsimLib, **fsimInvalidateMap( )**, **fsimUpdateMapStatus( )**

Name

fsimUpdateMapStatus( ) - update the map status of the given page

Layer

BDM_SIM

Synopsis

```
int fsimUpdateMapStatus
    (
    FSIM_ID fsimId,
    size_t  pageNo,
    UINT8   status
    )
```

Description

none

Returns

OK, or **ERROR** if invalid fsim

Errno

NONE

See also

fsimLib, **fsimInvalidateMap( )**, **fsimIsPageMapped( )**

9. FSMAGIC

Name

fsMagic - File System Magic Number Utility Library

Layer

FS

Routines

deviceFsInfoShow( ) - display file system information

Description

This file contains routines to allow a user to display a block device file system name and basic information.

Include files

none

Name

deviceFsInfoShow( ) - display file system information

Layer

FS

Synopsis

```
int deviceFsInfoShow
    (
    char * pDevName
    )
```

Description

This routine display basic file system information by giving a specific device name.

The file system information contains:

File System

File system name of the device

Block Size

Fundamental file system block size

Total Blocks

Total blocks in file system

Free Blocks

Free block in file system

Available Block

Free blocks avail to non-superuser

Total Files

Total file nodes in file system, only support in node based file system

Free Files

Free file nodes in file system, only support in node based file system

File System ID

File system ID

If the specific file system does not support a value, it may return -1 for the value.

Returns

OK or **ERROR**

Errno

N/A

See also

fsMagic

10. FSMONITOR

Name

fsMonitor - The File System Monitor

Layer

FS

Routines

fsMonitorInit( ) - Initialize the fsMonitor **fsmNameMap( )** - map an XBD name to a Core I/O path **fsmProbeInstall( )** - install F/S probe and instantiator functions **fsmProbeUninstall( )** - remove a file system probe **fsmNameInstall( )** - Add a mapping between an XBD name and a pathname **fsmNameUninstall( )** - remove an XBD name to pathname mapping **fsmGetDriver( )** - Get the XBD name of a mapping based on the path **fsmGetVolume( )** - get the pathname based on an XBD name mapping **fsmUnmountHookAdd( )** - Add an unmount hook function **fsmUnmountHookDelete( )** - Remove an unmount hook function **fsmUnmountHookRun( )** - Runs the unmount hook functions **fsmDevGetByName( )** - get XBD device handle by CoreIO device name

Description

This library implements the File System Monitor, which controls the autodetection and instantiation of File Systems.

Include files

fsMonitor.h

See also

VxWorks 7 File Systems and I/O Programmer's Guide

Name

fsMonitorInit( ) - Initialize the fsMonitor

Layer

FS

Synopsis

```
STATUS fsMonitorInit(void)
```

Description

This routine initializes the fsMonitor library.

Returns

OK or **ERROR**

Errno

Not Available

See also

fsMonitor

Name

fsmNameMap( ) - map an XBD name to a Core I/O path

Layer

FS

Synopsis

```
STATUS fsmNameMap
    (
    devname_t xbdName,    /* XBD name */
    fsmName_t volName     /* core I/O path */
    )
```

Description

This function maps an XBD name to a path in Core I/O, either by an explicit mapping specified by **fsmNameInstall( )** or by using the XBD name to create a Core I/O pathname. This function always succeeds.

The xbdName parameter specifies the name of the device to be mapped.

The volName parameter specifies the resultant mapped name.

Returns

OK

Errno

Not Available

See also

fsMonitor

Name

fsmProbeInstall( ) - install F/S probe and instantiator functions

Layer

FS

Synopsis

```
STATUS fsmProbeInstall
    (
    fsmProbeFunc probe,   /* probe routine to install */
    fsmInstFunc  inst     /* instantiator routine to install */
    )
```

Description

This routine installs the file system probe and instantiator functions. When a new file system is discovered, the file system monitor will call these functions to test for, and instantiate a particular file system type. If either function fails, then the file system is not created.

The probe parameter specifies the probe function to be used.

The inst parameter specifies the instantiator function to be used when the probe function succeeds.

Returns

OK on success, or **ERROR** if an error is detected.

Errno

Not Available

See also

fsMonitor

Name

fsmProbeUninstall( ) - remove a file system probe

Layer

FS

Synopsis

```
STATUS fsmProbeUninstall
    (
    fsmProbeFunc probe   /* probe routine to uninstall */
    )
```

Description

This routine removes all probe-instantiator pairs that match the probe parameter.

The probe parameter specifies the probe function of the probe-instantiator pair to be removed.

Returns

0 if a probe is removed or **ERROR** if the probe is not found

Errno

Not Available

See also

fsMonitor

Name

fsmNameInstall( ) - Add a mapping between an XBD name and a pathname

Layer

FS

Synopsis

```
STATUS fsmNameInstall
    (
    char *driver,   /* XBD driver name */
    char *volume    /* core I/O pathname */
    )
```

Description

This routine creates a mapping between the driver name and the pathname specified by volume. This mapping will persist until removed by fsmNameUninstall.

The driver parameter specifies the XBD name to be mapped.

The volume parameter specifies the Core I/O path that driver is to be mapped into.

Returns

OK if successful or **ERROR** if the name cannot be added

Errno

Not Available

See also

fsMonitor

Name

fsmNameUninstall( ) - remove an XBD name to pathname mapping

Layer

FS

Synopsis

```
STATUS fsmNameUninstall
    (
    char *driver   /* driver name */
    )
```

Description

This routine removes a name mapping added by fsmNameInstall. After invocation of this routine, the F/S monitor will create a pathname based on the XBD name instead of using a name mapping.

The driver parameter specifies the name of the XBD for which a mapping is to be removed. All occurences of driver are removed.

Returns

OK if the name mapping is removed or **ERROR** if the mapping is not found.

Errno

Not Available

See also

fsMonitor

Name

fsmGetDriver( ) - Get the XBD name of a mapping based on the path

Layer

FS

Synopsis

```
STATUS fsmGetDriver
    (
    char *    volume,   /* core I/O pathname */
    devname_t driver    /* xbd driver name */
    )
```

Description

This routine gets the XBD name which is currently mapped to volume, if such a mapping exists.

The volume parameter specifies the pathname for which a driver name is to be retrieved.

The driver parameter is the resultant of the name mapping, if found.

Returns

OK if the mapping is found, or **ERROR** if no such mapping exists.

Errno

Not Available

See also

fsMonitor, **devGetByName( )**, **fsmDevGetByName( )**

Name

fsmGetVolume( ) - get the pathname based on an XBD name mapping

Layer

FS

Synopsis

```
STATUS fsmGetVolume
    (
    char      *driver,   /* XBD driver name */
    fsmName_t volume     /* core I/O pathname */
    )
```

Description

This routine retrieves the pathname associated with an XBD name if such a mapping exists.

The driver parameter specifies a driver name for which a pathname is to be retrieved.

The volume parameter specifies the resultant pathname.

Returns

OK if the mapping is retrieved, or **ERROR** if it does not exist.

Errno

Not Available

See also

fsMonitor

Name

fsmUnmountHookAdd( ) - Add an unmount hook function

Layer

FS

Synopsis

```
STATUS fsmUnmountHookAdd
    (
    FUNCPTR fn
    )
```

Description

This routine adds a hook routine to run when a vnode based file system unmounts.

The fn parameter specifies the hook function.

Returns

OK if there is space in the table and fn is not **NULL**

/NOMANUAL

Errno

Not Available

See also

fsMonitor

Name

fsmUnmountHookDelete( ) - Remove an unmount hook function

Layer

FS

Synopsis

```
STATUS fsmUnmountHookDelete
    (
    FUNCPTR fn
    )
```

Description

This routine removes a hook routine to run when a vnode based file system unmounts.

The fn parameter specifies the hook function.

Returns

OK if fn is found in the table. **ERROR** otherwise

/NOMANUAL

Errno

Not Available

See also

fsMonitor

Name

fsmUnmountHookRun( ) - Runs the unmount hook functions

Layer

FS

Synopsis

```
void fsmUnmountHookRun
    (
    DEV_HDR *pDev
    )
```

Description

This routine is called my the vnode layer when a file system unmounts to run the unmount hook functions.

The pDev parameter is passed to each hook function

/NOMANUAL

Returns

Not Available

Errno

Not Available

See also

fsMonitor

Name

fsmDevGetByName( ) - get XBD device handle by CoreIO device name

Layer

FS

Synopsis

```
device_t fsmDevGetByName
    (
    char * ioName
    )
```

Description

This routine get XBD device handle by CoreIO device name.

The ioName specifies the device name in CoreIO. The CoreIO device name may be different with XBD device name. The XBD device name may be mapped, renamed before adding to CoreIO.

Returns

device handle, NULLDEV otherwise

Errno

Not Available

See also

fsMonitor, **devGetByName( )**

11. FSUTILITYFUNCS

Name

fsUtilityFuncs - file system Utility functions

Layer

FS

Routines

fsDeviceValidateMount( ) - verify a device is mounted **fsDeviceRename( )** - Rename a file system IOS device

Description

This file defines a routine to allow a user to rename a signle file system IOS device and/or to delay VxWorks initalization until that device is available.

Include files

vxWorks.h stdio.h stdlib.h ioLib.h iosLib.h private/iosLibP.h sys/stat.h, **string.h fsUtilityFuncs.h drv/xbd/xbd.h taskLib.h taskLibCommon.h iosLib.h**, **fsMonitor.h tickLib.h sysLib.h**

Name

fsDeviceValidateMount( ) - verify a device is mounted

Layer

FS

Synopsis

```
STATUS fsDeviceValidateMount
    (
    const char * pName,      /* Name of the device to verify */
    UINT32       timeOut_s   /* Max number of seconds to wait */
    )
```

Description

This routine waits for a specified device to be mounted with a timeout. It returns **OK** when the name is mounted with a valid file system, including raw file system.

pName - Name of the device or a path to verify timeOut_s - Max number of seconds to wait

Returns

OK or **ERROR**

Errno

Not Available

See also

fsUtilityFuncs

Name

fsDeviceRename( ) - Rename a file system IOS device

Layer

FS

Synopsis

```
STATUS fsDeviceRename
    (
    const char * pOrigDevName,   /* Name of the device to be renamed */
    const char * pNewDevName,    /* Name that will be used for rename */
    UINT32       timeOut_s       /* Max number of seconds to wait */
    )
```

Description

This routine will rename the specified file system IOS device

pOrigDevName - Name of the device to be renamed pNewDevName - Name that will be used for rename

Only when pOrigDevName is a file system IOS device name, that is to say that it is a block device name formatted with rawFs/HRFS/dosFS, fsDeviceRename will succeed. When pOrigDevName is other devices, such as /null, host:, tty and etc, fsDeviceRename will fail.

For example, fsDeviceRename ("/ata1:1", "/fsNewName1", 5) will return **OK**, fsDeviceRename ("host:", "/fsNewName2", 5) will return **ERROR**.

Returns

OK or **ERROR**

Errno

Not Available

See also

fsUtilityFuncs

12. FTLFORMAT

Name

ftlFormat - ftl low-level format library

Layer

BDM_FLASH

Routines

ftlFormat( ) - format the flash device and make it ready for use

Description

This library implements functionality of formatting a flash device and making the device ready for use by upper layers of ftl.

It provides two APIs for formatting flash devices. The recommended one for use between them is the ftlAdvFormat. It takes more flexible arguments than the ftlFormat that only accept a fixed number of arguments. The ftlFormat is easier for use in interactive enivonment through.

Include files

none

Name

ftlFormat( ) - format the flash device and make it ready for use

Layer

BDM_FLASH

Synopsis

```
STATUS ftlFormat
    (
    const char * pDevName,   /* pointer to flash device name */
    int          sizeGC,     /* size to faciliate GC and WL in percentage */
    int          sizeBBM,    /* size to do replace bad blocks */
    int          options     /* options for additional functionalities */
    )
```

Description

This function formats the given flash device and make it ready for use. Note that it is destructive, any existing data on it would be destroyed.

A description of the arguments are as follows:

flashDevName

The name of the device to be formatted.

sizeGC

The size (in percentage of the total size) of the flash space that could be reserved by FTL to faciliate GC and WL. Giving a appropriate value would bring performance gain at the expense of usable flash space. In most cases, the larger this value, the better the performance. However, it is limited up to 30% because the management overhead would grow as the value gets larger. The acceptable range of this parameter is from 3 to 30 (means 3% to 30%). Specifying a value that is not in this range would get a **ERROR** in return.

sizeBBM

The size (in percentage of the total size) of the flash space that could be reserved by FTL to replace runtime bad blocks. Because the blocks in NAND Flash could not be guaranteed to have the same P/E cycles and even the best WL algorithm can not make all the blocks be erased the same times as system running, it is necessary to provide some spare blocks for emergency of handling runtime bad blocks or erronous blocks. The minimum value that is acceptable is 1 (1%), while the maximum is 10 (%10).

options

The options to specify additional functionalities. if user has no ftl defined option to specify, set it to 0. Currently it is ignored by formatter because no format specific options is defined so far.

Returns

OK on success or **ERROR** on failure

Errno

NONE

See also

ftlFormat, **ftlAdvFormat( )**, VxWorks 7 File Systems and I/O Programmer's Guide, VxWorks 7 BSP and Driver Guide

13. FTLMOUNT

Name

ftlMount - ftl mount library

Layer

BDM_FLASH

Routines

ftlMount( ) - mount the ftl formatted flash device

Description

This file contains the routines that are used to mount a ftl managed device as well as routines for recovering ftl from erronous meta data.

The major job of mount is to scan out the valid meta data that is needed at runtime and keep them in RAM, and then to make the device ready for use.

The mounting process is tedious because it is not known where saves the valid meta data exactly. A number of meta blocks have to be scanned, checked for errors, and determined as valid or not.

Mountint could fail only when irrecoverrable fatal errors occurred.

Include files

none

Name

ftlMount( ) - mount the ftl formatted flash device

Layer

BDM_FLASH

Synopsis

```
STATUS ftlMount
    (
    const char * flashDevName,   /* flash device name */
    int          options         /* additional options, not used now */
    )
```

Description

This function mounts a ftl formatted flash device and makes it ready for use by upper layer of ftl. Note that mounting an already mounted device would fail. Note that the options parameter is ignored currently

Returns

OK on success or **ERROR** on failure

Errno

NONE

See also

ftlMount

14. FTLSHOW

Name

ftlShow - ftl show library

Layer

BDM_FLASH

Routines

ftlDescShow( ) - show the description of the ftl managed device **ftlMetaLogShow( )** - show the meta log **ftlStatShow( )** - show the statistics of given ftl device **ftlBmShow( )** - show the geometry of given device. **ftlBmShowAll( )** - show the geometry of all flash device known by ftl. **ftlDataCacheShowAll( )** - show the internals of data cache for a given ftl device **ftlDataCacheShow( )** - show the cache info for the lbn of given ftl device **ftlLbdShow( )** - show the status of logical block control module **ftlLockShow( )** - show all the internal locks used by ftl **ftlMapShow( )** - show the mapping for the lbn of given ftl device **ftlCursorCacheShow( )** - show the internals of ftl cursor cache **ftlGrpBlkMapShow( )** - show the block map for the group of given ftl device **ftlPgMapShowAll( )** - show the associate page map cache **ftlAssocBlkShow( )** - show the information about the associate blocks **ftlWlShow( )** - show the wear-leveling related information **ftlCfgShow( )** - show the user configuration of the ftl device

Description

This library contains all the ftl show routines that could be used to desplay many aspects of the ftl managed flash device including the geometry, the status, the statistics, etc. For example, **ftlMemShow** could be used to show the memory usage by ftl, **ftlDescShow** for showing the layout of the ftl device, **ftlMapShow** for showing the mapping for a given lbn, and so on.

Note that this library is configurable through **INCLUDE_FTL_SHOW** and is not configured by default. It could be used for debugging purpose as well as performance evaluation.

Include files

none

Name

ftlDescShow( ) - show the description of the ftl managed device

Layer

BDM_FLASH

Synopsis

```
int ftlDescShow
    (
    const char * pDevName
    )
```

Description

This function shows the description of the ftl managed device including internal layout, etc.

Returns

ERROR if device not found, otherwise **OK**.

Errno

N/A

See also

ftlShow

Name

ftlMetaLogShow( ) - show the meta log

Layer

BDM_FLASH

Synopsis

```
int ftlMetaLogShow
    (
    const char * pDevName
    )
```

Description

This function shows the meta log of given ftl device.

Returns

ERROR if device not found, otherwise **OK**.

Errno

N/A

See also

ftlShow

Name

ftlStatShow( ) - show the statistics of given ftl device

Layer

BDM_FLASH

Synopsis

```
int ftlStatShow
    (
    const char * pDevName
    )
```

Description

This function shows the statistics of given ftl device.

Returns

ERROR if device not found, otherwise **OK**.

Errno

N/A

See also

ftlShow

Name

ftlBmShow( ) - show the geometry of given device.

Layer

BDM_FLASH

Synopsis

```
int ftlBmShow
    (
    const char * pDevName
    )
```

Description

This function shows the geometry of given device.

Returns

ERROR if device not found, otherwise **OK**.

Errno

N/A

See also

ftlShow

Name

ftlBmShowAll( ) - show the geometry of all flash device known by ftl.

Layer

BDM_FLASH

Synopsis

```
int ftlBmShowAll
    (
    void
    )
```

Description

none

Returns

ERROR if device not found, otherwise **OK**.

Errno

N/A

See also

ftlShow

Name

ftlDataCacheShowAll( ) - show the internals of data cache for a given ftl device

Layer

BDM_FLASH

Synopsis

```
int ftlDataCacheShowAll
    (
    const char * pDevName
    )
```

Description

none

Returns

ERROR if device not found, otherwise **OK**.

Errno

N/A

See also

ftlShow

Name

ftlDataCacheShow( ) - show the cache info for the lbn of given ftl device

Layer

BDM_FLASH

Synopsis

```
int ftlDataCacheShow
    (
    const char * pDevName,
    size_t       lbn
    )
```

Description

none

Returns

ERROR if device not found, otherwise **OK**.

Errno

N/A

See also

ftlShow

Name

ftlLbdShow( ) - show the status of logical block control module

Layer

BDM_FLASH

Synopsis

```
int ftlLbdShow
    (
    const char * pDevName
    )
```

Description

none

Returns

ERROR if device not found, otherwise **OK**.

Errno

N/A

See also

ftlShow

Name

ftlLockShow( ) - show all the internal locks used by ftl

Layer

BDM_FLASH

Synopsis

```
int ftlLockShow
    (
    const char * pDevName
    )
```

Description

none

Returns

ERROR if device not found, otherwise **OK**.

Errno

N/A

See also

ftlShow

Name

ftlMapShow( ) - show the mapping for the lbn of given ftl device

Layer

BDM_FLASH

Synopsis

```
int ftlMapShow
    (
    const char * pDevName,
    fblock_t     lbn
    )
```

Description

none

Returns

ERROR if device not found, otherwise **OK**.

Errno

N/A

See also

ftlShow

Name

ftlCursorCacheShow( ) - show the internals of ftl cursor cache

Layer

BDM_FLASH

Synopsis

```
int ftlCursorCacheShow
    (
```

```
    const char * pDevName
    )
```

Description

none

Returns

ERROR if device not found, otherwise **OK**.

Errno

N/A

See also

ftlShow

Name

ftlGrpBlkMapShow( ) - show the block map for the group of given ftl device

Layer

BDM_FLASH

Synopsis

```
int ftlGrpBlkMapShow
    (
    const char * pDevName,
    int          grpNo
    )
```

Description

none

Returns

ERROR if device not found, otherwise **OK**.

Errno

N/A

See also

ftlShow

Name

ftlPgMapShowAll( ) - show the associate page map cache

Layer

BDM_FLASH

Synopsis

```
int ftlPgMapShowAll
    (
    const char * pDevName
    )
```

Description

none

Returns

ERROR if device not found, otherwise **OK**.

Errno

N/A

See also

ftlShow

Name

ftlAssocBlkShow( ) - show the information about the associate blocks

Layer

BDM_FLASH

Synopsis

```
int ftlAssocBlkShow
    (
    const char * pDevName
    )
```

Description

none

Returns

ERROR if device not found, otherwise **OK**.

Errno

N/A

See also

ftlShow

Name

ftlWlShow( ) - show the wear-leveling related information

Layer

BDM_FLASH

Synopsis

```
int ftlWlShow
    (
    const char * pDevName
    )
```

Description

none

Returns

ERROR if device not found, otherwise **OK**.

Errno

N/A

See also

ftlShow

Name

ftlCfgShow( ) - show the user configuration of the ftl device

Layer

BDM_FLASH

Synopsis

```
int ftlCfgShow
    (
    const char * pDevName
    )
```

Description

none

Returns

ERROR if device not found, otherwise **OK**.

Errno

N/A

See also

ftlShow

15. GPT

Name

gpt - GUID Partition Table library

Layer

FS

Routines

xbdGptPartStart( ) - start to add gpt partition **xbdGptPartAdd( )** - prepare to add gpt part **xbdGptPartAddWithType( )** - prepare to add gpt part with specific type **xbdGptPartFin( )** - finish add the gpt part **xbdGptPartVecAdd( )** - add gpt partition by vector

Description

This module implements the support of GUID partition table.

Include files

vxWorks.h stdio.h stdlib.h string.h strings.h errno.h, **fcntl.h stddef.h fsMonitor.h fsEventUtilLib.h xbdPartition.h**, **drv/xbd/bio.h drv/xbd/xbd.h drv/erf/erfLib.h semLib.h gpt.h**

See also

VxWorks 7 File Systems and I/O Programmer's Guide

Name

xbdGptPartStart( ) - start to add gpt partition

Layer

FS

Synopsis

```
int xbdGptPartStart
    (
    char * devName
    )
```

Description

This routine start to add gpt partition. After call this routine, use xbdGptPartAdd or xbdGptPartAddWithType to add partitions, use xbdGptPartFin to finish adding and flush gpt partition table to disk.

Returns

OK, or **ERROR** if fail

Errno

N/A

See also

gpt, xbdGptPartAdd, xbdGptPartFin

Name

xbdGptPartAdd( ) - prepare to add gpt part

Layer

FS

Synopsis

```
STATUS xbdGptPartAdd
    (
    char * devName,
    UINT64 parStart,
    UINT64 parSize
    )
```

Description

This routine prepares to add gpt part.

If both partition start and size is specified, this routine will try to create partition at this location with this size.

If both partition start and size is NOT specified, this routine will try to find first free space and create partition with maximum size.

If only partition start is specified, and disk is free at the location, this routine will try to create partition at the location with maximum size.

If only partition size is specified, this routine search whole disk and find the FIRST free space which is equal or larger than parameter and create a partition here with specific size.

Returns

OK, or **ERROR** if fail

Errno

N/A

See also

gpt

Name

xbdGptPartAddWithType( ) - prepare to add gpt part with specific type

Layer

FS

Synopsis

```
STATUS xbdGptPartAddWithType
    (
    char * devName,
    UINT64 parStart,
    UINT64 parSize,
    char * uuidStr
    )
```

Description

This routine prepares to add gpt part with specific type, the type should be a valid UUID string format, or **NULL** with default value.

Returns

OK, or **ERROR** if fail

Errno

N/A

See also

gpt, xbdGptPartAdd

Name

xbdGptPartFin( ) - finish add the gpt part

Layer

FS

Synopsis

```
int xbdGptPartFin
    (
    char * devName
    )
```

Description

This routine finish add the gpt part, will update gpt information to disk

Returns

OK, or **ERROR** if fail

Errno

N/A

See also

gpt

Name

xbdGptPartVecAdd( ) - add gpt partition by vector

Layer

FS

Synopsis

```
int xbdGptPartVecAdd
    (
    char *        devName,
    int           nParts,
    pGPT_PART_VEC pPGptPartVec
    )
```

Description

This routine adds gpt partition by vector

Returns

OK, or **ERROR** if fail

Errno

N/A

See also

gpt

16. GPTSHOW

Name

gptShow - GUID Partition Table Show Routine library

Layer

FS

Routines

gptShowMbr( ) - show mbr information **gptShowPartHdr( )** - show gpt header information

Description

This module implements the support of GUID partition table show routines.

Include files

none

Name

gptShowMbr( ) - show mbr information

Layer

FS

Synopsis

```
void gptShowMbr
    (
    char * devName
    )
```

Description

This routine shows mbr information

Returns

N/A

Errno

N/A

See also

gptShow

Name

gptShowPartHdr( ) - show gpt header information

Layer

FS

Synopsis

```
void gptShowPartHdr
    (
    char * devName
    )
```

Description

This routine shows gpt header information

Returns

N/A

Errno

N/A

See also

gptShow

17. HRFSCHKDSKLIB

Name

hrfsChkDskLib - HRFS Check disk library - Readonly version

Layer

FS_HRFS

Routines

hrfsChkDsk( ) - check the HRFS file system **hrfsUpgrade( )** - upgrade the HRFS file system to the latest version

Description

This library contains routines for the Highly Reliable File System (HRFS) consistency disk checker or check disk. This is a read-only utility in that it does not attempt to correct any errors it detects but simply reports them.

Include files

vxWorks.h sys/stat.h stdlib.h string.h strings.h ioLib.h stdio.h, **fsEventUtilLib.h fsMonitor.h private/hrFsLibP.h**

Name

hrfsChkDsk( ) - check the HRFS file system

Layer

FS_HRFS

Synopsis

```
STATUS hrfsChkDsk
    (
    char * path,        /* path to check */
    int    verbLevel,   /* verbosity level */
    int    flags        /* additional control information */
    )
```

Description

This routine is the HRFS consistency checker. It checks to see if the file system referenced by the path is stable and consistent.

WARNING. This function can only run on an inactive volume. Any currently
opened files will be closed as this routine will eject the current
file system. The volume will also be unaccessible while the consistency checker executes.

The path argument should be a valid path to the HRFS formatted disk or partition to be checked.

The verbLevel argument is used to specify how much information is outputted to the console. A value of one indicates maximum verbosity. A value of zero indicates minimum verbosity.

The flags parameter is used to specify additional control information to the consistency checker. If the **HRFS_CHKDSK_FLAG_UPGRADE** bit is set, the checker will attempt to upgrade the file system to the newest version. All other flags are ignored if this bit is set. If the **HRFS_CHKDSK_FLAG_REWIND_INODE_JOURNAL** bit is set, the checker will attempt to rewind the inode journal. That is, if the inode journal is not empty and is marked as being out of sync with the other disk structures, inodes contained in the inode journal are copied back into the inode table on disk. This has the effect of restoring the inodes to the previous transaction.

Returns

OK if media contains no errors or **ERROR** if one of more problems are detected.

Errno

Not Available

See also

hrfsChkDskLib

Name

hrfsUpgrade( ) - upgrade the HRFS file system to the latest version

Layer

FS_HRFS

Synopsis

```
STATUS hrfsUpgrade
    (
    char *path   /* path to upgrade */
    )
```

Description

This routine is the HRFS consistency checker. It checks to see if the file system referenced by the path is stable and consistent.

WARNING. This function can only run on an inactive volume. Any currently opened files will closed as this routine will eject the current file system. The volume will also be unaccessible while the consistency checker executes.

The path argument should be a valid path to the HRFS formatted disk or partition to be checked.

Returns

OK if media was upgraded without errors. **ERROR** if one of more problems are detected.

Errno

Not Available

See also

hrfsChkDskLib

18. HRFSFORMATLIB

Name

hrfsFormatLib - HRFS format library

Layer

FS_HRFS

Routines

hrfsFormatLibInit( ) - prepare to use the HRFS formatter **hrfsFormatFd( )** - format the HRFS file system via a file descriptor **hrfsAdvFormatFd( )** - format the HRFS file system using advanced options via a file descriptor **hrfsFormat( )** - format the HRFS file system via a path **hrfsAdvFormat( )** - format the HRFS file system using advanced options

Description

This library contains routines for formatting the Highly Reliable File System (HRFS).

Include files

hrFsLib.h

See also

VxWorks 7 File Systems and I/O Programmer's Guide, Overview of HRFS

Name

hrfsFormatLibInit( ) - prepare to use the HRFS formatter

Layer

FS_HRFS

Synopsis

```
STATUS hrfsFormatLibInit(void)
```

Description

This routine initializes the HRFS formatter library. This initialization is enabled when the configuration macro **INCLUDE_HRFS_FORMAT** is defined.

Returns

OK always

Errno

Not Available

See also

hrfsFormatLib

Name

hrfsFormatFd( ) - format the HRFS file system via a file descriptor

Layer

FS_HRFS

Synopsis

```
STATUS hrfsFormatFd
    (
    int    fd,         /* open file descriptor on disk */
    UINT64 diskSize,   /* size of disk in bytes */
    UINT32 blkSize,    /* size of block in bytes */
    UINT32 numInodes   /* number of Inodes */
    )
```

Description

This routine formats a disk or partition referenced by an open file on the media. The file is closed when formatting is complete.

The fd argument should be a valid file descriptor representing the root directory of the current file system. This file descriptor will be marked closed and invalid upon return from this function reguardless of outcome.

The diskSize argument is used to specifiy how many bytes of the media the HRFS file system should occupy. It can be used to prevent HRFS for using the end portion of the media. In general this value should be 0 to specify that the entire media is to be used.

The blkSize parameter is used to specify what block size, in bytes, HRFS should use. This block size must be a power of 2, greater than the physical sector size, and be within 512 to 32768 bytes inclusively. In general this value should be specified as 0 so the formatter can determine the most efficient block size to use for the media size.

The numInodes parameter is used to specify the absolute maximum number of files and directories the file system can ever have. Note this does not include the root directory which the formatter creates automatically. Specifiying a value of 0 will tell the HRFS formatter to allow for the maximum number of files/directories the file system can have based on the amount of data blocks. I.e. One inode per data block.

Returns

OK on success or **ERROR** on failure

Errno

Not Available

See also

hrfsFormatLib

WNDRVR

Name

hrfsAdvFormatFd( ) - format the HRFS file system using advanced options via a file descriptor

Layer

FS_HRFS

Synopsis

```
STATUS hrfsAdvFormatFd
    (
    int    fd,             /* open file descriptor on disk */
    UINT64 diskSize,       /* size of disk in bytes */
    UINT32 blkSize,        /* size of block in bytes */
    UINT32 numInodes,      /* number of Inodes */
    UINT32 majorVersion,   /* file system version to format */
    UINT32 minorVersion,
    UINT32 options         /* misc options */
    )
```

Description

This routine formats a disk or partition referenced by an open file on the media. The file is closed when formatting is complete.

The fd argument should be a valid file descriptor representing the root directory of the current file system. This file descriptor will be marked closed and invalid upon return from this function reguardless of outcome.

The diskSize argument is used to specifiy how many bytes of the media the HRFS file system should occupy. It can be used to prevent HRFS for using the end portion of the media. In general this value should be 0 to specify that the entire media is to be used.

The blkSize parameter is used to specify what block size, in byte, HRFS should use. This block size must be a power of 2, greater than the physical sector size, and be within 512 to 32768 bytes inclusively. In general this value should be specified as 0 so the formatter can determine the most efficient block size to use for the media size.

The numInodes parameter is used to specify the absolute maximum number of files and directories the file system can ever have. Note this does not include the root directory which the formatter creates automatically. Specifiying a value of 0 will tell the HRFS formatter to allow for the maximum number of files/directories the file system can have based on the amount of data blocks. I.e. One inode per data block.

The majorVersion and minorVersion parameters are used to specify which particular version of the file system layout should be used when formatting the disk. A value of zero for both the major and minor version is used to indicate that the latest version of the file system should be used.

The options parameter is used to specify additional formatting options. It is currently unused

Returns

OK on success or **ERROR** on failure

Errno

Not Available

See also

hrfsFormatLib

Name

hrfsFormat( ) - format the HRFS file system via a path

Layer

FS_HRFS

Synopsis

```
STATUS hrfsFormat
    (
    char * path,        /* path to format */
    UINT64 diskSize,    /* size of disk in bytes */
    UINT32 blkSize,     /* size of block in bytes */
    UINT32 numInodes    /* number of Inodes */
    )
```

Description

This routine formats a disk or partition referenced by the path to the media. The routine will calculate and work with values of parameter diskSize, blkSize and numInodes, if these values are passed improper, format will fail. So it is strongly suggested to use 0 as parameter value so that format routine can find most proper parameter value by itself. In this case, disk space utilization will be the best. Of course customer can pass parameter value by himself, format may be ok. But disk space utilization should not be better and customer needs to adjust parameter value if format fails. So we suggest to use 0 as parameter value unless customer has a specific need for himself.

The path argument should be a valid path to the disk or partition to be formatted.

The diskSize argument is used to specifiy how many bytes of the media the HRFS file system should occupy. It can be used to prevent HRFS for using the end portion of the media. In general this value should be 0 to specify that the entire media is to be used. Non-zero value of diskSize may format successfully, but mount fail.

The blkSize parameter is used to specify what block size, in bytes, HRFS should use. This block size must be a power of 2, greater than the physical sector size, and be within 512 to 32768 bytes inclusively. In general this value should be specified as 0 so the formatter can determine the most efficient block size to use for the media size.

The numInodes parameter is used to specify the absolute maximum number of files and directories the file system can ever have. Note this does not include the root directory which the formatter creates automatically. Specifiying a value of 0 will tell the HRFS formatter to allow for the maximum number of files/directories the file system can have based on the amount of data blocks. I.e. One inode per data block.

Returns

OK on success or **ERROR** on failure.

Errno

Not Available

See also

hrfsFormatLib

Name

hrfsAdvFormat( ) - format the HRFS file system using advanced options

Layer

FS_HRFS

Synopsis

```
STATUS hrfsAdvFormat
    (
    char * path,            /* path to format */
    UINT64 diskSize,        /* size of disk in bytes */
    UINT32 blkSize,         /* size of block in bytes */
    UINT32 numInodes,       /* number of Inodes */
    UINT32 majorVersion,    /* file system version to format */
    UINT32 minorVersion,
    UINT32 options          /* misc options */
    )
```

Description

This routine formats a disk or partition referenced by the path to the media.

The path argument should be a valid path to the disk or partition to be formatted.

The diskSize argument is used to specifiy how many bytes of the media the HRFS file system should occupy. It can be used to prevent HRFS for using the end portion of the media. In general this value should be 0 to specify that the entire media is to be used.

The blkSize parameter is used to specify what block size, in byte, HRFS should use. This block size must be a power of 2, greater than the physical sector size, and be within 512 to 32768 bytes inclusively. In general this value should be specified as 0 so the formatter can determine the most efficient block size to use for the media size.

The numInodes parameter is used to specify the absolute maximum number of files and directories the file system can ever have. Note this does not include the root directory which the formatter creates automatically. Specifiying a value of 0 will tell the HRFS formatter to allow for the maximum number of files/directories the file system can have based on the amount of data blocks. I.e. One inode per data block.

The majorVersion and minorVersion parameters are used to specify which particular version of the file system layout should be used when formatting the disk. A value of zero for both the major and minor version is used to indicate that the latest version of the file system should be used.

The options parameter is used to specify additional formatting options. It is currently unused

Returns

OK on success or **ERROR** on failure.

Errno

Not Available

See also

hrfsFormatLib

19. HRFSLIB

Name

hrFsLib - highly reliable file system library

Layer

FS_HRFS

Routines

hrfsDevCreate( ) - create an HRFS device

Description

This library contains routines for creating and using the Highly Reliable File System (HRFS).

Include files

hrFsLib.h

Name

hrfsDevCreate( ) - create an HRFS device

Layer

FS_HRFS

Synopsis

```
HRFS_DEV_ID hrfsDevCreate
    (
    char     * pDevName,    /* Name of the HRFS device (mount point). */
    device_t   xbdId,       /* XBD for the device on which to mount. */
    int        numBufs,     /* # of [struct buf] to allocate. */
    int        maxFiles     /* Maximum # of simultaneously open files */
    )
```

Description

This routine creates an HRFS device.

Returns

HRFS_DEV_ID if created and installed in Core I/O, **NULL** if not.

Errno

Not Available

See also

hrFsLib

20. HRFSTIMELIB

Name

hrFsTimeLib - time routines for HRFS

Layer

FS_HRFS

Routines

hrfsTimeGet( ) - return # of milliseconds since midnight Jan 1, 1970 **hrfsTimeSplit( )** - split time in msec into **HRFS_TM** format **hrfsTimeCondense( )** - condense time in **HRFS_TM** to time in msec

Description

This library contains routines for handling the HRFS timestamps.

Include files

none

Name

hrfsTimeGet( ) - return # of milliseconds since midnight Jan 1, 1970

Layer

FS_HRFS

Synopsis

```
hrfsTime_t hrfsTimeGet(void)
```

Description

This routine returns the number of milliseconds since midnight, January 1, 1970.

Returns

of milliseconds since midnight, January 1, 1970

Errno

Not Available

See also

hrFsTimeLib

Name

hrfsTimeSplit( ) - split time in msec into **HRFS_TM** format

Layer

FS_HRFS

Synopsis

```
STATUS hrfsTimeSplit
    (
    INT64     milliSeconds,   /* milliseconds to convert to HRFS_TM format */
    HRFS_TM * pHrfsTm         /* Buffer to store HRFS_TM format time */
    )
```

Description

This routine splits the time specified in milliseconds into the "broken-down" format of the **HRFS_TM** structure. Should the equivalent number of seconds exceed what can be represent by a signed integer, this routine will currently return **ERROR**, and the split will not have occurred.

Returns

OK success, or **ERROR** if the split did not occur

Errno

Not Available

See also

hrFsTimeLib

Name

hrfsTimeCondense( ) - condense time in **HRFS_TM** to time in msec

Layer

FS_HRFS

Synopsis

```
INT64 hrfsTimeCondense
    (
    HRFS_TM * pHrfsTm   /* Pointer to where HRFS_TM format time is stored */
    )
```

Description

This routine condenses the "broken-down" time pointed to by pHrfsTm into the number of milliseconds since midnight Jan 1, 1970.

Returns

of milliseconds since midnight Jan 1, 1970

Errno

Not Available

See also

hrFsTimeLib

21. LOOPFSLIB

Name

loopFsLib - Loop File System Library

Layer

BDM_LOOPFS

Routines

loopFsMountFile( ) - mount a image file to a device **loopFsMountImgFile( )** - mount a flat image file to a device
loopFsUnmount( ) - unmount a flat image file

Description

This module is a common library for Loop FS. This module includes basic Loop FS APIs and frameworks and a simple flat Loop FS support.

For a flat image file, the whole file will be mounted as a block device with 512 bytes per sector. All bios will be translated to file read request. Write file does not support.

For a encrypt image file, external xbd functions are needed when trying to mount image file.

Include files

[**loopFsLib.h**](https://docs.windriver.com/bundle/vxworks_7_file_systems_and_IO_programmers_guide_sr0650/page/eri1450462435754.html)

See also

VxWorks 7 File Systems and I/O Programmer's Guide, Loop File System and Tcplay

Name

loopFsMountFile( ) - mount a image file to a device

Layer

BDM_LOOPFS

Synopsis

```
void * loopFsMountFile
    (
    char *              imageName,
    char *              mountPath,
    struct xbd_funcs * xbdFuncs,
    LOOP_FS_MOUNTPARAM *mountParam
    )
```

Description

This routine mounts a image file to a device, and sends ERF event to FS monitor. For a flat image file, only give imageName and mountPath will work. For a encrypt image file, all parameters are needed.

Returns

Loop FS Node handle or **NULL** if there is any failure.

Errno

N/A

See also

loopFsLib

Name

loopFsMountImgFile( ) - mount a flat image file to a device

Layer

BDM_LOOPFS

Synopsis

```
void * loopFsMountImgFile
    (
    char * imageName,
    char * mountPath
    )
```

Description

This routine mounts a flat image file to a device, and sends ERF event to FS monitor. It packages routine loopFsMountFile with less parameters.

Returns

Loop FS Node handle or **NULL** if there is any failure.

Errno

N/A

See also

loopFsLib

Name

loopFsUnmount( ) - unmount a flat image file

Layer

BDM_LOOPFS

Synopsis

```
STATUS loopFsUnmount
    (
    void * loopFsHandle
    )
```

Description

This routine unmounts a flat image file by a specific handle of Loop FS.

Returns

OK, or **ERROR** if there is any failure.

Errno

N/A

See also

loopFsLib

22. MOUNTD

Name

mountd - Mount protocol library

Layer

FS_NFS

Routines

mountdInit( ) - initialize the mount daemon **nfsExport( )** - specify a file system to be NFS exported **nfsUnexport( )** - remove a file system from the list of exported file systems

Description

This file implements the initialization routines for the MOUNT daemon.

The mount server is initialized by calling **mountdInit( )**. This routine, will be invoked by **nfsdInit( )**.

Including any of the NFS SERVER components (**INCLUDE_NFS_SERVER_ALL**, **INCLUDE_NF3_SERVER**, **INCLUDE_NFS2_SERVER**) enables the call to **nfsdInit( )** during the boot process, which in turn calls **mountdInit( )**, so there is normally no need to call either routine manually. **mountdInit( )** spawns one task, **tMountd**, which registers as an RPC service with the portmapper. The registration is done for MOUNT V1 and/or MOUNT V3 depending on the NFS server version included. If only NFS V3 server is included then both MOUNT V1 and MOUNT V3 are registered. If only NFS V2 is included then only the MOUNT V1 is registered.

NOTE: The only routines in this library that are normally called by applications are **nfsExport( )** and **nfsUnexport( )**. The mount daemon is normally initialized indirectly by **nfsdInit( )**.

File systems are exported with the **nfsExport( )** call.

To export VxWorks file systems via NFS, you need facilities from both this library and from **nfsdLib**. To include the **nfsLib** components, use either **INCLUDE_NFS2_CLIENT** or **INCLUDE_NFS3_CLIENT** or **INCLUDE_NFS_CLIENT_ALL** and rebuild VxWorks.

Example

The following example illustrates how to export an existing dosFs file system.

First, initialize the block device containing your file system.

Then assuming the dosFs system is called "/export" execute the following code on the target:

```
nfsExport ("/export", 0, FALSE, 0);           /* make available remotely */
```

This makes it available to all clients to be mounted using the client's NFS mounting command. (On UNIX systems, mounting file systems normally requires root privileges.)

VxWorks does not normally provide authentication services for NFS requests, and the DOS file system does not provide file permissions. If you need to authenticate incoming requests, see the documentation for **nfsdInit( )** and **mountdInit( )** for information about authorization hooks.

The following requests are accepted from clients. For details of their use, see Appendix I of RFC 1813, "NFSv3 Protocol Specification."

Procedure Name	Procedure Number
MOUNTPROC_NULL	0
MOUNTPROC_MNT	1
MOUNTPROC_DUMP	2
MOUNTPROC_UMNT	3
MOUNTPROC_UMNTALL	4
MOUNTPROC_EXPORT	5

Include files

nfs/mountd.h

See also

dosFsLib, nfsCommon, RFC 1813, VxWorks 7 File Systems and I/O Programmer's Guide, Overview of NFS

Name

mountdInit( ) - initialize the mount daemon

Layer

FS_NFS

Synopsis

```
STATUS mountdInit
    (
    int     priority,    /* priority of the mount daemon            */
    int     stackSize,   /* stack size of the mount daemon          */
    FUNCPTR authHook,    /* hook to run to authorize each request   */
    int     nExports,    /* maximum number of exported file systems */
    int     options      /* Currently unused                        */
    )
```

Description

This routine spawns a mount daemon if one does not already exist. Defaults for the priority and stackSize arguments are in the global variables **mountdPriorityDefault** and **mountdStackSizeDefault**, and are initially set to **MOUNTD_PRIORITY_DEFAULT** and **MOUNTD_STACKSIZE_DEFAULT** respectively.

Normally, no authorization checking is performed by either mountd or nfsd. To add authorization checking, set authHook to point to a routine declared as follows:

WNDRVR

```
nfsstat routine
    (
    int                 progNum,       /* RPC program number */
    int                 versNum,       /* RPC program version number */
    int                 procNum,       /* RPC procedure number */
    struct sockaddr_in clientAddr,     /* address of the client */
    void            *   mountdArg      /* argument of the call */
    )
```

The mountdArg will be of type MOUNT3D_ARGUMENT when versNum is 3 and it will be of type **MOUNTD_ARGUMENT** when versNum is 1. The user routine must typecast the mountdArg accoringly and use it. The definitions of MOUNT3D_ARGUMENT & **MOUNTD_ARGUMENT** are available in **mountd.h** file.

The authHook callback must return **OK** if the request is authorized, and any defined NFS error code (usually **NFSERR_ACCESS**) if not.

Returns

OK, or **ERROR** if the mount daemon could not be correctly initialized.

Errno

Not Available

See also

mountd

Name

nfsExport( ) - specify a file system to be NFS exported

Layer

FS_NFS

Synopsis

```
STATUS nfsExport
    (
    char * directory,   /* Directory to export - FS must support NFS */
    int    id,          /* ID number for file system */
    BOOL   readOnly,    /* TRUE if file system is exported read-only */
    int    options      /* Reserved for future use - set to 0 */
    )
```

Description

This routine makes a file system available for mounting by a client. The client should be in the local host table (see **hostAdd( )**), although this is not required.

The id parameter can either be set to a specific value, or to 0. If it is set to 0, an ID number is assigned sequentially. Every time a file system is exported, it must have the same ID number, or clients currently mounting the file system will not be able to access files.

Note

exporting a file system requires at least 512kb of free space available on the file system for creation of configuration files.

To display a list of exported file systems, use:

```
    -> nfsExportShow "localhost"
```

Returns

OK, or **ERROR** if the file system could not be exported.

Errno

Not Available

See also

mountd, **nfsLib**, **nfsExportShow( )**, **nfsUnexport( )**

Name

nfsUnexport( ) - remove a file system from the list of exported file systems

Layer

FS_NFS

Synopsis

```
STATUS nfsUnexport
    (
    char * dirName   /* Name of the directory to unexport */
    )
```

Description

This routine removes a file system from the list of file systems exported from the target. Any client attempting to mount a file system that is not exported will receive an error (**NFSERR_ACCESS**).

Returns

OK, or **ERROR** if the file system could not be removed from the exports list.

Errno

ENOENT

See also

mountd, **nfsLib**, **nfsExportShow( )**, **nfsExport( )**

23. MTDSHOW

Name

mtdShow - MTD show routines

Layer

BDM_FLASH

Routines

mtdDevShow( ) - show MTD device information

Description

This library provides the interfaces which can be used for displaying information about the MTD module.

The MTD show facility can be configured into VxWorks by **INCLUDE_MTD_SHOW** when **INCLUDE_MTD** component is already configured:

Routines in this library are meant to be used as debugging aids that display MTD information to standard output.

Include files

mtdCore.h

See also

mtdCore.c, VxWorks 7 File Systems and I/O Programmer's Guide, VxWorks 7 BSP and Driver Guide

Name

mtdDevShow( ) - show MTD device information

Layer

BDM_FLASH

Synopsis

```
void mtdDevShow (void)
```

Description

This routine shows the MTD device information.

Returns

N/A

Errno

N/A

See also

mtdShow

24. NFSCOMMON

Name

nfsCommon - Network File System (NFS) I/O driver

Layer

FS_NFS

Routines

nfsMount( ) - mount an NFS file system **nfsMountAll( )** - mount all file systems exported by a specified host **nfsDevShow( )** - display the mounted NFS devices **nfsUnmount( )** - unmount an NFS device **nfsDevInfoGet( )** - read configuration information from the requested device **nfsDevListGet( )** - create list of all the NFS devices in the system **nfsDrvNumGet( )** - Get driver number of NFS device **nfsMntDump( )** - display all NFS file systems mounted on a particular host **nfsExportShow( )** - display the exported file systems of a remote host **nfsHelp( )** - display the NFS help menu **nfsAuthUnixPrompt( )** - modify the NFS UNIX authentication parameters **nfsAuthUnixShow( )** - display the NFS UNIX authentication parameters **nfsAuthUnixSet( )** - set the NFS UNIX authentication parameters **nfsAuthUnixGet( )** - get the NFS UNIX authentication parameters **nfsIdSet( )** - set the ID number of the NFS UNIX authentication parameters **nfsChkFilePerms( )** - check the NFS file permissions with a given permission. **nfsErrnoSet( )** - set NFS status

Description

This driver provides facilities for accessing files transparently over the network via NFS (Network File System). By creating a network device with **nfsMount( )**, files on a remote NFS system (such as a UNIX system) can be handled as if they were local.

User-callable routines

The **nfs2Drv( )** routine initializes the NFS version 2 driver. The **nfs3Drv( )** routine initializes the NFS version 3 driver. The **nfsMount( )** and **nfsUnmount( )** routines mount and unmount file systems. The **nfsMountAll( )** routine mounts all file systems exported by a specified host.

Initialization

Before using the NFS v2 driver, it must be initialized by calling **nfs2Drv( )**. Before using the NFS v3 driver, it must be initialized by calling **nfs3Drv( )**. These routines must be called before any reads, writes, or other NFS calls. This is done automatically by adding the component **INCLUDE_NFS2_CLIENT**, **INCLUDE_NFS3_CLIENT** or **INCLUDE_NFS_CLIENT_ALL** (as appropriate) to your project.

Creating nfs devices

In order to access a remote file system, an NFS device must be created by calling **nfsMount( )**. For example, to create the device **/myd0/** for the file system **/d0/** on the host **wrs**, call:

```
    nfsMount ("wrs", "/d0/", "/myd0/");
```

The file **/d0/dog** on the host **wrs** can now be accessed as **/myd0/dog**.

Before mounting a file system, the host must already have been created with **hostAdd( )**. The routine **nfsDevShow( )** displays the mounted NFS devices.

Include files

nfs/nfsCommon.h

See also

dirLib, **hostAdd( )**, **ioctl( )**, VxWorks 7 File Systems and I/O Programmer's Guide Overview of NFS

Name

nfsMount( ) - mount an NFS file system

Layer

FS_NFS

Synopsis

```
STATUS nfsMount
    (
    const char *host,          /* name of remote host */
    const char *fileSystem,    /* name of remote directory to mount */
    const char *localName      /* local device name for remote dir */
    )
```

Description

This routine mounts a remote file system. It creates a local device localName for a remote file system on a specified host. The host must have already been added to the local host table with **hostAdd( )**.

Returns

OK, or **ERROR** if the driver is not installed, host is invalid, or memory is insufficient.

Errno

S_nfsLib_NFSERR_INVAL

Provided arguments are invalid.

S_objLib_OBJ_UNAVAILABLE

This operation is not supported by the included NFS versions.

S_nfsLib_NFSERR_NOTSUPP

Remote system does not have a compatible mount version.

See also

nfsCommon, **nfsUnmount( )**, **hostAdd( )**

Name

nfsMountAll( ) - mount all file systems exported by a specified host

Layer

FS_NFS

Synopsis

```
STATUS nfsMountAll
    (
    const char *pHostName,      /* name of remote host */
    const char *pClientName,    /* name of a client specified in access list, if any */
    BOOL       quietFlag        /* FALSE = print name of each mounted file system */
    )
```

Description

This routine mounts the file systems exported by the host pHostName which are accessible by pClientName. A pClientName entry of **NULL** will only mount file systems that are accessible by any client. The **nfsMount( )** routine is called to mount each file system. It creates a local device for each mount that has the same name as the remote file system.

If the quietFlag setting is **FALSE**, each file system is printed on standard output after it is mounted successfully.

Returns

OK, or **ERROR** if any mount fails.

Errno

S_nfsLib_NFSERR_INVAL

Invalid arguments.

S_objLib_OBJ_UNAVAILABLE

This operation is not supported by the included NFS versions.

S_nfsLib_NFSERR_NOTSUPP

Incompatible mount version on remote system.

See also

nfsCommon, **nfsMount( )**

Name

nfsDevShow( ) - display the mounted NFS devices

Layer

FS_NFS

Synopsis

```
void nfsDevShow (void)
```

Description

This routine displays the device names and their associated NFS file systems.

Example

```
    -> nfsDevShow
    device name          file system
    -----------          -----------
    /yuba1/              yuba:/yuba1
    /wrs1/               wrs:/wrs1
```

Returns

N/A

Errno

Not Available

See also

nfsCommon

Name

nfsUnmount( ) - unmount an NFS device

Layer

FS_NFS

Synopsis

```
STATUS nfsUnmount
    (
    const char *localName   /* local of nfs device */
    )
```

Description

This routine unmounts file systems that were previously mounted via NFS.

Returns

OK, or **ERROR** if localName is not an NFS device or cannot be mounted.

Errno

S_nfsLib_NFSERR_INVAL

localName is invalid.

S_nfsDrv_NOT_AN_NFS_DEVICE

localName is not an NFS device.

S_objLib_OBJ_UNAVAILABLE

This operation is not supported by the included NFS versions.

See also

nfsCommon, **nfsMount( )**

Name

nfsDevInfoGet( ) - read configuration information from the requested device

Layer

FS_NFS

Synopsis

```
STATUS nfsDevInfoGet
    (
    unsigned long  nfsDevHandle,   /* NFS device handle */
    NFS_DEV_INFO * pnfsInfo        /* ptr to struct to hold config info */
    )
```

Description

This routine accesses the NFS device specified in the parameter nfsDevHandle and fills in the structure pointed to by pnfsinfo. The calling function should allocate memory for pnfsinfo and for the two character buffers "remFileSys" and "locFileSys" , that are part of pnfsInfo. These buffers should have a size of nfsMaxPath

Returns

OK, if pnfsInfo information is valid, otherwise **ERROR**

Errno

S_objLib_OBJ_UNAVAILABLE

This operation is not supported by the available NFS versions.

See also

nfsCommon, **nfsDevListGet( )**

Name

nfsDevListGet( ) - create list of all the NFS devices in the system

Layer

FS_NFS

Synopsis

```
int nfsDevListGet
    (
    unsigned long nfsDevList[],   /* NFS dev list of handles */
    int           listSize        /* number of elements available in the list */
    )
```

Description

This routine fills the array nfsDevlist up to listSize, with handles to NFS devices currently in the system.

Returns

The number of entries filled in the nfsDevList array.

Errno

N/A

See also

nfsCommon, **nfsDevInfoGet( )**

Name

nfsDrvNumGet( ) - Get driver number of NFS device

Layer

FS_NFS

Synopsis

```
STATUS nfsDrvNumGet
    (
    int version   /* Version number of NFS */
    )
```

Description

This routine returns the NFS driver number for the version requested. If the user specifies NFS version 2, this routine will return the value stored in variable nfs2DrvNum. If the user specifies NFS version 3, this routine will return the value stored in the variable nfs3DrvNum.

If the NFS driver of the user-specified version is yet to be initialized, or if initialization failed, nfsDrvNumGet will return **ERROR**.

Returns

Returns the NFS driver number or **ERROR**.

Errno

S_objLib_OBJ_UNAVAILABLE

This version does not support this operation.

See also

nfsCommon

Name

nfsMntDump( ) - display all NFS file systems mounted on a particular host

Layer

FS_NFS

Synopsis

```
STATUS nfsMntDump
    (
    const char *hostName   /* host machine */
    )
```

Description

This routine displays all the NFS file systems mounted on a specified host machine.

Returns

OK or **ERROR**.

Errno

S_nfsLib_NFSERR_INVAL

hostName is invalid.

S_nfsLib_NFSERR_NOTSUPP

Remote system does not have a compatible mount version.

S_objLib_OBJ_UNAVAILABLE

This routine is not supported by the included NFS versions.

See also

nfsCommon

Name

nfsExportShow( ) - display the exported file systems of a remote host

Layer

FS_NFS

Synopsis

```
STATUS nfsExportShow
    (
    char *hostName   /* host machine to show exports for */
    )
```

Description

This routine displays the file systems of a specified host and the groups that are allowed to mount them.

Example

```
   -> nfsExportShow "wrs"
   /d0                staff
   /d1                staff eng
   /d2                eng
   /d3
   value = 0 = 0x0
```

Returns

OK or **ERROR**.

Errno

S_hostLib_INVALID_PARAMETER

hostName is invalid.

S_objLib_OBJ_UNAVAILABLE

This operation is not supported by the available NFS versions.

S_nfsLib_NFSERR_NOTSUPP

Remote system does not have a compatible mount version.

See also

nfsCommon

Name

nfsHelp( ) - display the NFS help menu

Layer

FS_NFS

Synopsis

```
void nfsHelp (void)
```

Description

This routine displays a summary of NFS facilities typically called from the shell:

```
    nfsHelp                        Print this list
    netHelp                        Print general network help list
    nfsMount "host","filesystem"[,"devname"]  Create device with
                                     file system/directory from host
    nfsUnmount "devname"           Remove an NFS device
    nfsAuthUnixShow                Print current UNIX authentication
    nfsAuthUnixPrompt              Prompt for UNIX authentication
    nfsIdSet id                    Set user ID for UNIX authentication
    nfsDevShow                     Print list of NFS devices
    nfsExportShow "host"           Print a list of NFS file systems which
                                     are exported on the specified host
    mkdir "dirname"                Create directory
    rm "file"                      Remove file

    EXAMPLE:  -> hostAdd "wrs", "90.0.0.2"
              -> nfsMount "wrs","/disk0/path/mydir","/mydir/"
              -> cd "/mydir/"
              -> nfsAuthUnixPrompt     /* fill in user ID, etc.     */
              -> ls                    /* list /disk0/path/mydir    */
              -> copy < foo            /* copy foo to standard out  */
              -> ld < foo.o            /* load object module foo.o  */
              -> nfsUnmount "/mydir/"  /* remove NFS device /mydir/ */
```

Returns

N/A

Errno

Not Available

See also

nfsCommon

Name

nfsAuthUnixPrompt( ) - modify the NFS UNIX authentication parameters

Layer

FS_NFS

Synopsis

```
void nfsAuthUnixPrompt (void)
```

Description

This routine allows UNIX authentication parameters to be changed from the shell. The user is prompted for each parameter, which can be changed by entering the new value next to the current one.

Example

```
   -> nfsAuthUnixPrompt
   machine name:   yuba
   user ID:        2001 128
   group ID:       100
   num of groups:  1 3
   group #1:        100 100
   group #2:        0 120
   group #3:        0 200
   value = 3 = 0x3
```

Returns

Not Available

Errno

Not Available

See also

nfsCommon, **nfsAuthUnixShow( )**, **nfsAuthUnixSet( )**, **nfsAuthUnixGet( )**, **nfsIdSet( )**

Name

nfsAuthUnixShow( ) - display the NFS UNIX authentication parameters

Layer

FS_NFS

Synopsis

```
void nfsAuthUnixShow (void)
```

Description

This routine displays the parameters set by **nfsAuthUnixSet( )** or **nfsAuthUnixPrompt( )**.

Example

```
-> nfsAuthUnixShow
machine name = yuba
user ID      = 2001
group ID     = 100
group [0]    = 100
value = 1 = 0x1
```

Returns

N/A

Errno

Not Available

See also

nfsCommon, **nfsAuthUnixPrompt( )**, **nfsAuthUnixSet( )**, **nfsAuthUnixGet( )**, **nfsIdSet( )**

Name

nfsAuthUnixSet( ) - set the NFS UNIX authentication parameters

Layer

FS_NFS

Synopsis

```
void nfsAuthUnixSet
    (
    char *machname,   /* host machine        */
    int  uid,         /* user ID             */
    int  gid,         /* group ID            */
    int  ngids,       /* number of group IDs */
    int  *aup_gids    /* array of group IDs  */
    )
```

Description

This routine sets UNIX authentication parameters. It is initially called by **usrNetInit( )**. **machname** should be set with the name of the mounted system (i.e. the target name itself) to distinguish hosts from hosts on a NFS network.

Returns

N/A

Errno

Not Available

See also

nfsCommon, **nfsAuthUnixPrompt( )**, **nfsAuthUnixShow( )**, **nfsAuthUnixGet( )**, **nfsIdSet( )**

Name

nfsAuthUnixGet( ) - get the NFS UNIX authentication parameters

Layer

FS_NFS

Synopsis

```
void nfsAuthUnixGet
    (
    char *machname,   /* where to store host machine        */
    int  *pUid,       /* where to store user ID             */
    int  *pGid,       /* where to store group ID            */
    int  *pNgids,     /* where to store number of group IDs */
    int  *gids        /* where to store array of group IDs  */
    )
```

Description

This routine gets the previously set UNIX authentication values.

Returns

N/A

Errno

Not Available

See also

nfsCommon, **nfsAuthUnixPrompt( )**, **nfsAuthUnixShow( )**, **nfsAuthUnixSet( )**, **nfsIdSet( )**

Name

nfsIdSet( ) - set the ID number of the NFS UNIX authentication parameters

Layer

FS_NFS

Synopsis

```
void nfsIdSet
    (
    int uid   /* user ID on host machine */
    )
```

Description

This routine sets only the UNIX authentication user ID number. For most NFS permission needs, only the user ID needs to be changed. Set uid to the user ID on the NFS server.

Returns

N/A

Errno

Not Available

See also

nfsCommon, **nfsAuthUnixPrompt( )**, **nfsAuthUnixShow( )**, **nfsAuthUnixSet( )**, **nfsAuthUnixGet( )**

Name

nfsChkFilePerms( ) - check the NFS file permissions with a given permission.

Layer

FS_NFS

Synopsis

```
STATUS nfsChkFilePerms
    (
    int nfsPerms,   /* permissions of the opened file */
    int ruid,       /* Remote uid */
    int rgid,       /* Remote gid */
    int perm        /* permission to be checked for 4:read 2:write 1:execute */
    )
```

Description

This routine compares the NFS file permissions with a given permission.

This routine is basically designed for **nfsOpen( )** to verify the target file's permission prior to deleting it due to **O_TRUNC**.

The parameter perm will take 4(read), 2(write), 1(execute), or combinations of them.

OK means the file has valid permission whichever group is.

Called only by the I/O system.

Returns

OK, **ERROR**

Errno

Not Available

See also

nfsCommon

Name

nfsErrnoSet( ) - set NFS status

Layer

FS_NFS

Synopsis

```
void nfsErrnoSet
    (
    enum nfsstat status
    )
```

Description

nfsErrnoSet calls **errnoSet( )** with the errno value corresponding to NFS status prefix status. The value chosen depends upon whether NFS is configured to use POSIX errno values (when possible), or configured to use the historical VxWorks-specific NFS errno values.

In the latter case, or when there is no common POSIX errno with a closely related meaning, the M_nfsStat module number is OR'ed with the status value to form the VxWorks errno value.

Returns

N/A

Errno

Not Available

See also

nfsCommon

25. NFSD

Name

nfsd - NFS Server Init routines

Layer

FS_NFS

Routines

nfsdInit( ) - initialize the NFS server **nfsdStart( )** - start the NFS server

Description

This file implements the initialization routines for the NFS daemon.

The nfs server is initialized by calling **nfsdInit( )**.

Including any of the NFS SERVER components (**INCLUDE_NFS_SERVER_ALL**, **INCLUDE_NFS3_SERVER**, **INCLUDE_NFS2_SERVER**) enables the call to **nfsdInit( )** during the boot process. So there is normally no need to call the routine manually.

The **nfsdInit( )** spawns several tasks. (see details below). The **nfsd( )** daemon registers the NFS services with the portmapper daemon. The registration is done depending on the components included by the user during vxWorks build.

If user includes **INCLUDE_NFS_SERVER_ALL**, nfsd will register NFS V2 and V3 services.

If user includes **INCLUDE_NFS2_SERVER** nfsd will register only NFS V2 service.

If user includes **INCLUDE_NFS3_SERVER** nfsd will register only NFS V3 service.

With the registration of NFS V3, MOUNT v1, v3 and NLM v4 are also registered.

With the registration of NFS V3 MOUNT v1 is also registered.

Authentication and permissions

Currently, no authentication is done on NFS requests. **nfsdInit( )** describes the authentication hooks that can be added should authentication be necessary.

Note that the DOS file system does not provide information about ownership or permissions on individual files.

Tasks

Several NFS tasks are created by **nfsdInit( )**. They are:

tMountd

The mount daemon, which handles all incoming mount requests. This daemon is created by **mountdInit( )**, which is called from **nfsdInit( )**.

tNfsd

The NFS daemon, which queues all incoming NFS requests.

tNfsdX

The NFS request handlers, which dequeues and processes all incoming NFS requests.

Include files

nfs/nfsd.h

See also

VxWorks 7 File Systems and I/O Programmer's Guide Overview of NFS

Name

nfsdInit( ) - initialize the NFS server

Layer

FS_NFS

Synopsis

```
STATUS nfsdInit
    (
    int     nServers,        /* the number of NFS servers to create */
    int     nExportedFs,     /* maximum number of exported file systems */
    int     priority,        /* the priority for the NFS servers */
    FUNCPTR authHook,        /* Authentication hook */
    FUNCPTR mountAuthHook,   /* authentication hook for mount daemon */
    int     options          /* 3 bits used only */
    )
```

Description

This routine initializes the NFS server. nServers specifies the number of Tasks to be spawned to handle NFS requests. priority is the priority that those tasks will run at. authHook is a pointer to an authorization routine. mountAuthHook is a pointer to a similar routine, passed to **mountdInit( )**. options (only 3 LSBs are used for specifying the NFS version). Currently options can take the following values. 0x01 to start the NFS V2 service only. 0x02 to start the NFS V3 service only. 0x00 to start the NFS V2 and V3 services (default)

Normally, no authorization is performed by either mountd or nfsd. If you want to add authorization, set authHook to a function pointer to a routine declared as follows:

```
nfsstat routine
    (
    int                progNum,       /* RPC program number */
    int                versNum,       /* RPC program version number */
    int                procNum,       /* RPC procedure number */
    struct sockaddr_in clientAddr,    /* address of the client */
    NFSD_ARGUMENT *    nfsdArg        /* argument of the call */
    )
```

The nfsdArg will be of type "**NFSD_ARGUMENT**" if versNum is 2. The nfsdArg will be of type "NFS3D_ARGUMENT" if versNum is 3. The user authentication hook must use the nfsdArg accordingly.

The authHook routine should return **NFS_OK** if the request is authorized, and NFS3ERR_ACCES if not. (**NFSERR_ACCESS** is not required; any legitimate NFS error code can be returned.)

See **mountdInit( )** for documentation on mountAuthHook. Note that mountAuthHook and authHook can point to the same routine. Simply use the progNum, versNum, and procNum fields to decide whether the request is an NFS request or a mountd request.

Returns

OK, or **ERROR** if the NFS server cannot be started.

Errno

Not Available

See also

nfsd, **nfsExport( )**, **mountdInit( )**

Name

nfsdStart( ) - start the NFS server

Layer

FS_NFS

Synopsis

```
STATUS nfsdStart (void)
```

Description

This function starts the Network File System (NFS) server. If the **INCLUDE_NFSD_START_DELAY** component is included in your system, the NFS server does not auto-start during the boot process. To start the NFS server, call this function.

Returns

OK or **ERROR**

Errno

Not Available

See also

nfsd

26. NFSDCOMMON

Name

nfsdCommon - Common functions for v2 and v3

Layer

FS_NFS

Routines

entryStatGet( ) - will replace stat for NFS **entryStat( )** - will replace stat for NFS **opendirplus( )** - open a directory for searching (POSIX) **readdirplus( )** - read one entry with extended data stat. **closedirplus( )** - close a directory **nfsStatusGet( )** - Get the statistics of the NFS server **nfsdStatusShow( )** - show the status of the NFS server

Description

This file implements the common routines that will be used by the NFS version 2 and NFS version 3 procedures.

User-callable routines

The **nfsStatusGet( )** routine gets the statistis of NFS procedure calls. The **nfsdStatusShow( )** routine displays the statistics of NFS procedure calls.

Include files

none

See also

VxWorks 7 File Systems and I/O Programmer's Guide Overview of NFS

Name

entryStatGet( ) - will replace stat for NFS

Layer

FS_NFS

Synopsis

```
STATUS entryStatGet
    (
    char *      fileName,
    struct stat *pStat
    )
```

Description

Given an NFS file handle, and filename this routine gets the **stat** structure for it.

Returns

OK on success, **ERROR** otherwise

Errno

Not Available

See also

nfsdCommon

Name

entryStat( ) - will replace stat for NFS

Layer

FS_NFS

Synopsis

```
STATUS entryStat
    (
    NFS_FILE_HANDLE * pFh,
    char *            fileName,
    struct stat       *pStat
    )
```

Description

Given an NFS file handle, and filename this routine gets the **stat** structure for it.

Returns

OK on success, **ERROR** otherwise

Errno

Not Available

See also

nfsdCommon

Name

opendirplus( ) - open a directory for searching (POSIX)

Layer

FS_NFS

Synopsis

```
DIRPLUS *opendirplus
    (
    const char* dirName   /* name of directory to open */
    )
```

Description

This routine opens the directory named by dirName and allocates a directory descriptor (DIRPLUS) for it. A pointer to the DIRPLUS structure is returned. The return of a **NULL** pointer indicates an error.

After the directory is opened, **readdirplus( )** is used to extract individual directory entries. Finally, **closedirplus( )** is used to close the directory.

Warning

For remote file systems mounted over **netDrv**, **opendirplus( )** fails, because the **netDrv** implementation strategy does not provide a way to distinguish directories from plain files. To permit use of **opendirplus( )** on remote files, use NFS rather than **netDrv**.

Returns

A pointer to a directory descriptor, or **NULL** if there is an error.

Errno

N/A.

See also

nfsdCommon, **closedirplus( )**, **readdirplus( )**, **rewinddir( )**, **ls( )**

Name

readdirplus( ) - read one entry with extended data stat.

Layer

FS_NFS

Synopsis

```
struct dirent *readdirplus
    (
    DIRPLUS *pDir   /* pointer to directory descriptor */
    )
```

Description

This routine obtains directory entry data for the next file from an open directory. The pDir parameter is the pointer to a directory descriptor (DIRPLUS) which was returned by a previous **opendirplus( )**.

This routine returns a pointer to a **dirent** structure which contains the name of the next file. Empty directory entries and MS-DOS volume label entries are not reported. The name of the file (or subdirectory) described by the directory entry is returned in the **d_name** field of the **dirent** structure. The name is a single null-terminated string.

The returned **dirent** pointer will be **NULL**, if it is at the end of the directory or if an error occurred. Because there are two conditions which might cause **NULL** to be returned, the task's error number (**errno**) must be used to determine if there was an actual error. Before calling **readdir( )**, set **errno** to **OK**. If a **NULL** pointer is returned, check the new value of **errno**. If **errno** is still **OK**, the end of the directory was reached; if not, **errno** contains the error code for an actual error which occurred.

Returns

A pointer to a **dirent** structure, or **NULL** if there is an end-of-directory marker or error from the IO system.

Errno

EBADF

Bad file descriptor number.

ENOTSUP

Device driver does not support the ioctl command.

Other

Other errors reported by device driver.

See also

nfsdCommon, **opendirplus( )**, **readdir_r( )**, **closedirplus( )**, **rewinddir( )**, **ls( )**

Name

closedirplus( ) - close a directory

Layer

FS_NFS

Synopsis

```
STATUS closedirplus
    (
    DIRPLUS *pDir   /* pointer to directory descriptor */
    )
```

Description

This routine closes a directory which was previously opened using **opendirplus( )**. The pDir parameter is the directory descriptor pointer that was returned by **opendirplus( )**.

Returns

OK or **ERROR**, the result of the **close( )** command.

Errno

EBADF

Invalid file descriptor.

Others

Other errors generated by device drivers.

See also

nfsdCommon, **opendirplus( )**, **readdirplus( )**, **rewinddir( )**

Name

nfsStatusGet( ) - Get the statistics of the NFS server

Layer

FS_NFS

Synopsis

```
STATUS nfsStatusGet
    (
    void * serverStats,   /* pointer to status struct */
    int    version        /* NFS v2 or NFS v3 */
    )
```

Description

This routine returns the statistics of the NFS procedure calls made by the remote NFS clients serverStats pointer to a memory location where the statistics information will be copied. version Statistics of which NFS version is desired. This parameter takes two values only. 0x01 for NFS Version 2 and 0x02 for NFS version 3.

Returns

OK or **ERROR** if version is invalid.

Errno

Not Available

See also

nfsdCommon

Name

nfsdStatusShow( ) - show the status of the NFS server

Layer

FS_NFS

Synopsis

```
STATUS nfsdStatusShow
    (
    int options
    )
```

Description

This routine shows statistics of procedure calls to the NFS server. This routine takes one parameter to specify the NFS server version whose statistics are to be shown. options takes one of the following 3 values. 0x01 Display statistics of NFS version 2 server only. 0x02 Display statistics of NFS version 3 server only. 0x00 Display statistics of NFS version 2 & 3.

Returns

OK, or **ERROR** if the information cannot be obtained.

Errno

Not Available

See also

nfsdCommon

27. NFSHASH

Name

nfsHash - file based hash table for file handle to file name and reverse

Layer

FS_NFS

Routines

nfsdHashTableParamsSet( ) - sets up the parameters for the NFS hash table

Description

This library implements the hash table maintained by the NFS server.

Include files

none

See also

VxWorks 7 File Systems and I/O Programmer's Guide Overview of NFS

Name

nfsdHashTableParamsSet( ) - sets up the parameters for the NFS hash table

Layer

FS_NFS

Synopsis

```
void nfsdHashTableParamsSet
    (
    int    bucketSize,
    int    tableLen,
    char * basePath
    )
```

Description

This function must be called prior to the call to nfsExport, and can be called repeatedly to configure different exports differently.

bucketSize represents the number of bytes available in each hash bucket. The selectable sizes are 512, 1k, 2k, 4k, 8k, 16k. It is probably a good idea to make this value the same size as a disk allocation unit in general. Never smaller, but perhaps larger if you plan on accessing lots of files with very long filenames. Note, that the code always keeps one bucket in memory, and reads and writes whole buckets at a time. So larger buckets can impact performance. A value of zero preserves the previous value. The default is 1k.

tableLen represents the number of buckets. As we all know, prime numbers work best for hashing, so selectable sizes for tableLen are all prime numbers that are close to powers of 2. Specifically: 7, 17, 31, 61, 127, 251, 509, 1021, and 2039. When the hash-function is applied to a filename or inode, it is the result of the modulus operation that then determines the bucket the value is placed into. A value of zero preserves the previous value. The default value is 509.

basePath allows you to move the hash file to a seperate location. Perhaps a disk volume dedicated to hash files, or a RAM drive if you do not need the inode-to-filename mappings preserved. The default is "" which indicates that the hash files will be saved to the exported volume. nfsExport("/foo",0,0) would result in /foo/nfsHashTbl.cfg and /foo/nfsHashTbl.cfg being created. With basePath set to "/**ramDrv**", the files would be placed in the /**ramDrv** volume and pre-pended with the export path. Specifically, the files created would be /ramDrv/foo_nfsHashTbl.cfg and /ramDrv/foo_nfsHashTbl.dat. Passing in **NULL** preserves the existing value. The default value is "";

bucketSize and tableLen are only used when creating a hash table for the first time. After that, their values are preserved in nfsHashTbl.cfg. However, basePath (if different from the default) must be called so the NFS server can locate the files.

Returns

Not Available

Errno

Not Available

See also

nfsHash

28. PARTLIB

Name

partLib - routines to create disk partitions on a rawFS

Layer

FS

Routines

partLibSetAlignment( ) - set partition creation routines' sector alignment **xbdCreatePartition( )** - partition a device **xbdCreatePartitionSz( )** - partition an device according to the size (mega bytes) **xbdCreatePartitionEx( )** - partition a device **xbdCleanPartition( )** - clean partition table for a device **xbdActivePartition( )** - set active flag for specific partition **xbdSetPartitionType( )** - set type value for specific partition **xbdPartitionFun( )** - XBD partition function

Description

This library contains routines for handling partitions.

Example

The following code initializes a disk that expects to have up to 4 partitions. The parameter passed in is the base name of the device to be partitioned. E.g. "/ata00". Partitions are automatically named using the base name followed by a ":" and then the partition number. Using the fsMonitor component, these automatic names can be mapped to something more meaningful:

```
STATUS  usrCreatePartitions( char *devName  )
    {
    devname_t  baseName;
    char       autoPartName[16];
    char *     newPartName[4] = {"/p1", "/p2", "/p3", "/p4"};
    STATUS     result;
    int        i, fd;

    /* Name mapping */

    /* Get the base name of the device */
    fd = open (devName, 0, 0666);
    if (fd < 0)
        {
        return (ERROR);
        }

    ioctl (fd, XBD_GETBASENAME, (_Vx_ioctl_arg_t)baseName);

    close (fd);

    for (i = 0; i < 4; i++)
        {
        xpPartName (baseName, i+1, autoPartName, FALSE);

        printf ("Installing mapping from %s to %s\n", autoPartName, newPartName[i]);
        fsmNameInstall (autoPartName, newPartName[i]);
        }
```

```
    /* create 4 partitions on the device all with equal sizes */

    result = xbdCreatePartition ( devName, 4, 25, 25, 25 );

    if (result != OK)
        {
        return (ERROR);
        }

    /* create file systems atop each partition */

    dosFsVolFormat ( newPartName[0], 0, NULL);
    dosFsVolFormat ( newPartName[1], 0, NULL);
    dosFsVolFormat ( newPartName[2], 0, NULL);
    dosFsVolFormat ( newPartName[3], 0, NULL);

    return (OK);
    }
```

Include files

fsMonitor.h, **fsEventUtilLib.h**, **drv/xbd/xbd.h**, **drv/erf/erfLib.h**

See also

dosFsLib, fsMonitor, VxWorks 7 File Systems and I/O Programmer's Guide Creating and Partitioning a Disk and Creating Volumes

Name

partLibSetAlignment( ) - set partition creation routines' sector alignment

Layer

FS

Synopsis

```
STATUS partLibSetAlignment
    (
    UINT32 alignment
    )
```

Description

This routine set partition creation routines' sector alignment. All routines in this library will be impacted.

Setting alignment to **PART_CYLINDER_ALIGN** will make partition creation routines to create partition with cylinder aligned.

If device size is too small, the alignment may be ignored.

Returns

OK always

Errno

Not Available

See also

partLib

Name

xbdCreatePartition( ) - partition a device

Layer

FS

Synopsis

```
STATUS xbdCreatePartition
    (
    char *pathName,   /* name of device to partition */
    int  nPart,       /* number of partitions */
    int  size1,       /* space percentage for second partition */
    int  size2,       /* space percentage for third partition */
    int  size3        /* space percentage for fourth partition */
    )
```

Description

This function is capable of creating only one partition table - the MBR, and will not create any Bootable or Extended partitions. Therefore, only 4 primary partitions are supported.

pathName is the name the device to be partitioned.

The nPart argument contains the number of partitions to create. If nPart is 0 or 1, a single partition covering the entire disk is created. If nPart is between 2 and 4, the arguments size1, size2 and size3 contain (as integers) the percentage of disk space to be assigned to the 2nd, 3rd, and 4th partitions respectively. The first partition (partition 0) will be assigned the remaining space. Thus, the sum of the three sizes should be less than 100.

This function will try to align partition head and size with the value set by **partLibSetAlignment( )**. If device size is too small, alignment parameter may be ignored.

Returns

OK upon success, **ERROR** otherwise

Errno

Not Available

See also

partLib

Name

xbdCreatePartitionSz( ) - partition an device according to the size (mega bytes)

Layer

FS

Synopsis

```
STATUS xbdCreatePartitionSz
    (
    char *pathName,   /* name of device to partition */
    int  nPart,       /* number of partitions */
    int  size1,       /* space size for second partition */
    int  size2,       /* space size for third partition */
    int  size3        /* space size for fourth partition */
    )
```

Description

This function is capable of creating only one partition table - the MBR, and will not create any Bootable or Extended partitions. Therefore, only 4 primary partitions are supported.

pathName is the name the device to be partitioned.

The nPart argument contains the number of partitions to create. If nPart is 0 or 1, a single partition covering the entire disk is created. If nPart is between 2 and 4, the arguments size1, size2 and size3 contain the size (mega-bytes) to be assigned to the 2nd, 3rd, and 4th partitions respectively. The first partition (partition 0) will be assigned the remaining space. Thus, the sum of the three sizes should be less than total size of the disk.

This function will try to align partition head and size with the value set by **partLibSetAlignment( )**. If device size is too small, alignment parameter may be ignored.

Returns

OK upon success, **ERROR** otherwise

Errno

Not Available

See also

partLib

Name

xbdCreatePartitionEx( ) - partition a device

Layer

FS

Synopsis

```
STATUS xbdCreatePartitionEx
    (
    char *pathName,   /* name of device to partition */
    int  nPart,       /* number of partitions */
    int  size1,       /* space percentage for second partition */
    int  size2,       /* space percentage for third partition */
    int  size3,       /* space percentage for fourth partition */
    int  resSize      /* reserve disk size as a sector num */
    )
```

Description

This function is capable of creating only one partition table - the MBR, and will not create any Bootable or Extended partitions. Therefore, only 4 primary partitions are supported.

pathName is the name the device to be partitioned.

The nPart argument contains the number of partitions to create. If nPart is 0 or 1, a single partition covering the entire disk is created. If nPart is between 2 and 4, the arguments size1, size2 and size3 contain (as integers) the percentage of disk space to be assigned to the 2nd, 3rd, and 4th partitions respectively. The first partition (partition 0) will be assigned the remaining space. Thus, the sum of the three sizes should be less than 100.

resSize is reserved disk size. The entire disk size will minus resSize then calculate the partition size.

This function will try to align partition head and size with the value set by **partLibSetAlignment( )**. If device size is too small, alignment parameter may be ignored.

Returns

OK upon success, **ERROR** otherwise

Errno

Not Available

See also

partLib

Name

xbdCleanPartition( ) - clean partition table for a device

Layer

FS

Synopsis

```
STATUS xbdCleanPartition
    (
    char * pathName,
    BOOL   resetMbr
    )
```

Description

This function cleans partition table for a device. pathName is the name the device to be cleaned.

By default, this routine only clean partition table data. If resetMbr is set to be **TRUE**, this routine will write a new MBR for device.

Returns

OK upon success, **ERROR** otherwise

Errno

Not Available

See also

partLib

Name

xbdActivePartition( ) - set active flag for specific partition

Layer

FS

Synopsis

```
STATUS xbdActivePartition
    (
    char * pathName,
    int    partIndex,
    BOOL   setActive
    )
```

Description

This function set or clean active flag in MBR for specifc partition.

pathName is the name the device contains the partition partIndex is the partition index in MBR, it should be 1 to 4. setActive indicates if need to set or clean the active flag.

Since there should be one active partition at most, setting active flag to a partition will make other partitions clean their active flag.

Returns

OK upon success, **ERROR** otherwise

Errno

Not Available

See also

partLib

Name

xbdSetPartitionType( ) - set type value for specific partition

Layer

FS

Synopsis

```
STATUS xbdSetPartitionType
    (
    char * pathName,
    int    partIndex,
    UINT8  type
    )
```

Description

This function set type value in MBR for specifc partition.

pathName is the name the device contains the partition partIndex is the partition index in MBR, it should be 1 to 4. type is the type value need to be set.

Returns

OK upon success, **ERROR** otherwise

Errno

Not Available

See also

partLib

Name

xbdPartitionFun( ) - XBD partition function

Layer

FS

Synopsis

```
STATUS xbdPartitionFun
    (
    char          *pathName,   /* name of device to partition */
    UINT32        funIndex,    /* function number */
    _Vx_usr_arg_t arg1,
    _Vx_usr_arg_t arg2,
    _Vx_usr_arg_t arg3,
    _Vx_usr_arg_t arg4,
    _Vx_usr_arg_t arg5,
    _Vx_usr_arg_t arg6,
    _Vx_usr_arg_t arg7,
    _Vx_usr_arg_t arg8,
    _Vx_usr_arg_t arg9,
    _Vx_usr_arg_t arg10
    )
```

Description

This function is used to provide some basic disk operations such as create, add, delete, show and so on, but now we only have implemented create function. In the future, we can add other function step by step.

pathName is the name the device to be partitioned.

The funIndex argument is the disk operation index. The operation index is defined in "part_function", please refer below. enum part_function { CREATE = 0 }; arg1 ~ arg10is used to pass the paremeter for the function specified by funIndex. Now to create function, we have supported to create disk partitions with reserved sectors. e.g., to create, you can call xbdPartitionFun("/ata0", 0, 4, 25, 25, 25, 1024, 0, 0, 0, 0, 0) to create 4 partitions, the size of every partition is 25% of total disk size, 1024 means to reserve 1024 sectors.

Note

The description and definition of the argument for every disk operation.

Function Number List:

CREATE

Create partitions with specific parameters arg1: number of partitions arg2: space percentage for second partition arg3: space percentage for third partition arg4: space percentage for fourth partition arg5: reserve disk size as a sector num arg6 ~ arg10: not used.

CLEAN

Clean partition table arg1: **TRUE** for reset and re-write default MBR, including boot code data.
FALSE for only clean partition table entry.

EJECT

Unmount all partitions and set device to non-partition state. This function open device and call ioctl function with **XBD_SOFT_EJECT XBD_BASE** to change device state.

REMOUNT

Unmount all partitions and re-mount all partitions again. This function call ioctl function with **XBD_SOFT_EJECT XBD_BASE** to change device to non-partition state. Then call ioctl function with **XBD_HARD_EJECT XBD_BASE** to make FS Monitor to re-mount all parititions on device.

ACTIVE

Set partition active flag for specific partition. There should be one active partition on device at most.

arg1: partition index, it should be 1 to 4. arg2: **TRUE** for setting active flag, **FALSE** for cleaning active flag.

SETTYPE

Set partition type flag for specific partition. Type value is an unsigned 8-bit value.

arg1: partition index, it should be 1 to 4. arg2: unsigned 8-bit type value.

Returns

OK upon success, **ERROR** otherwise

Errno

Not Available

See also

partLib

29. RAWFSLIB

Name

rawFsLib - raw block device file system library

Layer

FS

Routines

rawFsDevInit( ) - associate a block device with raw volume functions **rawFsInit( )** - prepare to use the raw volume library

Using this library

The various routines provided by the VxWorks raw "file system" (rawFs) may be separated into three broad groups: general initialization, device initialization, and file system operation.

The **rawFsInit( )** routine is the principal initialization function; it need only be called once, regardless of how many rawFs devices will be used.

A separate rawFs routine is used for device initialization. For each rawFs device, **rawFsDevInit( )** must be called to install the device.

Initialization

Before any other routines in **rawFsLib** can be used, **rawFsInit( )** must be called to initialize the library. This call specifies the maximum number of raw device file descriptors that can be open simultaneously and allocates memory for that many raw file descriptors. Any attempt to open more raw device file descriptors than the specified maximum will result in errors from **open( )** or **creat( )**.

During the **rawFsInit( )** call, the raw device library is installed as a driver in the I/O system driver table. The driver number associated with it is then placed in a global variable, **rawFsDrvNum**.

This initialization is enabled when the configuration macro **INCLUDE_RAWFS** is defined; **rawFsInit( )** is then called (normally via components from vxprj).

Defining a raw device

To use this library for a particular device, an eXtended Block Device (XBD) must be initialized as the backing media; the second parameter to **rawFsDevInit( )** must be a device_t referring to this XBD.

Use the **rawFsDevInit( )** routine to associate a device with the **rawFsLib** functions. The pVolName parameter expected by **rawFsDevInit( )** is a pointer to a name string, to be used to identify the device. This parameter will serve as the pathname for I/O operations which operate on the device. This name will appear in the I/O system device table, which may be displayed using **iosDevShow( )**.

The syntax of the **rawFsDevInit( )** routine is as follows:

```
rawFsDevInit
    (
    char     *pVolName, /* name to be used for volume - iosDevAdd  */
    device_t xbd        /* handle to the backing XBD */
    )
```

When **rawFsLib** receives a request from the I/O system, after **rawFsDevInit( )** has been called, it calls the appropriate device driver routines to access the device.

Ioctl functions

The VxWorks raw block device file system supports the following **ioctl( )** functions. The functions listed are defined in the header **ioLib.h**.

FIOGETNAME

Gets the file name of the file descriptor and copies it to the buffer nameBuf:

```
status = ioctl (fd, FIOGETNAME, &nameBuf);
```

FIOSEEK

Sets the current byte offset on the disk to the position specified by newOffset, range from 0 to **UINT_MAX**:

```
unsigned int newOffset = ...;
status = ioctl (fd, FIOSEEK, newOffset);
```

FIOSEEK64

Changes the current file position. This is the 64-bit version.

```
long long  newOffset = 0;
status = ioctl (fd, FIOSEEK64, (_Vx_ioctl_arg_t)&newOffset);
```

FIOWHERE

Returns the current byte position from the start of the device for the specified file descriptor, returns value range from 0 to **UINT_MAX** - 1, **ERROR** on failure. This is the byte offset of the next byte to be read or written. It takes no additional argument:

```
position = ioctl (fd, FIOWHERE, 0);
```

FIOWHERE64

Returns the current byte position from the start of the device for the specified file descriptor.This is the 64-bit version of FIOWHERE.

```
long long  where64 = 0;
status = ioctl (fd, FIOWHERE64, (_Vx_ioctl_arg_t)&where64);
```

FIOFLUSH

Writes all modified file descriptor buffers to the physical device.

```
status = ioctl (fd, FIOFLUSH, 0);
```

FIOSYNC

Performs the same function as FIOFLUSH.

FIONREAD

Copies to unreadCount the number of bytes from the current file position to the end of the device, support 32-bit value:

```
unsigned int unreadCount = 0;
status = ioctl (fd, FIONREAD, &unreadCount);
```

FIONREAD64

Copies to unreadCount the number of bytes from the current file position to the end of the device, this is the 64-bit version of FIONREAD:

```
    long long unreadCount = 0;
    status = ioctl (fd, FIONREAD64, (_Vx_ioctl_arg_t)&unreadCount);
```

FIOFSTATFSGET

Returns file system status information. The information contains file system type id, block size, and total blocks.

```
    struct statfs data;
    status = ioctl (fd, FIOFSTATFSGET, (_Vx_ioctl_arg_t)&data);
```

FIOFSTATFSGET64

Returns file system status information. The information contains file system type id, block size, and total blocks. This is the 64-bit version of FIOFSTATFSGET.

```
    struct statfs64 data;
    status = ioctl (fd, FIOFSTATFSGET64, (_Vx_ioctl_arg_t)&data);
```

Include files

rawFsLib.h

See also

ioLib, **iosLib**, **rawFsLib**, **ramDrv**, VxWorks 7 File Systems and I/O Programmer's Guide [Overview of rawFs] (VXWEXEC-4927https://docs.windriver.com/bundle/vxworks_7_file_systems_and_IO_programmers_guide_sr0650/page/eri1450462366250.html)

Name

rawFsDevInit( ) - associate a block device with raw volume functions

Layer

FS

Synopsis

```
RAW_VOL_DESC *rawFsDevInit
    (
    char      * pVolName,   /* volume name to be used with iosDevAdd */
    device_t    xbd         /* XBD device */
    )
```

Description

This routine takes a block device created by a device driver and defines it as a raw file system volume. As a result, when high-level I/O operations, such as **open( )** and **write( )** are performed, on the device, the calls will be routed through **rawFsLib**.

This routine associates pVolName with a device and installs it in the VxWorks I/O System's device table. The driver number used when the device is added to the table is that which was assigned to the raw library during **rawFsInit( )**. (The driver number is kept in the global variable **rawFsDrvNum**.)

WNDRVR

The xbd is a device_t referring to an XBD device which represents the backing media for this rawFs. The XBD device will not be accessed until an I/O operation occurs on the file system.

In normal usage, this routine will be called by File System Monitor automatically if the volume do not have any valid file system. User should not call this routine directly, unless there is File System Monitor is not configured.

Returns

A pointer to the volume descriptor (**RAW_VOL_DESC**), or **NULL** if there is an error.

Errno

Not Available

See also

rawFsLib

Name

rawFsInit( ) - prepare to use the raw volume library

Layer

FS

Synopsis

```
STATUS rawFsInit
    (
    int maxFiles,   /* max no. of simultaneously open files */
    int maxBlocks   /* max no. of blocks per bio */
    )
```

Description

This routine initializes the raw volume library. It must be called exactly once, before any other routine in the library. The argument specifies the number of file descriptors that may be open at once. This routine allocates and sets up the necessary memory structures and initializes semaphores.

This routine also installs raw volume library routines in the VxWorks I/O system driver table. The driver number assigned to **rawFsLib** is placed in the global variable **rawFsDrvNum**. This number will later be associated with system file descriptors opened to rawFs devices.

Returns

OK or **ERROR**.

Errno

Not Available

See also

rawFsLib

30. ROMFSDRV

Name

romfsDrv - ROMFS block device driver

Layer

FS_ROMFS

Routines

romfsBlkDevCreate( ) - create a ROMFS block device

User-callable routines

All of the routines in this driver are accessible only through the I/O system with the exception of **romfsBlkDevCreate( )**. **romfsBlkDevCreate( )** must be called to initialize the driver before any of the other routines are accessible.

Once the device has been created, it must be associated with the ROMFS file system and given a name. To associate the device with the needed information, pass the value returned by **romfsBlkDevCreate( )**, a pointer to a block device structure, to the ROMFS **romfsFsDevCreate( )** initialization routine.

Include file

romfsDrv.h

See also

romfsFsDevInit( ), VxWorks 7 File Systems and I/O Programmer's Guide Overview of ROMFS

Name

romfsBlkDevCreate( ) - create a ROMFS block device

Layer

FS_ROMFS

Synopsis

```
BLK_DEV *romfsBlkDevCreate
    (
    char *pAddr,   /* starting address of ROMFS device image */
    int  size,     /* size of ROMFS device image, in bytes */
    int  offset    /* offset, in bytes, from start of device */
    )
```

Description

This routine creates a ROMFS block device using the specified image. The block size of the device is fixed at **ROMFS_BLOCK_SIZE** bytes per block, which is what the romfs file system is expecting (see **romfs.h**).

The addr parameter specifies the starting address of the ROMFS image in memory. The address must be aligned on a word boundary.

The size parameter specifies the size of the ROMFS image in bytes.

The offset parameter specifies an offset, in bytes, from the start of the device to be used when reading the ROMFS image.

Returns

A pointer to a block device structure (**BLK_DEV**) or **NULL** if memory cannot be allocated for the device structure.

Errno

Not Available

See also

romfsDrv, **romfsFsInit( )**

31. ROMFSLIB

Name

romfsLib - ROMFS file system driver

Layer

FS_ROMFS

Routines

romfsVolumeCreate( ) - create a romfs volume.

Description

This library implements the VxWorks interface to the BSD/OS ROMFS file system. ROMFS is a simple, read-only file system that stores and represents files and directories in a hierarchical manner. The name ROMFS stands for Read-Only Memory File System (it does not imply any particular relationship to ROM media). No local disk or network connection to a remote disk is required.

Using this library

The **romfsFsInit( )** routine is the principal initialization function; it should be called once during system initialization, regardless of how many romfs devices are to be used.

Another romfs routine is used for device initialization. For each romfs device, **romfsFsDevCreate( )** must be called to install the device in VxWorks device list.

For example:

```
    pBlkDev=(BLK_DEV *)romfsBlkDevCreate((char *)&romfsImage,
                                         romfsImageLen, 0);
    if (pBlkDev == NULL)
        {
        printErr ("Error creating romfs block device!\n");
        }
    else
        {
        pVolDesc = (void *)romfsFsDevCreate ("/romfs", pBlkDev);
        }
```

Include files

romfs.h, **romfsLib.h**

See also

romfs, **romfsDrv**, VxWorks 7 File Systems and I/O Programmer's Guide

Name

romfsVolumeCreate( ) - create a romfs volume.

Layer

FS_ROMFS

Synopsis

```
STATUS romfsVolumeCreate
    (
    char   *pDevName,       /* Device name of the ROMFS                    */
    char   *pAddr,          /* Starting address of the ROMFS device image  */
    size_t romfsImageLen,   /* Size of ROMFS device image, in bytes        */
    off_t  offset           /* offset, in bytes, from start of the device  */
    )
```

Description

This routine creates an instance of a romfs file system. The routine call be called multiple time to create multiple volumes.

This function requires a pointer to the romfs device name, the beginning address of the romfs image, size of the romfs image, and the bytes offset value of the romfs image.

The romfs device name has to be unique in the system.

Returns

OK or **ERROR**

Errno

Not Available

See also

romfsLib

32. TARLIB

Name

tarLib - UNIX tar compatible library

Layer

FS_UTIL

Routines

tarExtract( ) - extract all files from a tar formatted tape **tarArchive( )** - archive named file/dir onto tape in tar format **tarToc( )** - display all contents of a tar formatted tape

Description

This library implements functions for archiving, extracting and listing of UNIX-compatible "tar" file archives. It can be used to archive and extract entire file hierarchies to/from archive files on local or remote disks, or directly to/from magnetic tapes.

Current limitations

This Tar utility does not handle MS-DOS file attributes, when used in conjunction with the MS-DOS file system. The maximum subdirectory depth supported by this library is 16, while the total maximum path name that can be handled by tar is limited at 100 characters.

The maximum file size supported is limited to 4GB. Which means that any attempt to archive a file larger than 4GB would fail with errno set to **EFBIG**.

Include files

none

See also

Name

tarExtract( ) - extract all files from a tar formatted tape

Layer

FS_UTIL

Synopsis

```
STATUS tarExtract
    (
    char * pTape,      /* tape device name */
    size_t bfactor,    /* requested blocking factor */
    BOOL   verbose     /* if TRUE print progress info */
    )
```

Description

This is a UNIX-tar compatible utility that extracts entire file hierarchies from tar-formatted archive. The files are extracted with their original names and modes. In some cases a file cannot be created on disk, for example if the name is too long for regular DOS file name conventions, in such cases entire files are skipped, and this program will continue with the next file. Directories are created in order to be able to create all files on tape.

The tape argument may be any tape device name or file name that contains a tar formatted archive. If tape is equal "-", standard input is used. If tape is **NULL** (or unspecified from Shell) the default archive file name stored in global variable TAPE is used.

The bfactor dictates the blocking factor the tape was written with. If 0, or unspecified from the shell, a default of 20 is used.

The verbose argument is a boolean, if set to 1, will cause informative messages to be printed to standard error whenever an action is taken, otherwise, only errors are reported.

All informative and error message are printed to standard error.

There is no way to selectively extract tar archives with this utility. It extracts entire archives.

Returns

Not Available

Errno

Not Available

See also

tarLib

Name

tarArchive( ) - archive named file/dir onto tape in tar format

Layer

FS_UTIL

Synopsis

```
STATUS tarArchive
    (
    char * pTape,     /* tape device name */
    size_t bfactor,   /* requested blocking factor */
    BOOL   verbose,   /* if TRUE print progress info */
    char * pName      /* file/dir name to archive */
    )
```

Description

This function creates a UNIX compatible tar formatted archives which contain entire file hierarchies from disk file systems. Files and directories are archived with mode and time information as returned by **stat( )**.

The tape argument can be any tape drive device name or a name of any file that will be created if necessary, and will contain the archive. If tape is set to "-", standard output will be used. If tape is **NULL** (unspecified from Shell), the default archive file name stored in global variable TAPE will be used.

Each **write( )** of the archive file will be exactly bfactor*512 bytes long, hence on tapes in variable mode, this will be the physical block size on the tape. With Fixed Mode tapes this is only a performance matter. If bfactor is 0, or unspecified from Shell, it will be set to the default value of 20.

The verbose argument is a boolean, if set to 1, will cause informative messages to be printed to standard error whenever an action is taken, otherwise, only errors are reported.

The name argument is the path of the hierarchy to be archived. if **NULL** (or unspecified from the Shell), the current directory path "." will be used. This is the path as seen from the target, not from the host.

All informative and error message are printed to standard error.

Note

Refrain from specifying absolute path names in path, such archives tend to be either difficult to extract or can cause unexpected damage to existing files if such exist under the same absolute name.

There is no way of specifying a number of hierarchies to dump.

Returns

Not Available

Errno

Not Available

See also

tarLib

Name

tarToc( ) - display all contents of a tar formatted tape

Layer

FS_UTIL

Synopsis

```
STATUS tarToc
    (
    char * tape,      /* tape device name */
    size_t bfactor    /* requested blocking factor */
    )
```

Description

This is a UNIX-tar compatible utility that displays entire file hierarchies from tar-formatted media, e.g. tape.

The tape argument may be any tape device name or file name that contains a tar formatted archive. If tape is equal "-", standard input is used. If tape is **NULL** (or unspecified from Shell) the default archive file name stored in global variable TAPE is used.

The bfactor dictates the blocking factor the tape was written with. If 0, or unspecified from Shell, default of 20 is used.

Archive contents are displayed on standard output, while all informative and eror message are printed to standard error.

Returns

Not Available

Errno

Not Available

See also

tarLib

33. TC_TCPLAY

Name

tc_tcplay - main library of TC play

Layer

TCPLAY

Routines

tcVolShow( ) - show a mapped tc volume information **tcVolMount( )** - mount an encrypt volume image. **tcVolUnmount( )** - unmount an encrypt volume image. **tcAutoMountRegister( )** - register some encrypt images for automatically mount

Description

This module contains main API defination of tcplay for users. These APIs call legacy tcplay routines for encrypting and decrypting images.

Initialization

The initialization routine will prepare cipher chains for encrypting and decrypting. After cipher chains were initialized, it will prepare tcplay XBD transfer module for working with loop filesystem.

Data flow

```
+--------+    +------------+    +-----------------+     | tcplay |----| xbd_tcplay |----| loop filesystem |     +--------+    +------------
+    +-----------------+
```

Include files

sys/types.h sys/stat.h stdio.h stdlib.h stdarg.h unistd.h, **errno.h string.h time.h uuid.h dirent.h tc_crc32.h tc_tcplay.h**, **tc_humanize.h drv/erf/erfLib.h taskLib.h ioLib.h sys/fcntlcom.h**, **fsEventUtilLib.h**

Name

tcVolShow( ) - show a mapped tc volume information

Layer

TCPLAY

Synopsis

```
int tcVolShow
    (
    const char * mountPoint
    )
```

Description

This routine shows a mapped tc volume information, e.g. encrypt method name, volume size ...

Returns

0 for success, otherwise if error occurred.

Errno

N/A

See also

tc_tcplay

Name

tcVolMount( ) - mount an encrypt volume image.

Layer

TCPLAY

Synopsis

```
int tcVolMount
    (
    const char * mountPoint,
    const char * imageName,
    const char * passphrase
    )
```

Description

This routine mounts an encrypt volume image with specific password.

Returns

0 for success, otherwise if error occurred.

Errno

N/A

See also

tc_tcplay

Name

tcVolUnmount( ) - unmount an encrypt volume image.

Layer

TCPLAY

Synopsis

```
int tcVolUnmount
    (
    const char * mountPoint
    )
```

Description

This routine unmounts an encrypt volume image for a specific mount point

Returns

0 for success, otherwise if error occurred.

Errno

N/A

See also

tc_tcplay

Name

tcAutoMountRegister( ) - register some encrypt images for automatically mount

Layer

TCPLAY

Synopsis

```
int tcAutoMountRegister
    (
    char *  paths[],
    char *  mountPoints[],
    char *  passphrases[],
    FUNCPTR convertFunc,
    void *  param,
    BOOL    erasePass
    )
```

Description

This routine is used for register some encrypt images for automatically mount with specific paramaters.

These parameters contain mount-points, passphrases and some other parameters.

A passphrase convert routine is optional paramaters. It is used for protecting real passphrase for building images.

Parameter paths indicates the full path filename of the encrypt images. This array must be ended with **NULL**. Number of element in parameter mountPoints and passphrases must equal or larger than number of element in parameter paths.

Returns

0 for success, otherwise if error occurred.

Errno

N/A

See also

tc_tcplay

34. TFFSDRV

Name

tffsDrv - TrueFFS interface for VxWorks

Layer

BDM_FLASH

Routines

tffsDrv( ) - initialize the TrueFFS system **tffsDevCreate( )** - create a TrueFFS block device suitable for use with dosFs **tffsDevOptionsSet( )** - set TrueFFS volume options **tffsDrvOptionsSet( )** - set TrueFFS volume options **tffsDevFormat( )** - format a flash device for use with TrueFFS **tffsRawio( )** - low level I/O access to flash components

Description

This module defines the routines that VxWorks uses to create a TrueFFS block device. Using this block device, dosFs can access a board-resident flash memory array or a flash memory card (in the PCMCIA slot) just as if it was a standard disk drive. Also defined in this file are functions that you can use to format the flash medium, as well as well as functions that handle the low-level I/O to the device.

To include TrueFFS in a VxWorks image, you must have the component **INCLUDE_TFFS** included, or, for some hardware, **INCLUDE_PCMCIA**. If you define **INCLUDE_TFFS**, this configures **usrRoot( )** to call **tffsDrv( )**. If you defined **INCLUDE_PCMCIA**, the call to **tffsDrv( )** is made from **pccardTffsEnabler( )**. The call to **tffsDrv( )** sets up the structures, global variables, and mutual exclusion semaphore needed to manage TrueFFS. This call to **tffsDrv( )** also registers socket component drivers for each flash device found attached to the target.

These socket component drivers are not quite block devices, but they are an essential layer within TrueFFS. Their function is to manage the hardware interface to the flash device, and they are intelligent enough to handle formatting and raw I/O requests to the flash device. The other two layers within TrueFFS are known as the translation layer and the MTD (the Memory Technology Driver). The translation layer of TrueFFS implements the error recover and wear-leveling features of TrueFFS. The MTD implements the low-level programming (map, read, write, and erase) of the flash medium.

To implement the socket layer, each BSP that supports TrueFFS includes a **sysTffs.c** file. This file contains the code that defines the socket component driver. This file also contains a set of defines that you can use to configure which translation layer modules and MTDs are included in TrueFFS. Which translation layer modules and MTDs you should include depends on which types of flash devices you need to support. Currently, there are three basic flash memory technologies, NAND-based, NOR-based, and SSFDC. Within **sysTffs.c**, define:

INCLUDE_TL_NFTL

To include the NAND-based translation layer module.

INCLUDE_TL_FTL

To include the NOR-based translation layer module.

INCLUDE_TL_SSFDC

To include the SSFDC-appropriate translation layer module.

To support these different technologies, TrueFFS ships with three different implementations of the translation layer. Optionally, TrueFFS can include all three modules. TrueFFS later binds the appropriate translation layer module to the flash device when it registers a socket component driver for the device.

Within these three basic flash device categories there are still other differences (largely manufacturer-specific). These differences have no impact on the translation layer. However, they do make a difference for the MTD. Thus, TrueFFS ships with eight different MTDs that can support a variety of flash devices from Intel, Sharp, Samsung, National, Toshiba, AMD, and Fujitsu. Within **sysTffs.c**, define:

INCLUDE_MTD_I28F016

For Intel 28f016 flash devices.

INCLUDE_MTD_I28F008

For Intel 28f008 flash devices.

INCLUDE_MTD_I28F008_BAJA

For Intel 28f008 flash devices on the Heurikon Baja 4000.

INCLUDE_MTD_AMD

For AMD, Fujitsu: 29F0{40,80,16} 8-bit flash devices.

INCLUDE_MTD_CDSN

For Toshiba, Samsung: NAND CDSN flash devices.

INCLUDE_MTD_DOC2

For Toshiba, Samsung: NAND DOC flash devices.

INCLUDE_MTD_CFISCS

For CFI/SCS flash devices.

INCLUDE_MTD_WAMD

For AMD, Fujitsu 29F0{40,80,16} 16-bit flash devices.

The socket component driver and the MTDs are provided in source form. If you need to write your own socket driver or MTD, use these working drivers as a model for your own.

Externally callable routines

Most of the routines defined in this file are accessible through the I/O system only. However, four routines are callable externally. These are: **tffsDrv( )**, **tffsDevCreate( )**, **tffsDevFormat( )**, and **tffsRawio( )**.

The first routine called from this library must be **tffsDrv( )**. Call this routine exactly once. Normally, this is handled automatically for you from within **usrRoot( )**, if **INCLUDE_TFFS** is defined, or from within **pccardTffsEnabler( )**, if **INCLUDE_PCMCIA** is defined.

Internally, this call to **tffsDrv( )** registers socket component drivers for all the flash devices connected to your system. After registering a socket component driver for the device, TrueFFS can support calls to **tffsDevFormat( )** or **tffsRawio( )**. However, before you can mount dosFs on the flash device, you must call **tffsDevCreate( )**. This call creates a block device on top of the socket component driver, but does not mount dosFs on the device. Because mounting dosFs on the device is what you will want to do most of the time, the **sysTffs.c** file defines a helper function, **usrTffsConfig( )**. Internally, this function calls **tffsDevCreate( )** and then does everything necessary (such as calling the **dosFsDevInit( )** routine) to mount dosFs on the resulting block device.

Low level i/o

Normally, you should handle your I/O to the flash device using dosFs. However, there are situations when that level of indirection is a problem. To handle such situations, this library defines **tffsRawio( )**. Using this function, you can bypass both dosFs and the TrueFFS translation services to program the flash medium directly.

However, you should not try to program the flash device directly unless you are intimately familiar with the physical limits of your flash device as well as with how TrueFFS formats the flash medium. Otherwise you risk not only corrupting the medium entirely but permanently damaging the flash device.

If all you need to do is write a boot image to the flash device, use the **tffsBootImagePut( )** utility instead of **tffsRawio( )**. This function provides safer access to the flash medium.

Ioctl

This driver responds to all ioctl codes by setting a global error flag. Do not attempt to format a flash drive using ioctl calls.

Include files

tffsDrv.h, **fatlite.h**

Name

tffsDrv( ) - initialize the TrueFFS system

Layer

BDM_FLASH

Synopsis

```
STATUS tffsDrv (void)
```

Description

This routine sets up the structures, the global variables, and the mutual exclusion semaphore needed to manage TrueFFS. This call also registers socket component drivers for all the flash devices attached to your target.

Because **tffsDrv( )** is the call that initializes the TrueFFS system, this function must be called (exactly once) before calling any other TrueFFS utilities, such as **tffsDevFormat( )** or **tffsDevCreate( )**. Typically, the call to **tffsDrv( )** is handled for you automatically. If you defined **INCLUDE_TFFS** in your BSP's **config.h**, the call to **tffsDrv( )** is made from **usrRoot( )**. If your BSP's **config.h** defines **INCLUDE_PCMCIA**, the call to **tffsDrv( )** is made from **pccardTffsEnabler( )**.

Returns

OK, or **ERROR** if it fails.

Errno

Not Available

See also

tffsDrv

Name

tffsDevCreate( ) - create a TrueFFS block device suitable for use with dosFs

Layer

BDM_FLASH

Synopsis

```
BLK_DEV * tffsDevCreate
    (
    int tffsDriveNo,          /* TFFS drive number (0 - DRIVES-1) */
    int removableMediaFlag    /* 0 - nonremovable flash media */
    )
```

Description

This routine creates a TFFS block device on top of a flash device. It takes as arguments a drive number, determined from the order in which the socket components were registered, and a flag integer that indicates whether the medium is removable or not. A zero indicates a non removable medium. A one indicates a removable medium. If you intend to mount dosFs on this block device, you probably do not want to call **tffsDevCreate( )**, but should call **usrTffsConfig( )** instead. Internally, **usrTffsConfig( )** calls **tffsDevCreate( )** for you. It then does everything necessary (such as calling the **dosFsDevInit( )** routine) to mount dosFs on the just created block device.

Returns

BLK_DEV pointer, or **NULL** if it failed.

Errno

Not Available

See also

tffsDrv

Name

tffsDevOptionsSet( ) - set TrueFFS volume options

Layer

BDM_FLASH

Synopsis

```
STATUS tffsDevOptionsSet
    (
    TFFS_DEV * pTffsDev   /* pointer to device descriptor */
    )
```

Description

This routine is intended to set various TrueFFS volume options. At present it do nothing.

Returns

OK, or **ERROR** if it failed.

Errno

Not Available

See also

tffsDrv

Name

tffsDrvOptionsSet( ) - set TrueFFS volume options

Layer

BDM_FLASH

Synopsis

```
STATUS tffsDrvOptionsSet
    (
    int tffsDriveNo   /* TFFS drive number (0 - DRIVES-1) */
    )
```

Description

This routine is intended to set various TrueFFS volume options. At present it do nothing.

Returns

OK, or **ERROR** if it failed.

Errno

Not Available

See also

tffsDrv

Name

tffsDevFormat( ) - format a flash device for use with TrueFFS

Layer

BDM_FLASH

Synopsis

```
STATUS tffsDevFormat
    (
    int   tffsDriveNo,   /* TrueFFS drive number (0 - DRIVES-1) */
    void* arg            /* pointer to tffsDevFormatParams structure */
    )
```

Description

This routine formats a flash device for use with TrueFFS. It takes two parameters, a drive number and a pointer to a device format structure. This structure describes how the volume should be formatted. The structure is defined in **dosformt.h**. The drive number is assigned in the order that the socket component for the device was registered.

The format process marks each erase unit with an Erase Unit Header (EUH) and creates the physical and virtual Block Allocation Maps (BAM) for the device. The erase units reserved for the "boot-image" are skipped and the first EUH is placed at number (boot-image length - 1). To write to the boot-image region, call **tffsBootImagePut( )**.

Warning

If any of the erase units in the boot-image region contains an erase unit header from a previous format call (this can happen if you reformat a flash device specifying a larger boot region) TrueFFS fails to mount the device. To fix this problem, use **tffsRawio( )** to erase the problem erase units (thus removing the outdated EUH).

The macro **TFFS_STD_FORMAT_PARAMS** defines the default values used for formatting a flask disk device. If the second argument to this routine is zero, **tffsDevFormat( )** uses these default values.

Returns

OK, or **ERROR** if it failed.

Errno

Not Available

See also

tffsDrv

Name

tffsRawio( ) - low level I/O access to flash components

Layer

BDM_FLASH

Synopsis

```
STATUS tffsRawio
    (
    int             tffsDriveNo,   /* TrueFFS drive number (0 - DRIVES-1) */
    int             functionNo,    /* TrueFFS function code */
    _Vx_ioctl_arg_t arg0,          /* argument 0 */
```

```
    _Vx_ioctl_arg_t arg1,          /* argument 1 */
    _Vx_ioctl_arg_t arg2           /* argument 2 */
    )
```

Description

Use the utilities provided by thisroutine with the utmost care. If you use these routines carelessly, you risk data loss as well as permanent physical damage to the flash device.

This routine is a gateway to a series of utilities (listed below). Functions such as **mkbootTffs( )** and **tffsBootImagePut( )** use these **tffsRawio( )** utilities to write boot sector information. The functions for physical read, write, and erase are made available with the intention that they be used on erase units allocated to the boot-image region by **tffsDevFormat( )**. Using these functions elsewhere could be dangerous.

The arg0, arg1, and arg2 parameters to **tffsRawio( )** are interpreted differently depending on the function number you specify for functionNo. The drive number is determined by the order in which the socket components were registered.

Function Name	arg0	arg1	arg2
TFFS_GET_PHYSICAL_INFO	user buffer address	N/A	N/A
TFFS_PHYSICAL_READ	address to read	byte count	user buffer address
TFFS_PHYSICAL_WRITE	address to write	byte count	user buffer address
TFFS_PHYSICAL_ERASE	first unit	number of units	N/A
TFFS_ABS_READ	sector number	number of sectors	user buffer address
TFFS_ABS_WRITE	sector number	number of sectors	user buffer address
TFFS_ABS_DELETE	sector number	number of sectors	N/A
TFFS_DEFRAGMENT_VOLUME	number of sectors	user buffer address	N/A

TFFS_GET_PHYSICAL_INFO

writes the flash type, erasable block size, and media size to the user buffer specified in arg0.

TFFS_PHYSICAL_READ

reads arg1 bytes from arg0 and writes them to the buffer specified by arg2.

TFFS_PHYSICAL_WRITE

copies arg1 bytes from the arg2 buffer and writes them to the flash memory location specified by arg0. This aborts if the volume is already mounted to prevent the versions of translation data in memory and in flash from going out of synchronization.

TFFS_PHYSICAL_ERASE

erases arg1 erase units, starting at the erase unit specified in arg0. This aborts if the volume is already mounted to prevent the versions of translation data in memory and in flash from going out of synchronization.

TFFS_ABS_READ

reads arg1 sectors, starting at sector arg0, and writes them to the user buffer specified in arg2.

TFFS_ABS_WRITE

takes data from the arg2 user buffer and writes arg1 sectors of it to the flash location starting at sector arg0.

TFFS_ABS_DELETE

deletes arg1 sectors of data starting at sector arg0.

TFFS_DEFRAGMENT_VOLUME

calls the defragmentation routine with the minimum number of sectors to be reclaimed, arg0, and writes the actual number reclaimed in the user buffer by arg1. Calling this function through some low priority task will make writes more deterministic. No validation is done of the user specified address fields, so the functions assume they are writable. If the address is invalid, you could see bus errors or segmentation faults.

Returns

OK, or **ERROR** if it failed.

Errno

Not Available

See also

tffsDrv

35. USRFSLIB

Name

usrFsLib - file system user interface subroutine library

Layer

FS_UTIL

Routines

cd( ) - change the default directory **pwd( )** - print the current default directory **rmdir( )** - remove a directory **rm( )** - remove a file/dir **copyStreams( )** - copy from/to specified streams **copy( )** - copy in (or stdin) to out (or stdout) **chkdsk( )** - perform consistency checking on a MS-DOS file system **dirList( )** - list contents of a directory (multi-purpose) **ls( )** - generate a brief listing of a directory **ll( )** - generate a long listing of directory contents **lsr( )** - list the contents of a directory and any of its subdirectories **llr( )** - do a long listing of directory and all its subdirectories contents **cp( )** - copy file into other file/directory. **mv( )** - mv file into other directory. **xcopy( )** - copy a hierarchy of files with wildcards **xdelete( )** - delete a hierarchy of files with wildcards **attrib( )** - modify MS-DOS file attributes on a file or directory **xattrib( )** - modify MS-DOS file attributes of many files **dosfsDiskFormat( )** - format a disk with dosFs **diskFormat( )** - format a disk with dosFs **hrfsDiskFormat( )** - format a disk with HRFS **commit( )** - commit current transaction to disk. **ioHelp( )** - print a synopsis of I/O utility functions

Description

This library provides user-level utilities for managing file systems. These utilities may be used either from the Kernel Shell, or from an application.

Wildcards

Some of the functions herein support wildcard characters in argument strings where file or directory names are expected. The wildcards are limited to "*" which matches zero or more characters and "?" which matches any single characters. Files or directories with names beginning with a "." are not normally matched with the "*" wildcard.

Directory listing

Directory listing is implemented in one function **dirList( )**, which can be accessed using one of these four front-end functions:

ls( )

produces a short list of files

lsr( )

is like **ls( )** but ascends into subdirectories

ll( )

produces a detailed list of files, with file size, modification date attributes etc.

llr( )

is like **ll( )** but also ascends into subdirectories

All functions used for directory listing accept a name of a directory, a single file to list, or a name that contain wildcards. These functions will result in listing of all objects that match the wildcard string provided.

Include files

usrFsLib.h

See also

ioLib, **dosFsLib**, **netDrv**, **nfsLib**, **hrFsLib**, VxWorks 7 Applications Programmer's Guide, VxWorks 7 File Systems and I/O Programmer's Guide, VxWorks Command-Line Tools User's Guide.

Name

cd( ) - change the default directory

Layer

FS_UTIL

Synopsis

```
STATUS cd
    (
    const char * name   /* new directory name */
    )
```

Description

This command sets the default directory to name. The default directory is a device name, optionally followed by a directory local to that device.

Note

This is a target resident function, which manipulates the target I/O system. It must be preceded with the **@** letter if executed from the Host Shell (windsh), which has a built-in command of the same name that operates on the Host's I/O system.

To change to a different directory, specify one of the following:

- an entire path name with a device name, possibly followed by a directory name. The entire path name will be changed.
- a directory name starting with a **~** or **/** or **$**. The directory part of the path, immediately after the device name, will be replaced with the new directory name.
- a directory name to be appended to the current default directory. The directory name will be appended to the current default directory.

An instance of ".." indicates one level up in the directory tree.

Note that when accessing a remote file system via RSH or FTP, the VxWorks network device must already have been created using **netDevCreate( )**.

Warning

The **cd( )** command does very little checking that name represents a valid path. If the path is invalid, **cd( )** may return **OK**, but subsequent calls that depend on the default path will fail.

Examples

The following example changes the directory to device **/fd0/**:

```
-> cd "/fd0/"
```

This example changes the directory to device **wrs:** with the local directory **~leslie/target**:

```
-> cd "wrs:~leslie/target"
```

After the previous command, the following changes the directory to **wrs:~leslie/target/config**:

```
-> cd "config"
```

After the previous command, the following changes the directory to **wrs:~leslie/target/demo**:

```
-> cd "../demo"
```

After the previous command, the following changes the directory to **wrs:/etc**.

```
-> cd "/etc"
```

Note that **~** can be used only on network devices (RSH or FTP).

Returns

OK or **ERROR**.

Errno

Not Available

See also

usrFsLib, **pwd( )**, the VxWorks programmer guides, the, VxWorks 7 Applications Programmer's Guide, VxWorks 7 File Systems and I/O Programmer's Guide, VxWorks Command-Line Tools User's Guide.

Name

pwd( ) - print the current default directory

Layer

FS_UTIL

Synopsis

```
void pwd (void)
```

Description

This command displays the current working device/directory.

Note

This is a target resident function, which manipulates the target I/O system. It must be preceded with the **@** letter if executed from the Host Shell (windsh), which has a built-in command of the same name that operates on the Host's I/O system.

Returns

N/A

Errno

Not Available

See also

usrFsLib, **cd( )**, VxWorks 7 Applications Programmer's Guide, VxWorks 7 File Systems and I/O Programmer's Guide, VxWorks Command-Line Tools User's Guide.

Name

rmdir( ) - remove a directory

Layer

FS_UTIL

Synopsis

```
int rmdir
    (
    const char * dirName   /* name of directory to remove */
    )
```

Description

This command removes an existing directory from a hierarchical file system. The dirName string specifies the pathname of the directory to be removed, which must be empty.

This call is supported by the VxWorks NFS and dosFs file systems.

Returns

0 on success, -1 on error (and errno is set)

Errno

EFAULT

dirName points outside of accessible memory space.

EINVAL

The last component of dirName is "." or ".." .

ENOENT

The pathname dirName could not be resolved.

ENOTDIR

dirName is not a directory.

ENOTEMPTY

The directory dirName is not empty.

ELOOP

Too many symbolic links were encountered in resolving dirName.

See also

usrFsLib, **mkdir( )**, VxWorks 7 Applications Programmer's Guide, VxWorks 7 File Systems and I/O Programmer's Guide

Name

rm( ) - remove a file/dir

Layer

FS_UTIL

Synopsis

```
STATUS rm
    (
    const char * fileName   /* name of file to remove */
    )
```

Description

This command is provided for UNIX similarity and calls the **remove( )** command. This command removes a file or a directory if the path name is a directory, for example, rm(path name).

Returns

OK, or **ERROR** if the file cannot be removed.

Errno

Not Available

See also

usrFsLib, **remove( )**, VxWorks 7 Applications Programmer's Guide, VxWorks 7 File Systems and I/O Programmer's Guide

Name

copyStreams( ) - copy from/to specified streams

Layer

FS_UTIL

Synopsis

```
STATUS copyStreams
    (
    int inFd,    /* file descriptor of stream to copy from */
    int outFd    /* file descriptor of stream to copy to */
    )
```

Description

This command copies from the stream identified by inFd to the stream identified by outFd until an end of file is reached in inFd. This command is used by **copy( )**.

Returns

OK, or **ERROR** if there is an error reading from inFd or writing to outFd.

Errno

Not Available

See also

usrFsLib, **copy( )**, VxWorks 7 Applications Programmer's Guide, VxWorks 7 File Systems and I/O Programmer's Guide

Name

copy( ) - copy in (or stdin) to out (or stdout)

Layer

FS_UTIL

Synopsis

```
STATUS copy
    (
    const char * in,    /* name of file to read  (if NULL assume stdin)  */
    const char * out    /* name of file to write (if NULL assume stdout) */
    )
```

Description

This command copies from the input file to the output file, until an end-of-file is reached.

Examples

The following example displays the file **dog**, found on the default file device:

```
    -> copy <dog
```

This example copies from the console to the file **dog**, on device **/ct0/**, until an **EOF** (default ^D) is typed:

```
    -> copy >/ct0/dog
```

This example copies the file **dog**, found on the default file device, to device **/ct0/**:

```
    -> copy <dog >/ct0/dog
```

This example makes a conventional copy from the file named **file1** to the file named **file2**:

```
    -> copy "file1", "file2"
```

Copy only simply check if the in file path is the same with the out file path, so the user should advoid to give the same file with the different file path.

Returns

OK, or **ERROR** if in or out cannot be opened/created, or if there is an error copying from in to out.

Errno

Not Available

See also

usrFsLib, **copyStreams( )**, **tyEOFSet( )**, **cp( )**, **xcopy( )**, VxWorks 7 Applications Programmer's Guide, VxWorks 7 File Systems and I/O Programmer's Guide

Name

chkdsk( ) - perform consistency checking on a MS-DOS file system

Layer

FS_UTIL

Synopsis

```
STATUS chkdsk
    (
    const char * pDevName,      /* device name */
    u_int        repairLevel,   /* how to fix errors */
```

```
    u_int        verbose        /* verbosity level */
    )
```

Description

This function invokes the integral consistency checking built into the **dosFsLib** file system, via FIOCHKDSK ioctl. During the test, the volume will be un-mounted and re-mounted, invalidating file descriptors to prevent any application code from accessing the volume during the test. If the drive was exported, it will need to be re-exported again as its file descriptors were also invalidated. Furthermore, the test will emit messages describing any inconsistencies found on the disk, as well as some statistics, depending upon the value of the verbose argument. Depending upon the value of repairLevel, the inconsistencies will be repaired, and changes written to disk.

These are the values for repairLevel:

0

Same as **DOS_CHK_ONLY** (1)

DOS_CHK_ONLY (1)

Only report errors, do not modify disk.

DOS_CHK_REPAIR (2)

Repair any errors found.

These are the values for verbose:

0

similar to **DOS_CHK_VERB_1**

DOS_CHK_VERB_SILENT (0xff00)

Do not emit any messages, except errors encountered.

DOS_CHK_VERB_1 (0x0100)

Display some volume statistics when done testing, as well as errors encountered during the test.

DOS_CHK_VERB_2 (0x0200)

In addition to the above option, display path of every file, while it is being checked. This option may significantly slow down the test process.

Note that the consistency check procedure will unmount the file system, meaning the all currently open file descriptors will be deemed unusable.

Returns

OK or **ERROR** if device can not be checked or could not be repaired.

Errno

Not Available

See also

usrFsLib, **dosFsLib**, VxWorks 7 Applications Programmer's Guide, VxWorks 7 File Systems and I/O Programmer's Guide

Name

dirList( ) - list contents of a directory (multi-purpose)

Layer

FS_UTIL

Synopsis

```
STATUS dirList
    (
    int          fd,          /* file descriptor to write on */
    const char * dirString,   /* name of the directory to be listed */
    BOOL         doLong,      /* if TRUE, do long listing */
    BOOL         doTree       /* if TRUE, recurse into subdirs */
    )
```

Description

This command is similar to UNIX ls. It lists the contents of a directory in one of two formats. If doLong is **FALSE**, only the names of the files (or subdirectories) in the specified directory are displayed. If doLong is **TRUE**, then the file name, size, date, and time are displayed. If doTree flag is **TRUE**, then each subdirectory encountered will be listed as well (i.e. the listing will be recursive).

The dirName parameter specifies the directory to be listed. If dirName is omitted or **NULL**, the current working directory will be listed. dirName may contain wildcard characters to list some of the directory's contents.

Limitations

- With **dosFsLib** file systems, MS-DOS volume label entries are not reported.
- Although an output format very similar to UNIX "ls" is employed, some information items have no particular meaning on some file systems.
- Some file systems which do not support the POSIX compliant **dirLib( )** interface, can not support the doLong and doTree options.

Returns

OK or **ERROR**.

Errno

Not Available

See also

usrFsLib, **dirLib**, **ls( )**, **ll( )**, **lsr( )**, **llr( )**, VxWorks 7 Applications Programmer's Guide, VxWorks 7 File Systems and I/O Programmer's Guide

Name

ls( ) - generate a brief listing of a directory

Layer

FS_UTIL

Synopsis

```
STATUS ls
    (
    const char * dirName,   /* name of dir to list */
    BOOL         doLong     /* switch on details */
    )
```

Description

This function is simply a front-end for **dirList( )**, intended for brevity and backward compatibility. It produces a list of files and directories, without details such as file size and date, and without recursion into subdirectories.

dirName is a name of a directory or file, and may contain wildcards. doLong is provided for backward compatibility.

Note

This is a target resident function, which manipulates the target I/O system. It must be preceded with the **@** letter if executed from the Host Shell (windsh), which has a built-in command of the same name that operates on the Host's I/O system.

Returns

OK or **ERROR**.

Errno

Not Available

See also

usrFsLib, **dirList( )**, VxWorks 7 Applications Programmer's Guide, VxWorks 7 File Systems and I/O Programmer's Guide, VxWorks Command-Line Tools User's Guide.

Name

ll( ) - generate a long listing of directory contents

Layer

FS_UTIL

Synopsis

```
STATUS ll
    (
    const char * dirName   /* name of directory to list */
    )
```

Description

This command causes a long listing of a directory's contents to be displayed. It is equivalent to:

```
    -> dirList 1, dirName, 1, 0
```

dirName is a name of a directory or file, and may contain wildcards.

Note 1

This is a target resident function, which manipulates the target I/O system. It must be preceded with the **@** letter if executed from the Host Shell (windsh), which has a built-in command of the same name that operates on the Host's I/O system.

Note 2

When used with **netDrv** devices (FTP or RSH), **ll( )** does not give directory information. It is equivalent to an **ls( )** call with no long-listing option.

Note 3

ll command can not list file property under fd exhausted.

Returns

OK or **ERROR**.

Errno

Not Available

See also

usrFsLib, **dirList( )**, VxWorks 7 Applications Programmer's Guide, VxWorks 7 File Systems and I/O Programmer's Guide

Name

lsr( ) - list the contents of a directory and any of its subdirectories

Layer

FS_UTIL

Synopsis

```
STATUS lsr
    (
    const char * dirName   /* name of dir to list */
    )
```

Description

This function is simply a front-end for **dirList( )**, intended for brevity and backward compatibility. It produces a list of files and directories, without details such as file size and date, with recursion into subdirectories.

dirName is a name of a directory or file, and may contain wildcards.

Returns

OK or **ERROR**.

Errno

Not Available

See also

usrFsLib, **dirList( )**, VxWorks 7 Applications Programmer's Guide, VxWorks 7 File Systems and I/O Programmer's Guide

Name

llr( ) - do a long listing of directory and all its subdirectories contents

Layer

FS_UTIL

Synopsis

```
STATUS llr
    (
    const char * dirName   /* name of directory to list */
    )
```

Description

This command causes a long listing of a directory's contents to be displayed. It is equivalent to:

```
    -> dirList 1, dirName, 1, 1
```

dirName is a name of a directory or file, and may contain wildcards.

Note

When used with **netDrv** devices (FTP or RSH), **ll( )** does not give directory information. It is equivalent to an **ls( )** call with no long-listing option. The list depth of a directory listing is limited by the opened file num configured by the file system. If the depth of a directory listing exceeds the number allowed by the opened file num, the entire directory will not be listed.

Returns

OK or **ERROR**.

Errno

Not Available

See also

usrFsLib, **dirList( )**, VxWorks 7 Applications Programmer's Guide, VxWorks 7 File Systems and I/O Programmer's Guide

Name

cp( ) - copy file into other file/directory.

Layer

FS_UTIL

Synopsis

```
STATUS cp
    (
    const char * src,   /* source file or wildcard pattern */
    const char * dest   /* destination file name or directory */
    )
```

Description

This command copies from the input file to the output file. If destination name is directory, a source file is copied into this directory, using the last element of the source file name to be the name of the destination file.

This function is very similar to **copy( )**, except it is somewhat more similar to the UNIX "cp" program in its handling of the destination.

src may contain a wildcard pattern, in which case all files matching the pattern will be copied to the directory specified in dest. This function does not copy directories, and is not recursive. To copy entire subdirectories recursively, use **xcopy( )**.

Examples

```
-> cp( "/sd0/FILE1.DAT","/sd0/dir2/f001.dat")
-> cp( "/sd0/dir1/file88","/sd0/dir2")
-> cp( "/sd0/*.tmp","/sd0/junkdir")
```

Returns

OK or **ERROR** if destination is not a directory while src is a wildcard pattern, or if any of the files could not be copied.

Errno

Not Available

See also

usrFsLib, **xcopy( )**, VxWorks 7 Applications Programmer's Guide, VxWorks 7 File Systems and I/O Programmer's Guide

Name

mv( ) - mv file into other directory.

Layer

FS_UTIL

Synopsis

```
STATUS mv
    (
    const char * src,   /* source file name or wildcard */
    const char * dest   /* destination name or directory */
    )
```

Description

This function is similar to **rename( )** but behaves somewhat more like the UNIX program "mv", it will overwrite files.

This command moves the src file or directory into a file which name is passed in the dest argument, if dest is a regular file or does not exist. If dest name is a directory, the source object is moved into this directory as with the same name, if dest is **NULL**, the current directory is assumed as the destination directory. src may be a single file name or a path containing a wildcard pattern, in which case all files or directories matching the pattern will be moved to dest which must be a directory in this case.

Examples

```
-> mv( "/sd0/dir1","/sd0/dir2")
-> mv( "/sd0/*.tmp","/sd0/junkdir")
-> mv( "/sd0/FILE1.DAT","/sd0/dir2/f001.dat")
```

Moving file requires source and destination in the same partition.

Returns

OK or error if any of the files or directories could not be moved, or if src is a pattern but the destination is not a directory.

Errno

Not Available

See also

usrFsLib, VxWorks 7 Applications Programmer's Guide, VxWorks 7 File Systems and I/O Programmer's Guide

Name

xcopy( ) - copy a hierarchy of files with wildcards

Layer

FS_UTIL

Synopsis

```
STATUS xcopy
    (
    const char * source,   /* source directory or wildcard name */
    const char * dest      /* destination directory */
    )
```

Description

source is a string containing a name of a directory, or a wildcard or both which will cause this function to make a recursive copy of all files residing in that directory and matching the wildcard pattern into the dest directory, preserving the file names and subdirectories.

Caveat

This function may call itself in accordance with the depth of the source directory, and allocates 3 kB of heap memory per stack frame, meaning that to accommodate the maximum depth of subdirectories which is 20, at least 60 kB of heap memory should be available.

Returns

OK, or **ERROR** if any operation has failed.

Errno

Not Available

See also

usrFsLib, **tarLib**, **cp( )**, VxWorks 7 Applications Programmer's Guide, VxWorks 7 File Systems and I/O Programmer's Guide

Name

xdelete( ) - delete a hierarchy of files with wildcards

Layer

FS_UTIL

Synopsis

```
STATUS xdelete
    (
    const char * source   /* source directory or wildcard name */
    )
```

Description

source is a string containing a name of a directory, or a wildcard or both which will cause this function to recursively remove all files and subdirectories residing in that directory and matching the wildcard pattern. When a directory is encountered, all its contents are removed, and then the directory itself is deleted.

Note that the wildcard matching is limited to a single directory level.

```
    dir         is valid
    *.c         is valid
    dir/ *.c    is valid
    *a/ *.c     is not valid
```

Returns

OK or **ERROR** if any operation has failed.

Errno

Not Available

See also

usrFsLib, **cp( )**, **copy( )**, **xcopy( )**, **tarLib**, VxWorks 7 Applications Programmer's Guide, VxWorks 7 File Systems and I/O Programmer's Guide

Name

attrib( ) - modify MS-DOS file attributes on a file or directory

Layer

FS_UTIL

Synopsis

```
STATUS attrib
    (
    const char * fileName,   /* file or dir name on which to change flags */
    const char * attr        /* flag settings to change */
    )
```

Description

This function provides means for the user to modify the attributes of a single file or directory. There are four attribute flags which may be modified: "Archive", "System", "Hidden" and "Read-only". Among these flags, only "Read-only" has a meaning in VxWorks, namely, read-only files can not be modified deleted or renamed.

The attr argument string may contain must start with either "+" or "-", meaning the attribute flags which will follow should be either set or cleared. After "+" or "-" any of these four letter will signify their respective attribute flags - "A", "S", "H" and "R".

For example, to write-protect a particular file and flag that it is a system file:

```
-> attrib( "bootrom.sys", "+RS")
```

Returns

OK, or **ERROR** if the file can not be opened.

Errno

Not Available

See also

usrFsLib, **dosFsLib**, VxWorks 7 Applications Programmer's Guide, VxWorks 7 File Systems and I/O Programmer's Guide

Name

xattrib( ) - modify MS-DOS file attributes of many files

Layer

FS_UTIL

Synopsis

```
STATUS xattrib
    (
    const char * source,   /* file or dir name on which to change flags */
    const char * attr      /* flag settings to change */
    )
```

Description

This function is essentially the same as **attrib( )**, but it accepts wildcards in fileName, and traverses subdirectories in order to modify attributes of entire file hierarchies.

The attr argument string may contain must start with either "+" or "-", meaning the attribute flags which will follow should be either set or cleared. After "+" or "-" any of these four letter will signify their respective attribute flags - "A", "S", "H" and "R".

Example

```
-> xattrib( "/sd0/sysfiles", "+RS") /* write protect "sysfiles" */
-> xattrib( "/sd0/logfiles", "-R")  /* unprotect logfiles before deletion */
-> xdelete( "/sd0/logfiles")
```

Caveat

This function may call itself in accordance with the depth of the source directory, and allocates 2 kB of heap memory per stack frame, meaning that to accommodate the maximum depth of subdirectories which is 20, at least 40 kB of heap memory should be available.

Returns

OK, or **ERROR** if the file can not be opened.

Errno

Not Available

See also

usrFsLib, **dosFsLib**, VxWorks 7 Applications Programmer's Guide, VxWorks 7 File Systems and I/O Programmer's Guide

Name

dosfsDiskFormat( ) - format a disk with dosFs

Layer

FS_UTIL

Synopsis

```
STATUS dosfsDiskFormat
    (
    const char * pDevName   /* name of the device to initialize */
    )
```

Description

This command formats a disk and creates the dosFs file system on it. The device must already have been created by the device driver and dosFs format component must be included.

Example

```
    -> dosfsDiskFormat "/fd0"
```

Returns

OK, or **ERROR** if the device cannot be opened or formatted.

Errno

Not Available

See also

usrFsLib, **dosFsLib**, VxWorks 7 Applications Programmer's Guide, VxWorks 7 File Systems and I/O Programmer's Guide

Name

diskFormat( ) - format a disk with dosFs

Layer

FS_UTIL

Synopsis

```
STATUS diskFormat
    (
```

```
    const char * pDevName   /* name of the device to initialize */
    )
```

Description

This command in now obsolete. Use dosfsDiskFormat or **dosFsVolFormat( )** instead

This command formats a disk and creates the dosFs file system on it. The device must already have been created by the device driver and dosFs format component must be included.

Example

```
    -> diskFormat "/fd0"
```

Returns

OK, or **ERROR** if the device cannot be opened or formatted.

Errno

Not Available

See also

usrFsLib, **dosFsLib**, VxWorks 7 Applications Programmer's Guide, VxWorks 7 File Systems and I/O Programmer's Guide

Name

hrfsDiskFormat( ) - format a disk with HRFS

Layer

FS_UTIL

Synopsis

```
STATUS hrfsDiskFormat
    (
    const char * pDevName,   /* name of the device to initialize */
    int          files,      /* the maximum number of files to support */
    UINT32       majorVer,   /* major version of fs to format */
    UINT32       minorVer,   /* minor version of fs to format */
    UINT32       options     /* formatter options */
    )
```

Description

This command formats a disk and creates the HRFS file system on it. The device must already have been created by the device driver and HRFS format component must be included.

Example

```
    -> hrfsDiskFormat "/fd0", 0    /* format "/fd0" with HRFS */
                                   /*allowing maximum files */
    -> hrfsDiskFormat "/fd0", 100  /* format "/fd0" with HRFS */
                                   /*allowing 100 files */
```

Returns

OK, or **ERROR** if the device cannot be opened or formatted.

Errno

Not Available

See also

usrFsLib, **hrFsLib**, VxWorks 7 Applications Programmer's Guide, VxWorks 7 File Systems and I/O Programmer's Guide

Name

commit( ) - commit current transaction to disk.

Layer

FS_UTIL

Synopsis

```
STATUS commit
    (
    const char * pDevName   /* name of the device to commit */
    )
```

Description

This command is for transactional based file systems only such as HRFS. It is a shortcut for the ioctl function FIOCOMMITFS which commits the current transaction to disk to make changes permanment.

Example

```
    -> commit "/ata0a"             /* commit transaction on "/fd0" */
```

Returns

OK, or **ERROR** if the device is not formatted with a file system
that does not support the FIOCOMMITFS ioctl function or pDevName is not valid.

Errno

Not Available

See also

usrFsLib, **hrFsLib**, VxWorks 7 Applications Programmer's Guide, VxWorks 7 File Systems and I/O Programmer's Guide

Name

ioHelp( ) - print a synopsis of I/O utility functions

Layer

FS_UTIL

Synopsis

```
void ioHelp (void)
```

Description

This function prints out synopsis for the I/O and File System utility functions.

Returns

N/A

Errno

Not Available

See also

usrFsLib, VxWorks 7 Applications Programmer's Guide, VxWorks 7 File Systems and I/O Programmer's Guide

36. VNODEAFF

Name

vnodeAff - Abstract Filesystem Framework for Vnode Library

Layer

FS

Routines

No user-callable routines

Description

This module provides an abstract filesystem framework (AFF) for the vnode library. The AFF provides an interface to the core I/O driver table, and also manages file entities and buffers in a unified fashion.

The module provides core I/O register routines for **create( )**, **delete( )**, **open( )**, **close( )**, **read( )**, **write( )**, and **ioct( )** requests.

It processes I/O requests from core I/O and dispatches them to the filesystem driver. It calls **vnodeAffLibInit( )** to register I/O routines with core I/O, and dispatches requests to a filesystem that has been registered with it. The filesystem registers its filesystem routines with this module.

This diagram illustrates the relationship between these elements, using HRFS as an example for the filesystem.

```
+---------------------------------+
|            Core I/O             |
+---------------------------------+
                |
+---------------------------------+
|  Abstract Filesystem Framework  |
+---------------------------------+
                |
+---------------------------------+
|  Vnode base file system (HRFS)  |
+---------------------------------+
```

Ioctl functions

While the AFF module processes ioctl request from core I/O, some work may be done by the filesystem itself.

FIOSEEK

Sets the current byte offset of the specific file to the position specified by newOffset, range from 0 to **UINT_MAX**:

```
unsigned int newOffset = ...;
status = ioctl (fd, FIOSEEK, newOffset);
```

FIOSEEK64

Changes the current file position. This is the 64 bit version.

```
long long newOffset = 0;
status = ioctl (fd, FIOSEEK64, (_Vx_ioctl_arg_t)&newOffset);
```

FIOWHERE

Returns the current byte position from the start of the file for the specified file descriptor, returns value range from 0 to **UINT_MAX** - 1, **ERROR** on failure. This is the byte offset of the next byte to be read or written. It takes no additional argument:

```
position = ioctl (fd, FIOWHERE, 0);
```

FIOWHERE64

Returns the current byte position from the start of the file for the specified file descriptor. This is the 64 bit version of FIOWHERE.

```
long long where64 = 0;
status = ioctl (fd, FIOWHERE64, (_Vx_ioctl_arg_t)&where64);
```

FIOFLUSH

Writes all modified file descriptor buffers to the physical device.

```
status = ioctl (fd, FIOFLUSH, 0);
```

FIOSYNC

Performs the same function as FIOFLUSH.

FIOCOMMITFS

Performs the same function as FIOFLUSH.

FIODATASYNC

Performs the same function as FIOFLUSH.

FIONREAD

Copies to unreadCount the number of bytes from the current file position to the end of the file, support 32-bit value:

```
unsigned int unreadCount = 0;
status = ioctl (fd, FIONREAD, &unreadCount);
```

FIONREAD64

Copies to unreadCount the number of bytes from the current file position to the end of the file, this is the 64 bit version of FIONREAD:

```
long long unreadCount = 0;
status = ioctl (fd, FIONREAD64, (_Vx_ioctl_arg_t)&unreadCount);
```

FIONFREE

Copies to freeCount the amount of free space, in bytes, on the volume:

```
unsigned int freeCount = 0;
status = ioctl (fd, FIONFREE, (_Vx_ioctl_arg_t)&freeCount);
```

FIONFREE64

Copies to freeCount the amount of free space, in bytes, on the volume. This function can return value in 64-bit range:

```
long long freeCount = 0;
status = ioctl (fd, FIONFREE64, (_Vx_ioctl_arg_t)&freeCount);
```

FIORENAME

Moves the file or directory to the string newname, old file and new file should be on same volume:

```
fd = open("oldname", O_RDONLY, 0);
status = ioctl (fd, FIORENAME, "newname");
```

FIOMOVE

Performs the same function as FIORENAME.

FIOLINK

Links the file to the string newname. Directory can not be linked.

```
fd = open("oldname", O_RDONLY, 0);
status = ioctl (fd, FIOLINK, "newname");
```

FIOUNLINK

Removes a link to a file. It shall remove the link named by name and decrease the link count of the file referenced by the link. If link count gets to zero, the file may be removed. Directory can not be unlinked.

```
fd = open("name", O_RDONLY, 0);
status = ioctl (fd, FIOLINK, 0);
```

FIOCHMOD

Changes or assigns the newmode of a file. The mode of a file specifies its permissions and other attributes:

```
mode_t mode = <newmode>;
status = ioctl (fd, FIOCHMOD, (_Vx_ioctl_arg_t)mode);
```

FIORMDIR

Removes the directory whose name is specified as dirName:

```
status = ioctl (fd, FIORMDIR, "dirName");
```

FIOREADDIR

Reads the next directory entry. The argument dirStruct is a DIR directory descriptor. Normally, the **readdir( )** routine is used to read a directory, rather than using the FIOREADDIR function directly:

```
DIR dirStruct;
fd = open ("directory", O_RDONLY);
status = ioctl (fd, FIOREADDIR, &dirStruct);
```

FIOREADDIRPLUS

Performs the same function as FIOREADDIR.

FIOFSTATGET

Gets file status information (directory entry data). The argument statStruct is a pointer to a stat structure that is filled with data describing the specified file. Normally, the **stat( )** or **fstat( )** routine is used to obtain file information, rather than using the FIOFSTATGET function directly.

```
struct stat statStruct;
fd = open ("file", O_RDONLY);
status = ioctl (fd, FIOFSTATGET, (_Vx_ioctl_arg_t)&statStruct);
```

FIOFSTATFSGET

Return filesystem status information. The information contains filesystem type-id, block size, total blocks, and some other information. Normally, the **statfs( )** routine is used to obtain filesystem information, rather than using the FIOFSTATFSGET function directly.

```
struct statfs data;
status = ioctl (fd, FIOFSTATFSGET, (_Vx_ioctl_arg_t)&data);
```

FIOFSTATFSGET64

Return filesystem status information. The information contains filesystem type-id, block size, total blocks, and some other information. Normally, the **statfs64( )** routine is used to obtain filesystem information, rather than using the FIOFSTATFSGET64 function directly. This is the 64 bit version of FIOFSTATFSGET.

```
struct statfs64 data;
status = ioctl (fd, FIOFSTATFSGET64, (_Vx_ioctl_arg_t)&data);
```

FIOTRUNC

Sets the specified file's length to newLength bytes, supports 32-bit length range from 0 to **UINT_MAX**.

```
status = ioctl (fd, FIOTRUNC, newLength);
```

FIOTRUNC64

This is the 64 bit version of FIOTRUNC64.

```
long long newLength = .....;
status = ioctl (fd, FIOTRUNC64, (_Vx_ioctl_arg_t) &newLength);
```

FIOTIMESET

Update time on a file. arg shall be a pointer to a utimbuf structure, see **utime.h**. If arg is value **NULL**, the current system time is used for both actime and modtime members. If arg is not **NULL** then the utimbuf structure members actime and modtime are used as passed. See also **utime( )**

```
struct utimbuf newTimeBuf;;
newTimeBuf.modtime = newTimeBuf.actime = fileNewTime;
fd = open ("file", O_RDONLY);
status = ioctl (fd, FIOTIMESET, (_Vx_ioctl_arg_t)&newTimeBuf);
```

FIOCHOWN

Change or assign the owner and group of a file. arg is a pointer to **CHOWN_ARG**. If either owner or group in arg is set to -1, then that aspect of the file ownership is not changed. If both owner and group in arg is set to -1, the timestamp of file will not be updated.

```
CHOWN_ARG arg;

arg.owner = owner;
arg.group = group;

status = ioctl (fd, FIOCHOWN, (_Vx_ioctl_arg_t)&arg);
```

FIOFCNTL

Get or set file flag. See also **fcntl( )**.

```
FCNTL_ARG arg;
status = ioctl (fd, FIOFCNTL, (_Vx_ioctl_arg_t) &arg);
```

FIOACCESS

Determine accessibility of a file. See also **access( )**.

```
mode_t amode = ...;
status = ioctl (fd, FIOACCESS, amode);
```

FIOPATHCONF

Get the current value of a configurable limit. See also **pathconf( )**.

```
long arg = _PC_PATH_MAX;
status = ioctl ioctl (fd, FIOPATHCONF, (_Vx_ioctl_arg_t)&ret);
```

Include files

errnoLib.h fcntl.h ioLib.h iosLib.h pathLib.h limits.h vnode.h, **mount.h namei.h stat.h utime.h stdlib.h string.h dirent.h**, **taskLib.h private/vnodeAffLibP.h private/lockfP.h drv/xbd/xbd.h**, **fsMagic.h**

37. VRFSLIB

Name

vrfsLib - the Virtual Root File System

Layer

FS_VRFS

Routines

vrfsInit( ) - set VRFS function pointers and variables used by I/O system

Description

This component provides a file system representing the contents of the Core IO device table, thus providing a root file system, the elements of which are all Core IO devices, whose names begin with "/" and do not contain an embedded "/".

Include files

none

See also

VxWorks 7 File Systems and I/O Programmer's Guide

Name

vrfsInit( ) - set VRFS function pointers and variables used by I/O system

Layer

FS_VRFS

Synopsis

```
void vrfsInit (void)
```

Description

This is the initialization function for VRFS. All it does is set the function pointer **_func_vrfsDrv** so that the I/O system can call **vrfsDrv( )** at the appropriate point in its initialization: after creating the driver table, but before creating any devices. The VRFS root device, created by **vrfsDrv( )**, must be the first device created.

Returns

N/A

Errno

Not Available

See also

vrfsLib

38. VXBFDTALT5QSPIFLASH

Name

vxbFdtAlt5QspiFlash - Altera Soc Gen 5 QSPI Flash Driver

Layer

BDM_FLASH

Routines

No user-callable routines

Description

This module implements a vxBus driver for Altera SoC GEN 5 QQSPI NOR Flash chips.

Include files

none

39. VXBFSLIMXSDHCCTRL

Name

vxbFslImxSdhcCtrl - Freescale IMX SDHC host controller driver

Layer

BDM_SDMMC

Routines

No user-callable routines

Description

This is the vxbus compliant freescale Secure Digital Host Controller (SDHC) driver which implements the functionality specified in "SD Specifications Part A2 SD Host Controller Simplified Specification Version 2.00"

The SDHC provides an interface between the freescale host controller and SD/MMC memory cards.

To add the driver to the vxWorks image, add the following component to the kernel configuration.

```
vxprj component add DRV_IMX_SDHC_CTRL
```

Vxwork-7 mode

This driver is bound to device tree, and the device tree node must specify below parameters:

compatible

This parameter specifies the name of the SDHC controller driver. It should be "imx6Ai,sdhc", "imx6Sd,sdhc" or "imx8,sdhc".

reg

This parameter specifies the register base address and length of this module.

clock-frequency

This parameter specifies the working clock frequency of this module.

clocks

This parameter specifies the list of clocks that this module use.

interrupts

This parameter specifies the interrupt number of this module.

interrupt-parent

This parameter specifies the offset of interrupt controller.

cd_type

This parameter specifies the card detect type. 0 means no hardware connection for card detecting; 1 means using host controller to detect card; 2 means using GPIO pin to detect card;

WNDRVR

wp_type

This parameter specifies the write protect type. 0 means no hardware connection for write protect; 1 means using host controller to detect write protect; 2 means using GPIO pin to detect write protect;

cd_pin

This parameter specifies the card detect GPIO pin index.

wp_pin

This parameter specifies the write protect GPIO pin index.

pinmux-0

This parameter specifies the list of phandles that each pointing at a PinMux configuration node.

An example of device node is show below:

```
    sdhc0: sdhc@0x02190000
        {
        compatible = "imx6Ai,sdhc";
        reg = <0x02190000 0x4000>;
        clock-frequency = <198000000>;
        clocks = <&clk 52>;
        interrupts = <54>;
        interrupt-parent = <&intc>;
        cd_type = <2>;
        wp_type = <2>;
        cd_pin = <1>;
        wp_pin = <148>;
        pinmux-0 = <&usdhc1_2>;
        };
```

Include files

none

See also

vxBus, xbd, "SD Specifications Part A1 Physical Layer Simplified Specification Version 2.00", "SD Specifications Part A2 SD Host Controller Simplified Specification Version 2.00"

40. VXBFSLSDHCCTRL

Name

vxbFslSdhcCtrl - Freescale SDHC host controller driver

Layer

BDM_SDMMC

Routines

No user-callable routines

Description

This is the vxbus compliant freescale Secure Digital Host Controller (SDHC) driver which implements the functionality specified in "SD Specifications Part A2 SD Host Controller Simplified Specification Version 2.00"

The SDHC provides an interface between the freescale host controller and SD/MMC memory cards.

To add the driver to the vxWorks image, add the following component to the kernel configuration.

```
vxprj component add DRV_FSL_SDHC_CTRL
```

Vxwork-7 mode

This driver is bound to device tree, and the device tree node must specify below parameters:

comptible

This parameter specifies the name of the SDHC controller driver. It must be "fsl,sdhc".

reg

This parameter specifies the register base address and length of this module.

clock-frequency

This parameter specifies the working clock frequency of this module.

interrupts

This parameter specifies the interrupt number of this module.

interrupt-parent

This parameter specifies the offset of interrupt controller.

bus-width

This is used to support eMMC device only. Host controller selects maximum bus-width (4bit or 8bit) it supported for eMMC device. This parameter doesn't influence SD cards.

An example of device node is show below:

```
    sdhc@2e000 {
        compatible = "fsl,sdhc";
        reg = <0x2e000 0x1000>;
        clock-frequency = <250000000>;
        interrupts = <72 2 0 0>;
        interrupt-parent = <&mpic>;
        bus-width = <4>;
    };
```

Include files

none

See also

vxBus, "SD Specifications Part A1 Physical Layer Simplified Specification Version 2.00", "SD Specifications Part A2 SD Host Controller Simplified Specification Version 2.00"

41. VXBGPMC

Name

vxbGpmc - TI GPMC NAND Flash controller driver

Layer

BDM_FLASH

Routines

vxbGpmcProbe( ) - probe for device presence at specific address **vxbGpmcShow( )** - show GPMC information

Description

This module implements a driver for TI GPMC NAND Flash controller.

The GPMC (general-purpose memory controller) is an unified memory controller dedicated to interfacing external memory devices, including NAND Flash.

This module will only initialize the GPMC CS (chip select) for NAND Flash chips based on the platform type.

Hamming Code ECC algorithm is used by default, and only one ECC engine for all the chips attached to the controller.

The controller has one Prefetch/Postwrite engine only, so any buffer access will try to use it first.

DMA access method is supported by Prefetch/Postwrite engine.

External interface

The driver provides the standard vxbus external interface **vxbNorFlashRegister( )**. This function registers the driver with the vxbus subsystem, and instances will be created as needed. Since it is a processor local bus device, each device instance must be specified in the **hwconf.c** file in a BSP.The hwconf entry must specify the following parameters:

regBase

Specifies the base address where the GPMC are mapped into the host's address space. All command/data offsets are computed relative to this address.

regBase1

Specifies the base address where the chips connected to GPMC are mapped into the host's address space. The address must be aligned with the size of each CS - 16M bytes.

platType

Specifies the type of the platform where GPMC controller is located. The default board type 0 will be used.

flag

specifies which features should be disabled. All the suported features will be enabled by default.

An example hwconf entry is shown below:

```
struct hcfResource gpmc0Resources[]= {
    { "regBase",        HCF_RES_INT,    { (void *) (TI81XX_GPMC_BASE) } },
```

```
    { "regBase1",       HCF_RES_INT,    { (void *) (TI81XX_GPMC_CS_BASE) } },

    /* optional for special purpose */

    { "platType",       HCF_RES_INT,    { (void *) (GPMC_PLAT_TI_DM81XX_EVM) } },
    { "flag",           HCF_RES_INT,    { (void *) (GPMC_FLAG_PFPWDIS | GPMC_FLAG_DMADIS | GPM
C_FLAG_CACHEDIS | GPMC_FLAG_ZCOPYDIS) } }
    };
#define gpmc0Num NELEMENTS(gpmc0Resources)
```

Include files

flash/vxbGpmcCore.h stdio.h vxBusLib.h, **hwif/util/vxbParamSys.h**

Name

vxbGpmcProbe( ) - probe for device presence at specific address

Layer

BDM_FLASH

Synopsis

```
STATUS vxbGpmcProbe
    (
    struct vxbDev * pDev
    )
```

Description

Check for GPMC contoller device at the specified base address.

Returns

OK if probe passes, **ERROR** otherwise.

Errno

Not Available

See also

vxbGpmc

Name

vxbGpmcShow( ) - show GPMC information

Layer

BDM_FLASH

Synopsis

```
STATUS vxbGpmcShow
    (
    VXB_DEV_ID    pDev,
    VXB_DEVICE_ID pDev,
    UINT32        verbose
    )
```

Description

This routine prints the GPMC information.

Returns

OK always

Errno

Not Available

See also

vxbGpmc

42. VXBGPMI

Name

vxbGpmi - freescale IMX GPMI nand controller driver

Layer

BDM_FLASH

Routines

vxbGpmiProbe( ) - probe for device presence at specific address

Description

This module implements a driver for freescale GPMI (General Purpose Media Interface) NAND Flash controller.

This module will only initialize the GPMI CS (chip select) for NAND Flash chips based on the platform type.

Include files

vxbGpmi.h vxbGpmiCore.h stdio.h vxBus.h vxbFdtLib.h

Name

vxbGpmiProbe( ) - probe for device presence at specific address

Layer

BDM_FLASH

Synopsis

```
STATUS vxbGpmiProbe
    (
    struct vxbDev * pDev
    )
```

Description

Check for GPMI NAND contoller device at the specified base address.

Returns

OK if probe passes, **ERROR** otherwise.

Errno

Not Available

See also

vxbGpmi

43. VXBNANDFLASHCMDSETLIB

Name

vxbNandFlashCmdSetLib - NAND Flash generic Command Set Library

Layer

BDM_FLASH

Routines

vxbNandSetInternalEcc( ) - enable/disable nand internal ECC

Description

This library provides the command-set-specific routines for generic NAND Flash chips.

This library identifies NAND Flash by general identify method, and then set up the suitable command sequence set, bad block flag layout, and block szie, etc.

ONFI identify method is used if there is no any matched device in nandFlashIdTable[]. But if ONFI is unsupported by the device, then one new item with the necessary parameters must be added to the nandFlashIdTable[].

Include files

ffsLib.h flash/vxbNandFlashLib.h

Name

vxbNandSetInternalEcc( ) - enable/disable nand internal ECC

Layer

BDM_FLASH

Synopsis

```
STATUS vxbNandSetInternalEcc
    (
    pNAND_FLASH_DESC pNandDesc,
    BOOL             enableEcc
    )
```

Description

This routine is enable/disable nand internal ECC

Returns

OK or **ERROR** if failed

Errno

N/A

See also

vxbNandFlashCmdSetLib

44. VXBNORFLASH

Name

vxbNorFlash - Generic NOR-Flash vxBus Driver

Layer

BDM_FLASH

Routines

No user-callable routines

Description

This module implements a driver for various NOR Flash chips.

This module identifies the NOR Flash chips by CFI interface, JEDEC interface, or user-defined legacy routines.

CFI(Common Flash memory Interface) method is supported widely by NOR Flash Vendors, and can be used to obtain device information and topology for NOR Flash, such as Command Set, Erase Block Region information, Operation voltage and timing, interface type, etc.

JEDEC method is supported by some older NOR Flash chips, and can be used to obtain Manufacturer and Device ID, then search one pre-defined table to decide the interface type, Command Set, etc.

Another method is that BSPs supply one legacy routine to identify the devices, but it is not recommended. One new item should always be added to the pre-defined table for JEDEC method.

This module doesn't include command set support. So the corresponding command set library must be include by the corresponding components, such as **DRV_VXB_NORFLASH_CMDSET_AMD**, **DRV_VXB_NORFLASH_CMDSET_ITL**. And new command sets or variants can be added easily if necessary.

Note: For the Flash chips that have several region with different block size, there are 2 assumptions:

1. The bigger block size is equal with the multiple of the less one;

2. The size of any region is equal with the multiple of the maximum block size.

The 2 assumptions are eastablished for the current known Flash Chips. If one exception is found, the module need be adjusted.

The Flash information and operation routines are composed into one Flash chip information structure, and can be obtained the method - **vxbFlashChipInfoGet( )**.

Note:The Nor Flash must be enabled before any other Flash operations are used, and disabled after any other Flash operations are finished.

Smp considerations

This driver uses atomic variables and semaphores to manage critical sections so that it can be safely used in SMP configurations.

Board layout

Some NOR Flash chips may be write-protected by jumpers.

External interface

The driver provides the standard vxbus external interface **vxbNorFlashRegister( )**. This function registers the driver with the vxbus subsystem, and instances will be created as needed. Since it is a processor local bus device, each device instance must be specified in the **hwconf.c** file in a BSP.The hwconf entry must specify the following parameters:

regBase

Specifies the base address where the NOR Flash chip are mapped into the host's address space. All command/data offsets are computed relative to this address.

maxSize

Specifies the size of the address space where the NOR Flash chip are mapped into the host's address space.

cfiPrior

Specifies whether CFI interface is used to identify the Flash chips first. If set, the default value 1 will be used and CFI interface will be used first. If it is set to 0, JEDEC interface will be used first.

bigEndian

Specifies whether the Flash chips are big-endian device. If not set, the big-endian mode will be tried first, and then little-endian will be tyied secondly.

portWidth

Specifies the width of the system port to the Flash chips. If not set, all the possible width will be tried to identify the Flash chips.

chipNum

Specifies the number of the Flash chips that are paralleled to obtain the bigger port width. If not set, all the possible number will be tried to identify the Flash chips.

chipWidth

Specifies the maximum bus width of the Flash chips. If not set, all the possible width will be tried to identify the Flash chips.

wrEna

Specifies the address of the write-enable routine supplied by BSPs. For some boards, the write-protection must be disabled or the VPP must be turned on before any write operations are sent to the Flash chips.

sharedMem

Specifies the location of the shared memory anchor. This must be set correctly for AMP systems.

legacyProbe

Specifies the address of the legacy identification routine supplied by BSPs. If Set, the CFI interface and JEDEC interface will be skipped.

An example hwconf entry is shown below:

```
struct hcfResource norFlash0Resources[]= {
    { "regBase",        HCF_RES_INT,    { (void *) (FLASH_BASE_ADRS) } },
    { "maxSize",        HCF_RES_INT,    { (void *) (FLASH_SIZE) } },
#ifdef INCLUDE_AMP_CPU
    { "sharedMem",      HCF_RES_INT,    { (void *) TM_ANCHOR_ADRS } }
#endif @* INCLUDE_AMP_CPU *@
```

```
    /* optional for special purpose */

    { "wrEna",          HCF_RES_ADDR,   { (void *) &sysFlashWriteEna } },
    { "legacyProbe",    HCF_RES_ADDR,   { (void *) &sysFlashProbe } }

    /* optional to speed initialization process */

    { "cfiPrior",       HCF_RES_INT,    { (void *) 1 } },
    { "bigEndian",      HCF_RES_INT,    { (void *) 1 } },
    { "portWidth",      HCF_RES_INT,    { (void *) 16 } },
    { "chipWidth",      HCF_RES_INT,    { (void *) 16 } },
    { "chipNum",        HCF_RES_INT,    { (void *) 1 } }
    };
#define norFlash0Num NELEMENTS(norFlash0Resources)
```

If not use the legacy vxbus, to add the driver to your vxWorks image, add the following component to the kernel configuration.

```
prj vip component add DRV_VXB_NORFLASH
```

This driver is bound to device tree, and the device tree node must specify below properties:

compatible

This property specifies the programming model for the device and is used by vxbus GEN2 for device driver selection. It should be set to one of below: "spansion,cfi-flash" - For spansion norflash "micron,cfi-flash" - For micron norflash

reg

This property specifies the register base address and length of this module.

max-size

The maxSize resource specifies the size of the address space where the NOR Flash chip are mapped into the host's address space.

cfi-prior

The cfiPrior resource specifies whether CFI interface is used to identify the Flash chips first. If set, the default value 1 will be used and CFI interface will be used first. If it is set to 0, JEDEC interface will be used first.

big-endian

The bigEndian resource specifies whether the Flash chips are big-endian device. If not set, the big-endian mode will be tried to identify the Flash chips first, and then little-endian will be tried.

port-width

The portWidth resource specifies the width of the system port to the Flash chips. If not set, all the possible width will be tried to identify the Flash chips.

chip-width

The chipWidth resource specifies the maximum bus width of the Flash chips. If not set, all the possible width will be tried to identify the Flash chips.

chip-num

The chipNum resource specifies the number of the Flash chips that are paralleled to obtain the bigger port width. If not set, all the possible number will be tried to identify the Flash chips.

```
norflash: norflash@60000000
    {
    compatible = "micron,cfi-flash";
    reg = <0 0x60000000 0 0x08000000>;
    max-size = <0x8000000>;
    port-width = <16>;
    chip-width = <16>;
    };
```

SEE ALSO: vxBus, **vxbNorFlashAmdCmdSetLib**, **vxbNorFlashItlCmdSetLib** "VxWorks 7 BSP and Driver Guide", "Spansion App Note - CFI Spec", "Spansion CFI Vendor & Device ID Code Assignments"

Include files

none

45. VXBNORFLASHAMDCMDSETLIB

Name

vxbNorFlashAmdCmdSetLib - AMD NOR Flash Command Set Library

Layer

BDM_FLASH

Routines

vxbNorFlashAmdCmdSetRegister( ) - register command set to vxbNorFlash driver

Description

This library provides the command-set-specific routines for AMD/Spansion NOR Flash chips.

The libarary registers the initilaization/clean routines to vxBus NOR-Flash driver for the corresponding command sets by **vxbNorFlashCmdSetReg( )**.

The initilaization routine need set up the supported features and the corresponding command sequences and/or install/overload the function hooks.

Note: The vxBus NOR-Flash driver has installed the default hook routines for program/block erase/write to buffer/block lock. The specified hook routines can be implemented here if necessary.

The unlock addresses has been installed by the vxBus NOR-Flash driver, and the vendor/device ID or the unlock addresses should be obtained. Then they can be used to decide the right command sequences or the function hooks.

Note: The current status polling routine just makes use of one common method, and can be replaced with the device-specified polling routine if it is necessary.

To use the command set library, just include the corresponding macro, **DRV_VXB_NORFLASH_CMDSET_AMD**.

Include files

none

See also

vxbNorFlash, "Spansion S29GL-N MirrorBit Flash Family Datasheet", "Spansion S29WS-P MirrorBit Flash Family Datasheet", "VxWorks 7 BSP and Driver Guide"

Name

vxbNorFlashAmdCmdSetRegister( ) - register command set to vxbNorFlash driver

Layer

BDM_FLASH

Synopsis

```
void vxbNorFlashAmdCmdSetRegister (void)
```

Description

This routine registers the command set library to vxbNorFlash driver.

Returns

N/A

Errno

N/A

See also

vxbNorFlashAmdCmdSetLib

46. VXBNORFLASHITLCMDSETLIB

Name

vxbNorFlashItlCmdSetLib - Intel NOR Flash Command Set Library

Layer

BDM_FLASH

Routines

vxbNorFlashItlCmdSetRegister( ) - register command set to vxbNorFlash driver

Description

This library provides the command-set-specific routines for Intel NOR Flash chips.

The libarary registers the initilaization/clean routines to vxBus NOR-Flash driver for the corresponding command sets by **vxbNorFlashCmdSetReg( )**.

The initilaization routine need set up the supported features and the corresponding command sequences and/or install/overload the function hooks.

Note: The vxBus NOR-Flash driver has installed he default hook routines for program/block erase/write to buffer/block lock, but not for block lock query. So one routine must be installed for the operation in this library.

The vendor/device ID or the unlock addresses should be obtained and can be used to decide the right command sequences or the function hooks.

To use the command set library, just include the corresponding macro, **DRV_VXB_NORFLASH_CMDSET_ITL**.

Include files

none

See also

vxbNorFlash, "Intel StrataFlash Memory Datasheet", "VxWorks 7 BSP and Driver Guide"

Name

vxbNorFlashItlCmdSetRegister( ) - register command set to vxbNorFlash driver

Layer

BDM_FLASH

Synopsis

```
void vxbNorFlashItlCmdSetRegister (void)
```

Description

This routine registers the command set library to vxbNorFlash driver.

Returns

N/A

Errno

N/A

See also

vxbNorFlashItlCmdSetLib

47. VXBNORFLASHLIB

Name

vxbNorFlashLib - NOR Flash Library

Layer

BDM_FLASH

Routines

vxbNorFlashLibInit( ) - initialize Flash library **vxbNorFlashLibClean( )** - clean resources for Flash libaray
vxbNorFlashCmdSetReg( ) - register command set **vxbNorFlashShow( )** - show Flash driver deatiled information

Description

This library provides the interfaces to connect the identify method libraries and command set libraries to the NOR-Flash vxBus driver.

Include files

none

See also

vxbNorFlash **vxbNorFlashCfiLib vxbNorFlashJedecLib**, **vxbNorFlashAmdCmdSetLib vxbNorFlashItlCmdSetLib**, VxWorks 7 BSP and Driver Guide

Name

vxbNorFlashLibInit( ) - initialize Flash library

Layer

BDM_FLASH

Synopsis

```
STATUS vxbNorFlashLibInit
    (
    NOR_FLASH_INFO * pFlashInfo
    )
```

Description

This routine identifies and sets up the information for the Flash chip.

Returns

OK or **ERROR** if failed.

Errno

N/A

See also

vxbNorFlashLib

Name

vxbNorFlashLibClean( ) - clean resources for Flash libaray

Layer

BDM_FLASH

Synopsis

```
void vxbNorFlashLibClean
    (
    NOR_FLASH_INFO * pFlashInfo
    )
```

Description

This routine cleans the resources for the Flash libaray.

Returns

N/A

Errno

N/A

See also

vxbNorFlashLib

Name

vxbNorFlashCmdSetReg( ) - register command set

Layer

BDM_FLASH

Synopsis

```
void vxbNorFlashCmdSetReg
    (
    UINT32                      cmdSet,
```

```
    NOR_FLASH_CMDSET_INIT_RTN initRtn,
    NOR_FLASH_CMDSET_CLEAN_RTN cleanRtn
    )
```

Description

This routine registers the command set initialization routine for the command set specified by cmdSet.

Returns

N/A

Errno

N/A

See also

vxbNorFlashLib

Name

vxbNorFlashShow( ) - show Flash driver deatiled information

Layer

BDM_FLASH

Synopsis

```
STATUS vxbNorFlashShow
    (
    NOR_FLASH_INFO * pFlashInfo,
    int              verbose
    )
```

Description

This routine prints the Flash driver deatiled information.

Returns

OK always

Errno

Not Available

See also

vxbNorFlashLib

48. VXBPCISDHCCTRL

Name

vxbPciSdhcCtrl - PCI SDHC host controller driver

Layer

BDM_SDMMC

Routines

No user-callable routines

Description

This is the vxbus compliant PCI Secure Digital Host Controller (SDHC) driver which implements the functionality specified in "SD Specifications Part A2 SD Host Controller Simplified Specification Version 2.00"

The SDHC provides an interface between the PCI host controller and SD/MMC memory cards.

This driver is used to SDHC controllers on PCI/PCIe bus. No device tree node is needed. However, there can be some specific configuration in a device tree node.

no-sd

> Specifies if send SD special commands (such as CMD8) during initialization. By default, CMD8 is sent to device. But to some SDHC, if it is connected with (e)MMC, CMD8 may cause not to work. In this case, need set it to 1 in dts.

An example of a device node configuration is as follows:

```
    devparam {
        pciSdhc@0 {
            no-sd = <0>;
            };
    };
```

Include files

none

See also

vxBus, "Intel ICH manual", "SD Specifications Part A1 Physical Layer Simplified Specification Version 2.00", "SD Specifications Part A2 SD Host Controller Simplified Specification Version 2.00"

49. VXBRCARH3MMC

Name

vxbRcarH3Mmc - Renesas R-Car H3 MMC driver

Layer

BDM_SDMMC

Routines

No user-callable routines

Description

This is the Vxbus compliant MMC interface driver which implements the functionality specific to the Renesas R-Car H3.

To add the driver to the vxWorks image, add the following component to the kernel configuration.

```
vxprj component add DRV_RCAR_H3_MMC
```

This driver is bound to the device tree, and the device tree node must specify the following parameters:

compatible

This parameter specifies the name of the MMC controller driver. It must be "renesas,rcar-h3-mmc".

reg

This parameter specifies the register base address and length of this module.

interrupts

This parameter specifies the interrupt number of this module.

interrupt-parent

This parameter specifies the offset of interrupt controller.

pinmux-0

This parameter specifies the default pin mux configuration, if any. The property encoding is defined by the pin mux source binding.

clocks

This parameter specifies the list of clocks that this module uses.

clock-names

This parameter specifies the name of every clock in clocks list. The "disableClk" is only required to disable the unused SD input clock if the MMC interface is implemented using 2 SD interfaces.

voltage-support

This parameter specifies the list of MMC interface voltages supported by the hardware. It must be at least one of "1V8" and "3V3".

driver-type

This parameter specifies the MMC driver type value, used to select the driver strength for HS200 and HS400. Refer to the eMMC Standard for further information.

cd-pin

This parameter specifies the card detect GPIO pin index. Using 0xfffe means the target device always exist for eMMC.

wp-pin

This parameter specifies the write protect GPIO pin index. Using 0xffff means the target device is always writeable.

An example of a device node is shown below:

```
    mmc0: mmc@ee140000
        {
        compatible = "renesas,rcar-h3-mmc";
        reg = <0xee140000 0x2000>;
        interrupts = <199 0 4>;     /* SDHI.ch2 */
        interrupt-parent = <&gic>;
        clocks = <&cpg, 39>,        /* SD2 is MMC0 input clock */
                 <&cpg 38>,         /* SD2H is SD2 parent */
                 <&cpg 37>,         /* SD1 is disabled for MMC0 */
                 <&cpg, 101>;       /* SD-IF2 is MMC0 module clock */
        clock-names = "inputClk", "parentClk", "disableClk", "moduleClk";
        pinmux-0;
        cd-pin   = <0xfffe>;        /* always exist for eMMC */
        wp-pin   = <0xffff>;        /* always writeable */
        voltage-support = "1V8";
        driver-type = <1>;
        status = "OK";
        }
```

This driver supports the following bus speed modes: Legacy 26MHz High Speed SDR 52MHz High Speed DDR 53MHz HS200 (200MHz SDR) HS400 (200MHz DDR) Not available in ES1.x

Include files

none

See also

vxBus, VxWorks 7 BSP and Driver Guide, R-Car Series, 3rd Generation User's Manual: Hardware, Embedded Multi-Media Card (eMMC) Electrical Standard (5.0)

50. VXBRZN1QSPIFLASH

Name

vxbRzn1QspiFlash - Renesas RZ/N1 QSPI Flash Controller Driver

Layer

BDM_FLASH

Routines

No user-callable routines

Description

This module implements a vxBus driver for Renesas RZ/N1 QSPI controller.

To add the driver to the vxWorks image, add the following component to the kernel configuration.

```
vxprj component add DRV_RZN1_QSPI_FLASH
```

Each QSPI Flash Controller device should be bound to a device tree node which requires below properties:

compatible

This parameter specifies the name of the QSPI controller driver. It should be "renesas,rzn1-qspi".

reg

This parameter specifies the register base address and length of this module. 2 memory resources are required: the first is controller registers; the second is AHB memory where full flash conntent is mapped.

interrupts

This parameter specifies the interrupt number of this module.

interrupt-parent

This parameter specifies the offset of interrupt controller.

pinmux-0

This parameter specifies the list of phandles that each pointing at a PinMux configuration node.

clocks

This parameter specifies the list of clocks that this module use.

renesas,rzn1-cs_mux

Optional. When defined, this property indicate usage of **mux-CS** instead of standard CS mode for chip selection.

Each serial flash device component must have his own device tree which requires below properties:

reg

This parameter specifies the number of chip select.

chip-name

This parameter specifies a string used by VxWorks flash library for selection.

feature-flag

Optional. This parameter specifies device feature flag. if it is not defined, driver will get it from known manufacture id.

spi-max-frequency

This parameter specifies maximum work frequency.

write-protect

Optional. This parameter specifies software write protected area(s) 1st_block n_blocks.

Define 4 couples of < **INSTR_RD_CFG INSTR_WR_CFG** > to be directly used for rw instruction. If none are defined, a default -norm- one is set. renesas,rzn1-norm-instr: < **INSTR_RD_CFG INSTR_WR_CFG** > for normal mode renesas,rzn1-fast-instr: < **INSTR_RD_CFG INSTR_WR_CFG** > for fast mode renesas,rzn1-dual-instr: < **INSTR_RD_CFG INSTR_WR_CFG** > for dual mode renesas,rzn1-quad-instr: < **INSTR_RD_CFG INSTR_WR_CFG** > for quad mode

rzn1-mode-instr

This parameter selects a mode in "norm", "fast", "dual" or "quad".

Device Feature Flag: Usually, manufacturer ident (lsb of device Id.) is used to define which and how commands should be used to realize a feature like a reset or reading status. For example, Spansion flashes require usage of command BRWR (0x17) to select 4 bytes address mode. Instead of defining all these information into a manufacture data base, requested features have been codified, several cases developed and user has to define relevant flag through dts. Spansion and Micron have been used has reference because they are often used and already tested with same Cadence QSPI controller on another design.

Here are current developed features + DFF coding:

```
b1-b0 = Device reset:
    0: use CFI reset command
    1: MICRON use Reset Enable + Reset Memory commands

b3-b2 = Set 3 or 4 Bytes address width:
    0: minimal = set 3/4Bytes addresse width into dev_size register
    1: Spansion 3/4 Bytes addresse width, using cmd 0x17 write bank address
    2: Micron 3/4 Bytes addresse width

b4 = Wait flash ready:
    1: use RD_FLAG_STATUS in wait flash ready (Only Spansion do such)
```

Below is an example:

```
    qspi0: qspi0@RZN1_QSPI_BASE
        {
        compatible = "renesas,rzn1-qspi";
        status = "okay";
        #address-cells = <1>;
        #size-cells = <0>;
        reg = <0x40005000 0x1000>,      @* ctrl registers *@
              <0x10000000 0x10000000>;  @* ahb mem 256MB  *@
        interrupts = <96 0 0x104>;      @* RZN1_IRQ_QSPI0=64 +32 *@
        interrupt-parent = <&gic>;
```

```
        pinmux-0 = <&qspi0_mux>;
        clocks = <&qspi0_clk>;
        clock-names = "qspi0_clk";

        qspiflash@0
            {
            #address-cells = <1>;
            #size-cells = <1>;
            reg = <0>;                      @* chip select *@
            chip-name = "qspiflash";        @* for flash lib. *@
            spi-max-frequency = <100000000>;
            write-protect = <0 17>;         @* sw write protected *@

            @* rzn1-xxxx-instr =  < INSTR_RD_CFG INSTR_WR_CFG >
             * Define up to 4 couples of < INSTR_RD_CFG INSTR_WR_CFG >.
             * If none are defined, a default -norm- one is set.
             *@
            renesas,rzn1-norm-instr = <0x06000003  0x00000002>;
            renesas,rzn1-fast-instr = <0x0600000B  0x00000002>;
            renesas,rzn1-dual-instr = <0x0601113B  0x00000002>;
            renesas,rzn1-quad-instr = <0x060220EB  0x00000002>;

            @* select the instruction mode inbtw "norm",..,"quad" *@
            rzn1-mode-instr = "quad";
            };
        };
```

Include files

vxBus.h vxbIntLib.h vxbFlashLib.h

See also

vxBus

51. VXBRZN1SDHC

Name

vxbRzn1Sdhc - RZ/N1 SD/MMC Host Controller Driver

Layer

BDM_SDMMC

Routines

No user-callable routines

Description

This is the vxBus compliant SD/MMC Host Controller driver (ARASAN IP) which implements the functionalities specific to RZ/N1.

To add the driver to the vxWorks image, add the following component to the kernel configuration.

```
vxprj component add DRV_RZN1_SDHC_CTRL
```

Each SD/MMC Host Controller device should be bound to a device tree node which requires below properties:

compatible

This parameter specifies the name of the SDHC controller driver. It should be "arasan,sdhci-8.9a".

reg

This parameter specifies the register base address and length of this module.

interrupts

This parameter specifies the interrupt number of this module.

interrupt-parent

This parameter specifies the offset of interrupt controller.

dma-mode

This parameter specifies the used DMA mode. Currently only 0 (SDMA) is supported.

pinmux-0

This parameter specifies the list of phandles that each pointing at a PinMux configuration node.

clocks

This parameter specifies the list of clocks that this module use.

Below is an example:

```
        sdio0: sdio@0x40100000
            {
            compatible = "arasan,sdhci-8.9a";
```

```
            status = "okay";
            reg = <0x40100000 0x1000>;
            interrupts = <101 0 0x104>;     @* 69 +32 RZN1_IRQ_SDIO0 *@
            interrupt-parent = <&gic>;
            dma-mode = <0>;                 @* 0 for SDMA *@
            pinmux-0 = <&sdio0_mux>;
            clock-names = "sdio0_clk";
            clocks = <&sdio0_clk>;
            };
```

Include files

vxBus.h vxbIntLib.h

See also

vxBus

52. VXBSATALIB

Name

vxbSataLib - SATA/ATA command Library

Layer

BDM_SATA

Routines

sataIdentify( ) - Send identify command to sata device **sataFeatureSet( )** - set sata device extend feature **sataSoftResetSet( )** - send soft reset command to sata device **sataDevDiagnostic( )** - send diagnostic command to sata device **sataSetMulSector( )** - set multiple sectors number to sata device **sataFlushCache( )** - flush sata device cache **sataAtapiTestReady( )** - send test ready command to ATAPI device **sataAtapiLoadDisc( )** - send start unit command to ATAPI device **sataAtapiReadCapacity( )** - read ATAPI device capacity **sataAtapiReadStatus( )** - read ATAPI device mechanism status **sataAtapiReadSense( )** - read ATAPI device sense code **sataBlkRead( )** - read block from sata device **sataBlkWrite( )** - write block to sata device **sataAtaNormalInit( )** - normal ATA init process for sata device **sataAtapiNormalInit( )** - Normal ATAPI init process to sata device **sataDevIdxAlloc( )** - allocate a global sata device index **sataDevIdxFree( )** - free a global sata device index **sataHostShow( )** - print SATA host related information **sataDeviceShow( )** - print SATA device related information

Description

This is the SATA/ATA library to implement necessary SATA/ATA commands for a SATA device which is compatible with ATA-ATAPI-6. ATA-ATAPI protocol used IO register to issue command. Such as Device register, Status register and so on. Different command need set different value to every register. The following is the identify command description.

|---------- |Register | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---------- |Features | na | |---------- |
Sec count| na | |---------- |LBA Low | na |
|---------- |LBA Mid | na | |---------- |
LBA High | na | |---------- |Device | obs | na | obs | Dev | na |
|---------- |Command | ECh |
|----------

SATA protocol transmit ATA command with Transport layer. The Transport layer simply constructs Frame Information Structures (FISes) for transmission and decomposes received Frame Information Structures. The following is one description of Host to Device FIS.

----------	0	Features	Command	c	R	R	R	PM port	FIS type(27h)
----------	1	Device	LBA High	LBA Mid	LBA Low
----------	2	Features(exp)	LBA High(exp)	LBA Mid(exp)	LBA Low(exp)
----------	3	Control	Reserved	Sec count(exp)	Sec count
|---------- | 4 | Reserved | Reserved | Reserved | Reserved |
|----------

Include files

none

Name

sataIdentify( ) - Send identify command to sata device

Layer

BDM_SATA

Synopsis

```
STATUS sataIdentify
    (
    SATA_DEVICE * device
    )
```

Description

This routine sends identify command to sata device.

Returns

OK or **ERROR**

Errno

N/A

See also

vxbSataLib

Name

sataFeatureSet( ) - set sata device extend feature

Layer

BDM_SATA

Synopsis

```
STATUS sataFeatureSet
    (
    SATA_DEVICE * device,
    UINT8         feature
    )
```

Description

This routine sets sata device extend feature.

Returns

OK or **ERROR**

Errno

N/A

See also

vxbSataLib

Name

sataSoftResetSet( ) - send soft reset command to sata device

Layer

BDM_SATA

Synopsis

```
void sataSoftResetSet
    (
    SATA_DEVICE * device,
    BOOL          isRst
    )
```

Description

This routine sends soft reset command to sata device.

Returns

N/A

Errno

N/A

See also

vxbSataLib

Name

sataDevDiagnostic( ) - send diagnostic command to sata device

Layer

BDM_SATA

Synopsis

```
STATUS sataDevDiagnostic
    (
```

```
    SATA_DEVICE * device
    )
```

Description

This routine sends diagnostic command to sata device and gets diagnostic code from sata device.

Returns

OK or **ERROR**

Errno

N/A

See also

vxbSataLib

Name

sataSetMulSector( ) - set multiple sectors number to sata device

Layer

BDM_SATA

Synopsis

```
STATUS sataSetMulSector
    (
    SATA_DEVICE * device
    )
```

Description

This routine sets multiple sectors number to sata device.

Returns

OK or **ERROR**

Errno

N/A

See also

vxbSataLib

Name

sataFlushCache( ) - flush sata device cache

Layer

BDM_SATA

Synopsis

```
STATUS sataFlushCache
    (
    SATA_DEVICE * device
    )
```

Description

This routine flushes sata device cache.

Returns

OK or **ERROR**

Errno

N/A

See also

vxbSataLib

Name

sataAtapiTestReady( ) - send test ready command to ATAPI device

Layer

BDM_SATA

Synopsis

```
STATUS sataAtapiTestReady
    (
    SATA_DEVICE * device
    )
```

Description

This routine sends test ready command to ATAPI device.

Returns

OK or **ERROR**

Errno

N/A

See also

vxbSataLib

Name

sataAtapiLoadDisc( ) - send start unit command to ATAPI device

Layer

BDM_SATA

Synopsis

```
STATUS sataAtapiLoadDisc
    (
    SATA_DEVICE * device
    )
```

Description

This routine sends start unit command to ATAPI device.

Returns

OK or **ERROR**

Errno

N/A

See also

vxbSataLib

Name

sataAtapiReadCapacity( ) - read ATAPI device capacity

Layer

BDM_SATA

Synopsis

```
STATUS sataAtapiReadCapacity
    (
    SATA_DEVICE * device
    )
```

Description

This routine reads ATAPI device capacity.

Returns

OK or **ERROR**

Errno

N/A

See also

vxbSataLib

Name

sataAtapiReadStatus( ) - read ATAPI device mechanism status

Layer

BDM_SATA

Synopsis

```
STATUS sataAtapiReadStatus
    (
    SATA_DEVICE * device
    )
```

Description

This routine reads the first 8 bytes ATAPI device mechanism status.

Returns

OK or **ERROR**

Errno

N/A

See also

vxbSataLib

Name

sataAtapiReadSense( ) - read ATAPI device sense code

Layer

BDM_SATA

Synopsis

```
STATUS sataAtapiReadSense
    (
    SATA_DEVICE * device
    )
```

Description

This routine reads 18 bytes ATAPI device sense codes.

Returns

OK or **ERROR**

Errno

N/A

See also

vxbSataLib

Name

sataBlkRead( ) - read block from sata device

Layer

BDM_SATA

Synopsis

```
STATUS sataBlkRead
    (
    SATA_DEVICE * device,
    sector_t      blkNo,
    UINT32        numBlks,
    UINT8 *       pBuf
    )
```

Description

This routine reads block from sata device.

Returns

OK or **ERROR**

Errno

N/A

See also

vxbSataLib

Name

sataBlkWrite( ) - write block to sata device

Layer

BDM_SATA

Synopsis

```
STATUS sataBlkWrite
    (
    SATA_DEVICE * device,
    sector_t      blkNo,
    UINT32        numBlks,
    UINT8 *       pBuf
    )
```

Description

This routine writes block to sata device.

Returns

OK or **ERROR**

Errno

N/A

See also

vxbSataLib

Name

sataAtaNormalInit( ) - normal ATA init process for sata device

Layer

BDM_SATA

Synopsis

```
STATUS sataAtaNormalInit
    (
    SATA_DEVICE * device
    )
```

Description

This routine handles the normal ATA init process for sata device.

Returns

OK or **ERROR**

Errno

N/A

See also

vxbSataLib

Name

sataAtapiNormalInit( ) - Normal ATAPI init process to sata device

Layer

BDM_SATA

Synopsis

```
STATUS sataAtapiNormalInit
    (
    SATA_DEVICE * device
    )
```

Description

This routine handles normal ATAPI init process to sata device.

Returns

OK or **ERROR**

Errno

N/A

See also

vxbSataLib

Name

sataDevIdxAlloc( ) - allocate a global sata device index

Layer

BDM_SATA

Synopsis

```
STATUS sataDevIdxAlloc
    (
    SATA_DEVICE * device
    )
```

Description

This routine allocates a global sata device index.

Returns

OK or **ERROR**

Errno

N/A

See also

vxbSataLib

Name

sataDevIdxFree( ) - free a global sata device index

Layer

BDM_SATA

Synopsis

```
void sataDevIdxFree
    (
```

```
    SATA_DEVICE * device
    )
```

Description

This routine frees a global sata device index.

Returns

N/A

Errno

N/A

See also

vxbSataLib

Name

sataHostShow( ) - print SATA host related information

Layer

BDM_SATA

Synopsis

```
void sataHostShow
    (
    VXB_DEV_HANDLE pDev
    )
```

Description

This routine prints SATA host related information.

Returns

N/A

Errno

N/A

See also

vxbSataLib

Name

sataDeviceShow( ) - print SATA device related information

Layer

BDM_SATA

Synopsis

```
void sataDeviceShow
    (
    VXB_DEV_HANDLE pDev,
    UINT32         portNum
    )
```

Description

This routine prints SATA device related information.

Returns

N/A

Errno

N/A

See also

vxbSataLib

53. VXBSATAXBD

Name

vxbSataXbd - Generic SATA block driver

Layer

BDM_SATA

Routines

sataXbdDevCreate( ) - create an XBD device for SATA **sataXbdDevDelete( )** - delete an XBD device for SATA
sataDeviceShowByName( ) - print SATA device related information by IOS name

Description

This is the generic XBD block driver used by SATA library to interface with upper level file systems.

Ioctl functions

The SATA driver supports the following XBD **ioctl( )** functions. The functions listed are defined in the header **xbd.h**. User can use **xbdIoctl( )** or **ioctl( )** to request the functions.

XBD_GETGEOMETRY

Get device geometry for SATA device: block size, total blocks, etc.

XBD_SYNC

Send a flush command to hardware to flush the cache to media in hardware.

XBD_SOFT_EJECT

Trigger a XBD ERF remove event, and soft insert event. It may cause XBD eject all mounted partitions.

XBD_HARD_EJECT

Trigger a XBD ERF remove event, and hard insert event. It may cause XBD remount all partitions.

XBD_GETBASENAME

Return XBD base name.

XBD_TEST

Return state if device is attached. **OK** for attached, **ERROR** for not attached.

XBD_STACK_COMPLETE

To notify file system monitor that all file system are mounted in this device, and trigger a XBD ERF Instantiated event.

XBD_GETINFO

Return XBD properties by set value to **XBD_INFO**: **XBD_INFO_READONLY** for read only device, **XBD_INFO_DISCARD** for support discard (trim) command.

XBD_GET_DEVTYPE

Return **XBD_HDD**, always.

FIODISCARDGET

Set **TRUE** or **FALSE** value to arg in **ioctl( )** to indicate if device support discard (trim) feature.

FIODISCARD

Do discard for a range of sector.

XBD_SMART_ENABLE

Enable SMART (Self-Monitoring Analysis and Reporting Technology) for device.

XBD_SMART_DISABLE

Disable SMART (Self-Monitoring Analysis and Reporting Technology) for device.

XBD_SMART_SAVEATTR

Save SMART (Self-Monitoring Analysis and Reporting Technology) for device

XBD_SMART_ISENABLED

Check if SMART (Self-Monitoring Analysis and Reporting Technology) is enabled on a device, by setting **TRUE** or **FALSE** to arg.

XBD_SMART_AUTOSAVEENABLE

Enable auto-saving of SMART attributes on a drive

XBD_SMART_AUTOSAVEDISABLE

Disable auto-saving of SMART attributes on a drive

XBD_SMART_OFFLINEDIAG

Start a SMART offline diagnosis test on a drive, by setting sub command in arg with UINT8 type.

XBD_SMART_RETURNSTATUS

Get the current SMART status of a drive

XBD_SMART_READDATA

Read SMART data from a drive, buffer is arg, its size must be 512 bytes.

XBD_SMART_READTHRESHO

Read SMART thresholds from a drive, buffer is arg, its size must be 512 bytes.

XBD_SATA_PARMSGET

Gets the ATA identify data from an AHCI drive, this command do not swap data reading from hardware.

Include files

none

Name

sataXbdDevCreate( ) - create an XBD device for SATA

Layer

BDM_SATA

Synopsis

```
device_t sataXbdDevCreate
    (
    void * sataDev,
    char * devName
    )
```

Description

This routine creates an XBD device for SATA.

Return

XBD device id

Returns

Not Available

Errno

N/A

See also

vxbSataXbd

Name

sataXbdDevDelete( ) - delete an XBD device for SATA

Layer

BDM_SATA

Synopsis

```
STATUS sataXbdDevDelete
    (
    void * sataDev
    )
```

Description

This routine deletes an XBD device for SATA.

Return

OK or **ERROR**

Returns

Not Available

Errno

N/A

See also

vxbSataXbd

Name

sataDeviceShowByName( ) - print SATA device related information by IOS name

Layer

BDM_SATA

Synopsis

```
STATUS sataDeviceShowByName
    (
    char * pDevName
    )
```

Description

This routine prints SATA device related information by IOS name.

Returns

OK, or **ERROR** if the parameters are invalid.

Errno

N/A

See also

vxbSataXbd

54. VXBSDHCCTRL

Name

vxbSdhcCtrl - SDHC host controller common driver library

Layer

BDM_SDMMC

Routines

No user-callable routines

Description

This is the VxBus-compliant Secure Digital Host Controller (SDHC) driver library which implements the functionality specified in "SD Specifications Part A2 SD Host Controller Simplified Specification Version 2.00".

The SDHC provides an interface between the host controller and the SD/SDIO target device.

This file is not applicable to a particular VxBus driver. Rather, it implements a number of functions for a standard SDHC controller. However, many SDHC controllers follow the standard implementation. Therefore, when a new SDHC controller driver is developed, only those functions that differ from the standard need to be implemented. Otherwise, the standard functions in this file are called as needed.

This driver is bound to a device tree, and the device tree node must specify the following properties:

#address-cells

This property specifies the number of register address parameters for a device node if the parent of the device node does not specify the value (for example, the device node for memory). The value of this property must be 1.

#size-cells

This property specifies the number of register size parameters for a device node if the parent of the device node does not specify the value (for example, the device node for memory). The value of this property must be 1.

compatible

This property specifies the name of the SDHC driver. For the LS1043ARDB, it is "fsl,sdhc".

clocks

This parameter specifies the clock that this module uses.

reg

This property specifies the register base address and length of this module.

interrupt-parent

This property is used to define an interrupt parent. If it is not defined for a device, the interrupt parent is assumed to be the device tree parent for the device.

big-endian

This property specifies the endianness of this module. Set the value to **0** for little endian. For all other values, big endian is used. If this property is not specified, little endian is used.

little-endian

This property is used for standard dts, which specifies the endianness of this module. Specify this property if the controller is little endian. If this property is not specified, big endian is used.

voltage-ranges

This property is used for standard dts, which specifies controller supporting voltage ranges.

sdhci,auto-cmd12

This property is used for standard dts, which specifies controller supporting atuo CMD12.

status

The initial status of the controller.

dma-mode

This property specifies the mode of data transfer, 0 means using SDMA, and 1 means using PIO mode. If this property is not specified, SDMA is used.

disable-wp

When set no physical WP line is present. This property should only be specified when the controller has a dedicated write-protect detection logic. If a GPIO is always used for the write-protect detection logic it is sufficient to not specify wp-gpios property in the absence of a WP line.

cd-gpios

This property is used for standard dts, which specifies GPIO used for SD card detection. Currently only first two arguments are used.

wp-gpios

This property is used for standard dts, which specifies GPIO used for SD card write proctection. Currently only first two arguments are used.

An example of a device node as follows:

Legacy DTS:

```
esdhc: esdhc@1560000
    {
    #address-cells = <1>;
    #size-cells = <1>;
    compatible = "fsl,sdhc";
    clocks = <&esdhc_en>;
    reg = <0 0x1560000 0 0x1000>;
    interrupt-parent = <&intc>;
    big-endian = <1>;
    cd-gpios = <&gpio5 22 0 0 0>;
    wp-gpios = <&gpio5 21 0 0 0>;
    status = "disabled";
    };
```

Standard DTS:

```
esdhc: esdhc@1560000
    {
    #address-cells = <1>;
```

```
    #size-cells = <1>;
    compatible = "fsl,sdhc";
    clocks = <&esdhc_en>;
    reg = <0 0x1560000 0 0x1000>;
    interrupt-parent = <&intc>;
    voltage-ranges = <1800 1800 3300 3300>;
    sdhci,auto-cmd12;
    little-endian;
    status = "disabled";
    };
```

Include files

See also

vxBus, xbd, "SD Specifications Part A1 Physical Layer Simplified Specification Version, 2.00", "SD Specifications Part A2 SD Host Controller Simplified Specification, Version 2.00"

55. VXBSPISDHI

Name

vxbSpiSdhi - vxBus SPI SD/MMC interface driver

Layer

BDM_SDMMC

Routines

No user-callable routines

Description

This is the Vxbus compliant SPI SD/MMC host interface driver.

To add the driver to the vxWorks image, add the following component to the kernel configuration.

```
vxprj component add DRV_SPI_SDMMC
```

This driver is bound to the device tree, and the device tree node must specify the following parameters:

compatible

This parameter specifies the name of the MMC controller driver. It must be "mmc-spi-slot".

reg

This parameter specifies the register base address and length of this module.

voltage-ranges

This parameter specifies the list of MMC interface voltages supported by the hardware. It must be at least one of "1V8" and "3V3".

gpios

This parameter specifies the card detect GPIO pin index and write protect pin index.

An example of a device node is shown below:

```
    mmc@0
        {
        compatible = "mmc-spi-slot";
        reg = <0>;
        spi-max-frequency = <20000000>;
        voltage-ranges = <3300 3300>;
        disable-wp;
        gpios = <&gpio 11 1>;
        };
```

Include files

none

See also

vxBus, VxWorks 7 BSP and Driver Guide, SD Specifications Part 1 Physical Layer Specifiction

56. VXBTIMMCHSCTRL

Name

vxbTiMmchsCtrl - TI MMCHS driver

Layer

BDM_SDMMC

Routines

No user-callable routines

Description

This is the vxbus compliant MMC/SD/SDIO Host Controller (MMCHS) driver which implements the functionality specific to the TI SOCs.

The MMCHS provides an interface between the host system and SD/MMC memory cards.

To add the driver to the vxWorks image, add the following component to the kernel configuration.

```
vxprj component add DRV_TI_MMCHS_CTRL
```

Vxworks-7 mode

This driver is bound to device tree, and the device tree node must specify below parameters:

compatible

This parameter specifies the name of the MMCHS controller driver. It must be "ti,mmchs" or "ti,emmchs".

reg

This parameter specifies the register base address and length of this module.

interrupts

This parameter specifies the interrupt number of this module.

interrupt-parent

This parameter specifies the offset of interrupt controller.

pinmux-0

This parameter specifies the list of phandles that each pointing at a PinMux configuration node.

clocks

This parameter specifies the list of clocks that this module use.

clock-names

This parameter specifies the name of every clock in clocks list.

dma-mode

This parameter specifies the mode of data transfer, 0 means using SDMA, and 1 means using PIO mode.

cd-gpios

This parameter specifies the card detect GPIO pin index. Only the first two arguments are used.

wp-gpios

This parameter specifies the write protect GPIO pin index. Only the first two arguments are used. Using 0xffff means the target device is always writeable.

debounce-time

This parameter specifies the debounce time for the card detect GPIO pin.

userUhsMode

This parameter allows you to specify the UHS mode to use: 0 (or unassigned) - the SD stack automatically selects the mode 1 - UHS is disabled. The SD card works in high speed or default speed 2 - SDR50 mode is selected, if supported

An example of device node is show below:

```
sdmmc0@48060000
    {
    #address-cells = <1>;
    #size-cells = <0>;
    compatible = "ti,mmchs";
    reg = <0x48060000 0x1000>;
    interrupts = <64>;
    interrupt-parent = <&intc>;
    pinmux-0 = <&mmc0_pads>;
    clocks = <&clk 143>,<&clk 144>,<&clk 145>;
    clock-names = "mmc_clk","ick","fck";
    dma-mode = <0>;
    cd-gpios = <&gpio6 27 0 0 0>;
    wp-gpios = <&gpio6 28 0 0 0>;
    debounce-time = <7936>;
    };
```

Include files

none

See also

vxBus, "OMAP35x_TRM", "AM335x_TRM"

57. VXBZYNQSDHCCTRL

Name

vxbZynqSdhcCtrl - Zynq SDHC host controller driver

Layer

BDM_SDMMC

Routines

No user-callable routines

Description

This is the vxbus compliant Secure Digital Host Controller (SDHC) driver which implements the functionality specified in "SD Specifications Part A2 SD Host Controller Simplified Specification Version 2.00"

The SDHC provides an interface between the host controller and SD/MMC memory cards.

To add the driver to the vxWorks image, add the following component to the kernel configuration.

```
vxprj component add DRV_ZYNQ_SDHC_CTRL
```

Vxwork-7 mode

This driver is bound to device tree, and the device tree node must specify below parameters:

compatible

This parameter specifies the name of the SDHC controller driver. It must be "zynq,sdhc".

reg

This parameter specifies the register base address and length of this module.

clocks

This parameter specifies parent clocks used for SD controller. SD driver will use the clocks listed here to retrieve SD controller reference clock frequency. If clock-frequency is assigned, this parameter will be ignored.

clock-frequency

This parameter specifies the SD controller reference clock frequency. SD driver will use it to calculate the clock on the **SD_CLK** pin.

interrupts

This parameter specifies the interrupt number of this module.

interrupt-parent

This parameter is available to define an interrupt parent. if it is missing from a device, it's interrupt parent is assumed to be its device tree parent.

pinmux-0

This parameter specifies the PinMux configuration used for this module.

disable-protect-check

This parameter specifies if we disable write protect or not. If 0 or not specified, we will check write protect pin. If other value, we will not check write protect pin and assume device is writable.

embedded

This parameter specifies which type of device is connected to the host controller. 1 - eMMC device. 0 - SD/MMC slot.

bus-width

This parameter specifies the maximum bus-width (4bit or 8bit) the host controller supported. This parameter is used for MMC/eMMC devices and doesn't influence SD cards. If not specified, use 4 bit mode by default.

max-dev-clock-frequency

This parameter specifies the maximum clock frequency on the **SD_CLK** pin. This parameter is only used for MMC/eMMC devices. If 0 or not specified, the driver will not check this limitation.

user-set-clock-frequency

This parameter specifies the clock frequency that the user setting for their MMC/eMMC card working. If 0 or not specified, the driver will set this parameter to 52MHz.

card-status-check

This parameter specifies if we need check device status after reading data(blocks) from MMC/eMMC devices. If 0 or not specified, do not check it. 1 will check it.

An example of device node is show below:

```
sdhc0: sdhc@ff160000
    {
    compatible = "zynq,sdhc";
    reg = <0x0 0xff160000 0x0 0x1000>;
    clocks = <&sdio0_ref_div_clk>;
    interrupts = <80 0 4>;
    interrupt-parent = <&intc>;
    pinmux-0 = <&sdio0_mux>;
    embedded = <0>;
    bus-width = <4>;
    max-dev-clock-frequency = <52000000>;
    user-set-clock-frequency = <25000000>;
    card-status-check = <0>;
    };
```

Include files

none

See also

vxBus, "ZYNQ 7000 Reference Manual", "SD Specifications Part A1 Physical Layer Simplified Specification Version 2.00", "SD Specifications Part A2 SD Host Controller Simplified Specification Version 2.00"

58. XBD

Name

xbd - Extended Block Device Library

Layer

BDM

Routines

xbdInit( ) - initialize the XBD library **xbdAttach( )** - attach an XBD device **xbdAttachExt( )** - allocate and init xbd_ext for an XBD device **xbdDetach( )** - detach an XBD device **xbdIoctl( )** - XBD device ioctl routine **xbdStrategy( )** - XBD strategy routine **xbdStrategyGeneric( )** - generic XBD strategy routine for devices **xbdGetNextBioGeneric( )** - get the next bio in the bio chain **xbdSrvTaskGeneric( )** - task level routine to handle read and write operation **xbdGetBioSched( )** - get what bio scheduler is used for an XBD device **xbdSetBioSched( )** - set bio scheduler for an XBD device **xbdDump( )** - XBD dump routine **xbdSize( )** - retrieve the total number of bytes **xbdNBlocks( )** - retrieve the total number of blocks **xbdBlockSize( )** - retrieve the block size **xbdMaxBlocks( )** - retrieve the max number of blocks supported by device **xbdMaxRequests( )** - retrieve the max number of requests supported by device. **xbdRoot( )** - retrieve the root xbd of the device

Description

This module implements the extended block device.

Include files

drv/xbd/xbd.h

See also

VxWorks 7 File Systems and I/O Programmer's Guide Block Devices Overview

Name

xbdInit( ) - initialize the XBD library

Layer

BDM

Synopsis

```
STATUS xbdInit (void)
```

Description

This routine initializes the XBD libarary.

Returns

OK upon success, **ERROR** otherwise

Errno

Not Available

See also

xbd

Name

xbdAttach( ) - attach an XBD device

Layer

BDM

Synopsis

```
int xbdAttach
    (
    XBD *               xbd,
    struct xbd_funcs *  funcs,
    const char *        name,
    unsigned            blocksize,
    sector_t            nblocks,
    device_t *          result
    )
```

Description

none

Returns

0 upon success, non-zero otherwise

Errno

Not Available

See also

xbd

Name

xbdAttachExt( ) - allocate and init xbd_ext for an XBD device

Layer

BDM

Synopsis

```
int xbdAttachExt
    (
    XBD                         *xbd,
    int                         sched,
    SEM_ID                      biomutex,
    SEM_ID                      biordy,
    struct bio                  **bioqh,
    struct bio                  **bioqt,
    void        (*f_biodo) (XBD *xbd, struct bio *bio),
    int         (*f_rmvtst) (XBD *xbd, struct bio *bio)
    )
```

Description

This routine needs to be called before driver's xbd service task spawning if generic xbd service task is to be used

Returns

0 upon success, non-zero otherwise

Errno

Not Available

See also

xbd

Name

xbdDetach( ) - detach an XBD device

Layer

BDM

Synopsis

```
void xbdDetach
    (
    XBD * xbd   /* pointer to XBD device to detach */
    )
```

Description

none

Returns

N/A

Errno

Not Available

See also

xbd

Name

xbdIoctl( ) - XBD device ioctl routine

Layer

BDM

Synopsis

```
int xbdIoctl
    (
    device_t d,
    int      cmd,
    void *   arg
    )
```

Description

none

Returns

varies

Errno

Not Available

See also

xbd

Name

xbdStrategy( ) - XBD strategy routine

Layer

BDM

Synopsis

```
int xbdStrategy
    (
    device_t     d,
    struct bio * bio
    )
```

Description

none

Returns

0 upon success, error code otherwise

Errno

Not Available

See also

xbd

Name

xbdStrategyGeneric( ) - generic XBD strategy routine for devices

Layer

BDM

Synopsis

```
int xbdStrategyGeneric
    (
    XBD        *xbd,
    struct bio *bio
    )
```

Description

none

Returns

0 upon success, error code otherwise

Errno

Not Available

See also

xbd

Name

xbdGetNextBioGeneric( ) - get the next bio in the bio chain

Layer

BDM

Synopsis

```
struct bio * xbdGetNextBioGeneric
    (
    XBD *xbd
    )
```

Description

This routine is used, combined with xbdStrategyGeneric, to get the next bio in the bio chain for an XBD device.

Returns

pointer to the next bio

Errno

none

See also

xbd

Name

xbdSrvTaskGeneric( ) - task level routine to handle read and write operation

Layer

BDM

Synopsis

```
void xbdSrvTaskGeneric
    (
    XBD *xbd
    )
```

Description

This routine waits on a semaphore from strategy routine. This task is spawned when the device is attached to the system and deleted after the device is detached and all the resources for this device are freed.

Return

N/A

Returns

Not Available

Errno

none

See also

xbd

Name

xbdGetBioSched( ) - get what bio scheduler is used for an XBD device

Layer

BDM

Synopsis

```
int xbdGetBioSched
    (
    XBD *xbd,
    int *retarg
    )
```

Description

This routine is usually called in dev-ioctl routine to handle user's request like ioctl(devfd, **XBD_GET_BIOSCHED**, &iRetArg)

Returns

0 upon success, error code otherwise

Errno

Not Available

See also

xbd

Name

xbdSetBioSched( ) - set bio scheduler for an XBD device

Layer

BDM

Synopsis

```
int xbdSetBioSched
    (
    XBD *xbd,
    int sched
    )
```

Description

This routine is usually called in dev-ioctl routine to handle user's request like ioctl(devfd, **XBD_SET_BIOSCHED**, **BIO_SCHED_ELEVATOR**)

Returns

0 upon success, error code otherwise

Errno

Not Available

See also

xbd

Name

xbdDump( ) - XBD dump routine

Layer

BDM

Synopsis

```
int xbdDump
    (
    device_t d,
    sector_t pos,
    void *   data,
    size_t   size
    )
```

Description

none

Returns

0 on success, error code otherwise

Errno

Not Available

See also

xbd

Name

xbdSize( ) - retrieve the total number of bytes

Layer

BDM

Synopsis

```
int xbdSize
    (
    device_t    d,
    long long * result
    )
```

Description

This routine retrieves the total number of bytes on the backing media.

Returns

0 on success, error code otherwise

Errno

Not Available

See also

xbd

Name

xbdNBlocks( ) - retrieve the total number of blocks

Layer

BDM

Synopsis

```
int xbdNBlocks
    (
    device_t   d,
    sector_t * result
    )
```

Description

none

Returns

0 on success, error code otherwise

Errno

Not Available

See also

xbd

Name

xbdBlockSize( ) - retrieve the block size

Layer

BDM

Synopsis

```
int xbdBlockSize
    (
    device_t   d,
    unsigned * result
    )
```

Description

none

Returns

0 on success, error code otherwise

Errno

Not Available

See also

xbd

Name

xbdMaxBlocks( ) - retrieve the max number of blocks supported by device

Layer

BDM

Synopsis

```
int xbdMaxBlocks
    (
    device_t   d,
    unsigned * result
    )
```

Description

This function retrieves the max number of blocks supported by device. The max number of blocks is stored in xbd_max_blocks field in XBD structure. Warning: This value may not be set correctly if driver does not initialize it.

Returns

0 on success, error code otherwise

Errno

Not Available

See also

xbd

Name

xbdMaxRequests( ) - retrieve the max number of requests supported by device.

Layer

BDM

Synopsis

```
int xbdMaxRequests
    (
    device_t   d,
    unsigned * result
    )
```

Description

This function retrieves the max number of requests supported by device. The max number of requests is stored in xbd_max_reqs field in XBD structure. Warning: This value may not be set correctly if driver does not initialize it.

Returns

0 on success, error code otherwise

Errno

Not Available

See also

xbd

Name

xbdRoot( ) - retrieve the root xbd of the device

Layer

BDM

Synopsis

```
struct xbd * xbdRoot
    (
    device_t d
    )
```

Description

none

Returns

pointer to the root xbd of the device

Errno

Not Available

See also

xbd

59. XBDBLKDEV

Name

xbdBlkDev - XBD / **BLK_DEV** Converter

Layer

FS

Routines

xbdBlkDevLibInit( ) - initialize the XBD block device wrapper **xbdBlkDevCreate( )** - create an XBD block device wrapper **xbdBlkDevDelete( )** - deletes an XBD block device wrapper **xbdBlkDevCreateSync( )** - synchronously create an XBD block device wrapper

Description

This library contains routines for wrapping an XBD around a **BLK_DEV**.

Include files

xbdBlkDev.h

Name

xbdBlkDevLibInit( ) - initialize the XBD block device wrapper

Layer

FS

Synopsis

```
STATUS xbdBlkDevLibInit
    (
    int  xbdServiceTskPri,
    int  xbdServiceTskStkSz,
    BOOL directMode
    )
```

Description

This routine initializes the XBD block device wrapper.

Returns

OK

Errno

N/A

See also

xbdBlkDev

Name

xbdBlkDevCreate( ) - create an XBD block device wrapper

Layer

FS

Synopsis

```
device_t xbdBlkDevCreate
    (
    BLK_DEV *    bd,    /* pointer to block device */
    const char * name   /* pointer to device name  */
    )
```

Description

This routine creates an XBD block device wrapper.

Returns

a device identifier upon success, or NULLDEV otherwise

Errno

EINVAL

See also

xbdBlkDev

Name

xbdBlkDevDelete( ) - deletes an XBD block device wrapper

Layer

FS

Synopsis

```
STATUS xbdBlkDevDelete
    (
    device_t   dev,   /* device_t returned from xbdBlkDevCreate */
    BLK_DEV ** ppbd   /* pointer to block device pointer */
    )
```

Description

This routine deletes or destroys an XBD block device wrapper.

The d parameter specifies the XBD block wrapper to delete. This should be the same value that was returned from **xbdBlkDevCreate( )**

The ppbd parameter is an out parameter that can be used to return the block device pointer used in **xbdBlkDevCreate( )**. If specified as **NULL** no attempt to return the block device pointer is attempted.

Returns

a device identifier upon success, or NULLDEV otherwise

Errno

See also

xbdBlkDev

Name

xbdBlkDevCreateSync( ) - synchronously create an XBD block device wrapper

Layer

FS

Synopsis

```
device_t xbdBlkDevCreateSync
    (
    BLK_DEV *    bd,    /* pointer to block device */
    const char * name   /* pointer to device name  */
    )
```

Description

This routine creates an XBD block device wrapper. It returns after the entire XBD stack has been created/initialized.

Returns

a device identifier upon success, or NULLDEV otherwise

Errno

See also

xbdBlkDev

60. XBDRAMDISK

Name

xbdRamDisk - XBD Ramdisk Implementation

Layer

FS

Routines

xbdRamDiskDevCreateEx( ) - create an XBD ram disk with a buffer **xbdRamDiskDevCreate( )** - create an XBD ram disk
xbdRamDiskDevDelete( ) - XBD Ram Disk Deletion routine

Description

This library implements an XBD ram disk.

Include files

xbdRamDisk.h

See also

VxWorks 7 File Systems and I/O Programmer's Guide RAM Drives, VxWorks 7 File Systems and I/O Programmer's Guide Creating and Formating a RAM Disk Volume

Name

xbdRamDiskDevCreateEx( ) - create an XBD ram disk with a buffer

Layer

FS

Synopsis

```
device_t xbdRamDiskDevCreateEx
    (
    unsigned     blockSize,   /* block size in bytes */
    uint64_t     totalSize,   /* disk size in bytes */
    BOOL         flag,        /* should the disk support partitions? */
    const char * name,        /* name of ram disk */
    char *       buffer       /* memory buffer for disk */
    )
```

Description

This routine creates an XBD ram disk with a buffer specific by buffer. The ram disk links into the file system monitor and eventing framework. If buffer is **NULL**, this routine will allocate buffer for ram disk.

blockSize should be equal or greater than 512, and the power of 2. buffer size should equal or greater than totalSize, or **NULL**.

If caller provide a buffer, this function do not erase any data. If caller do not provide a buffer, the buffer's first block allocated by this function will be clean to avoid data corruption.

Returns

The ID of the XBD created(device_t) or NULLDEV if the routine fails

Errno

Not Available

See also

xbdRamDisk

Name

xbdRamDiskDevCreate( ) - create an XBD ram disk

Layer

FS

Synopsis

```
device_t xbdRamDiskDevCreate
    (
    unsigned     blockSize,   /* block size in bytes */
    unsigned     totalSize,   /* disk size in bytes */
    BOOL         flag,        /* should the disk support partitions? */
    const char * name         /* name of ram disk */
    )
```

Description

This routine creates an XBD ram disk. The ram disk links into the file system monitor and eventing framework.

blockSize should be equal or greater than 512, and the power of 2.

Returns

The ID of the XBD created(device_t) or NULLDEV if the routine fails

Errno

Not Available

See also

xbdRamDisk, xbdRamDiskDevCreateEx

Name

xbdRamDiskDevDelete( ) - XBD Ram Disk Deletion routine

Layer

FS

Synopsis

```
STATUS xbdRamDiskDevDelete
    (
    device_t dev   /* device_t returned from xbdRamDiskDevCreate */
    )
```

Description

This routine deletes or destroy an instantion of an XBD ram disk. The ram disk to be deleted is identified by the supplied device_t. This value must have been previously returned from the corresponding xbdRamDiskDevCreate function. All resource associated with the ram disk are freed. Any file systems sitting on top of the ram disk are ejected.

Returns

OK on success or **ERROR** if the supplied device_t doesn't map to an existing and valid XBD.

Errno

Not Available

See also

xbdRamDisk